



A Siemens Business

UVM Framework

Student Workbook

© 2020 Mentor Graphics Corporation

All rights reserved.

This document contains information that is trade secret and proprietary to Mentor Graphics Corporation or its licensors and is subject to license terms. No part of this document may be photocopied, reproduced, translated, distributed, disclosed or provided to third parties without the prior written consent of Mentor Graphics.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/trademarks.

The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

End-User License Agreement: You can print a copy of the End-User License Agreement from:

www.mentor.com/eula.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: supportnet.mentor.com/

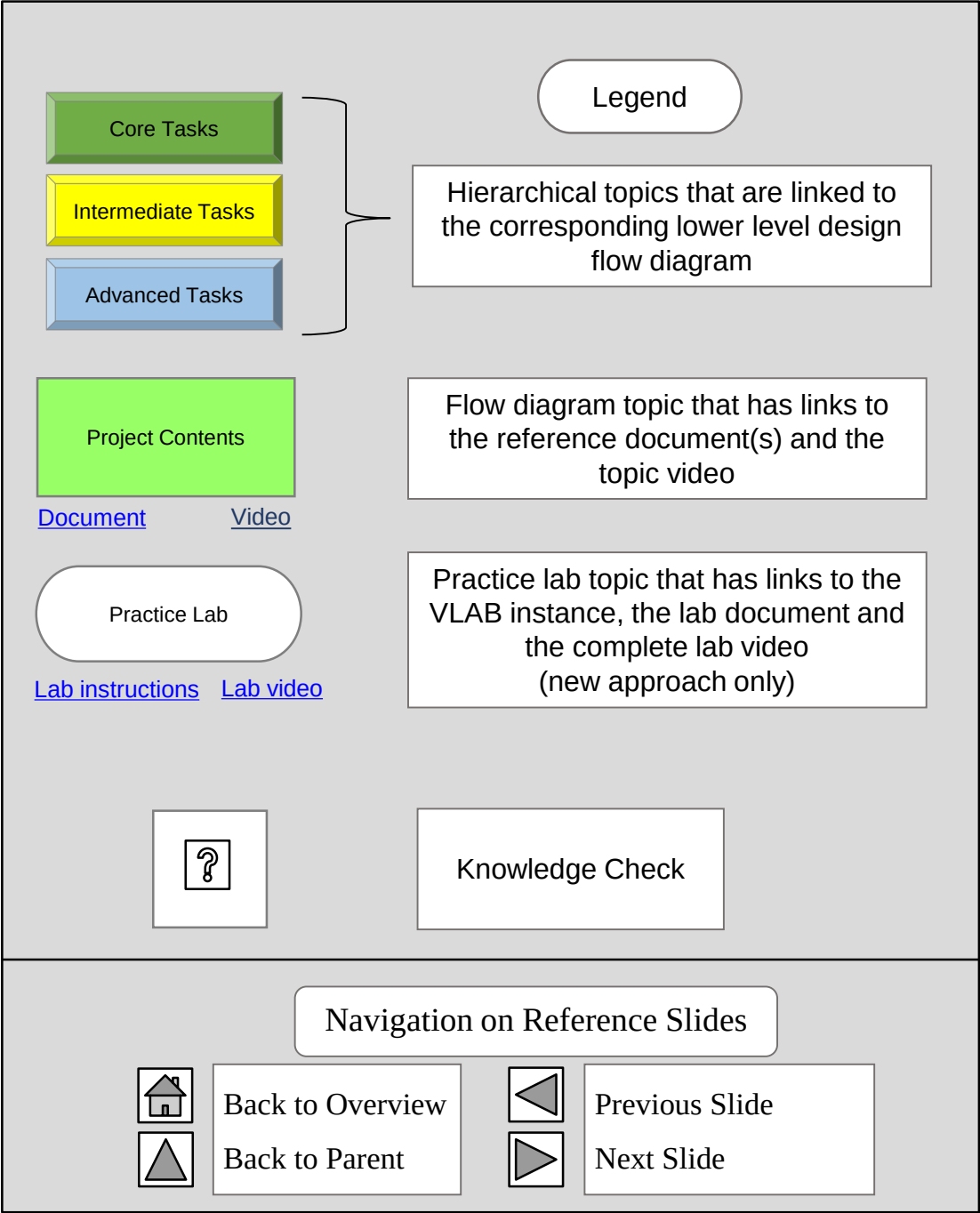
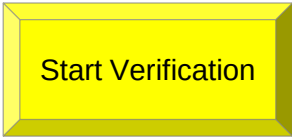
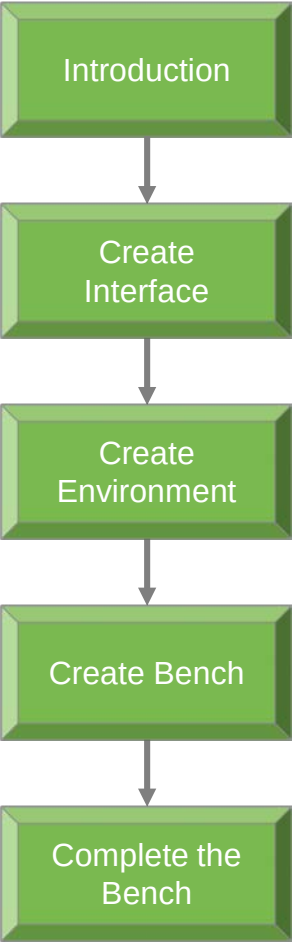
Send Feedback on Documentation: supportnet.mentor.com/doc_feedback_form

Part Number: 073856

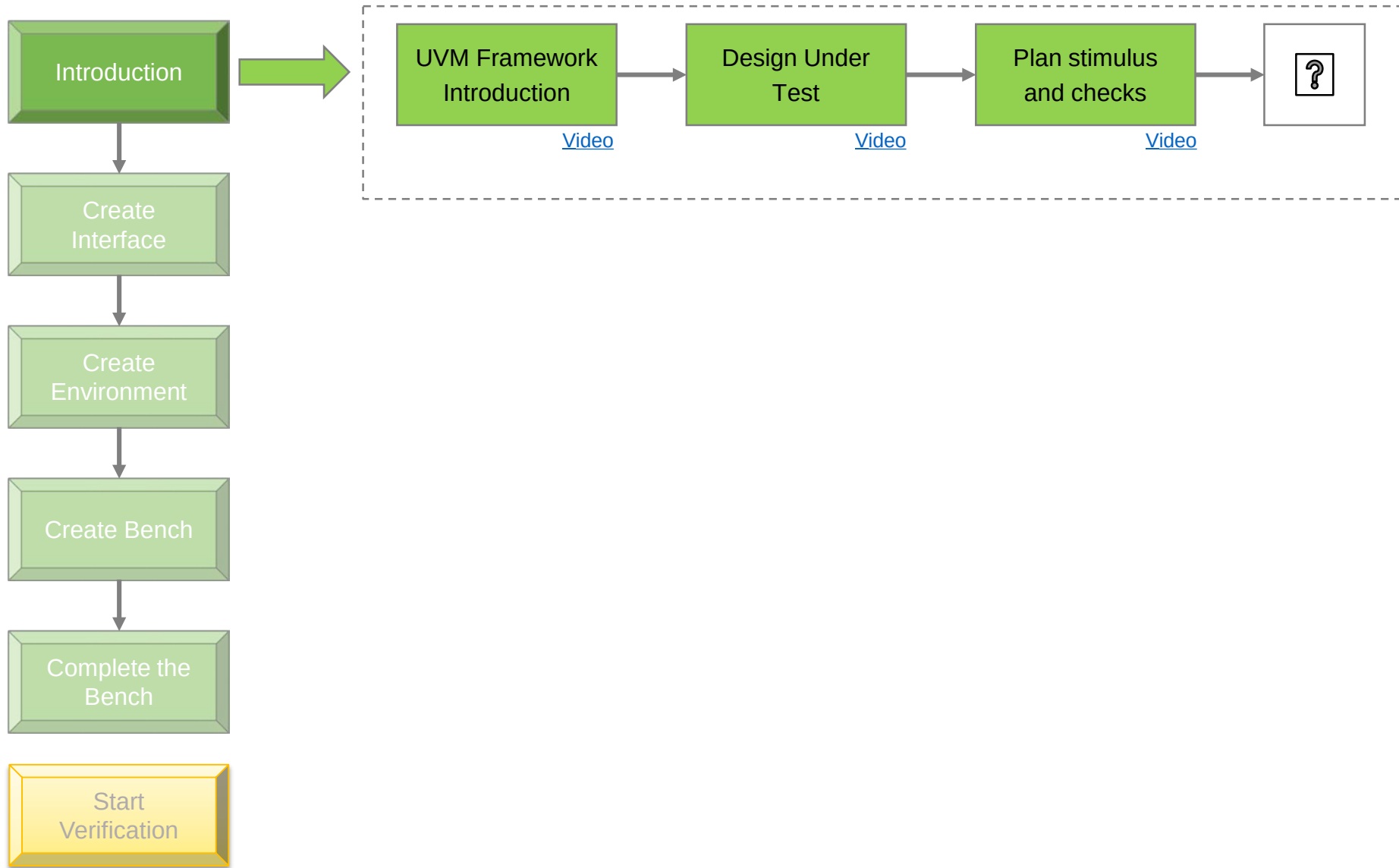
Table of Contents

1. [Introduction](#)
2. [Create the Protocol Interface](#)
3. [Create the Environment](#)
4. [Create the Bench](#)
5. [Complete the Bench](#)
6. [Start Verification](#)

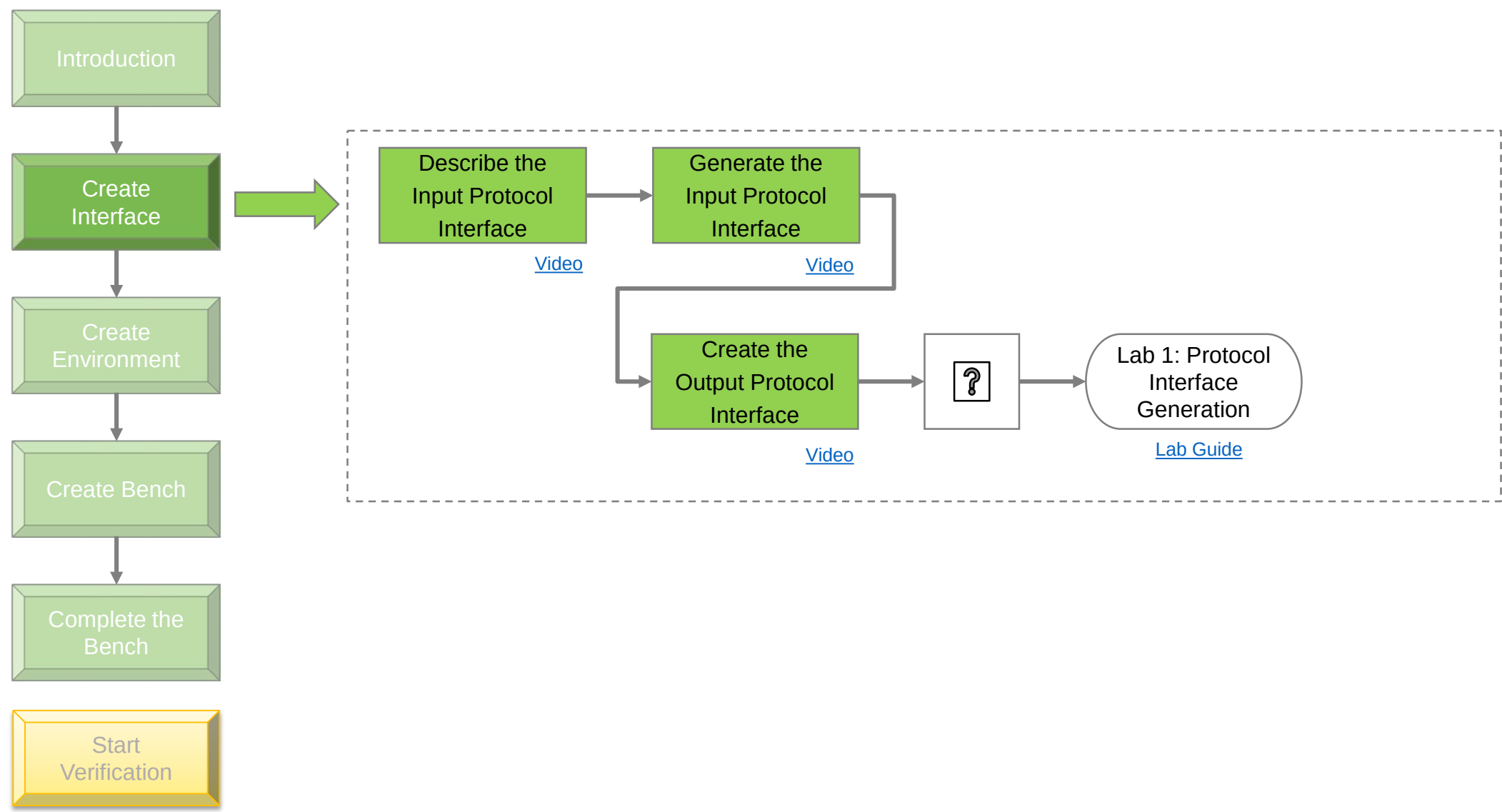
Course Flow / Legend



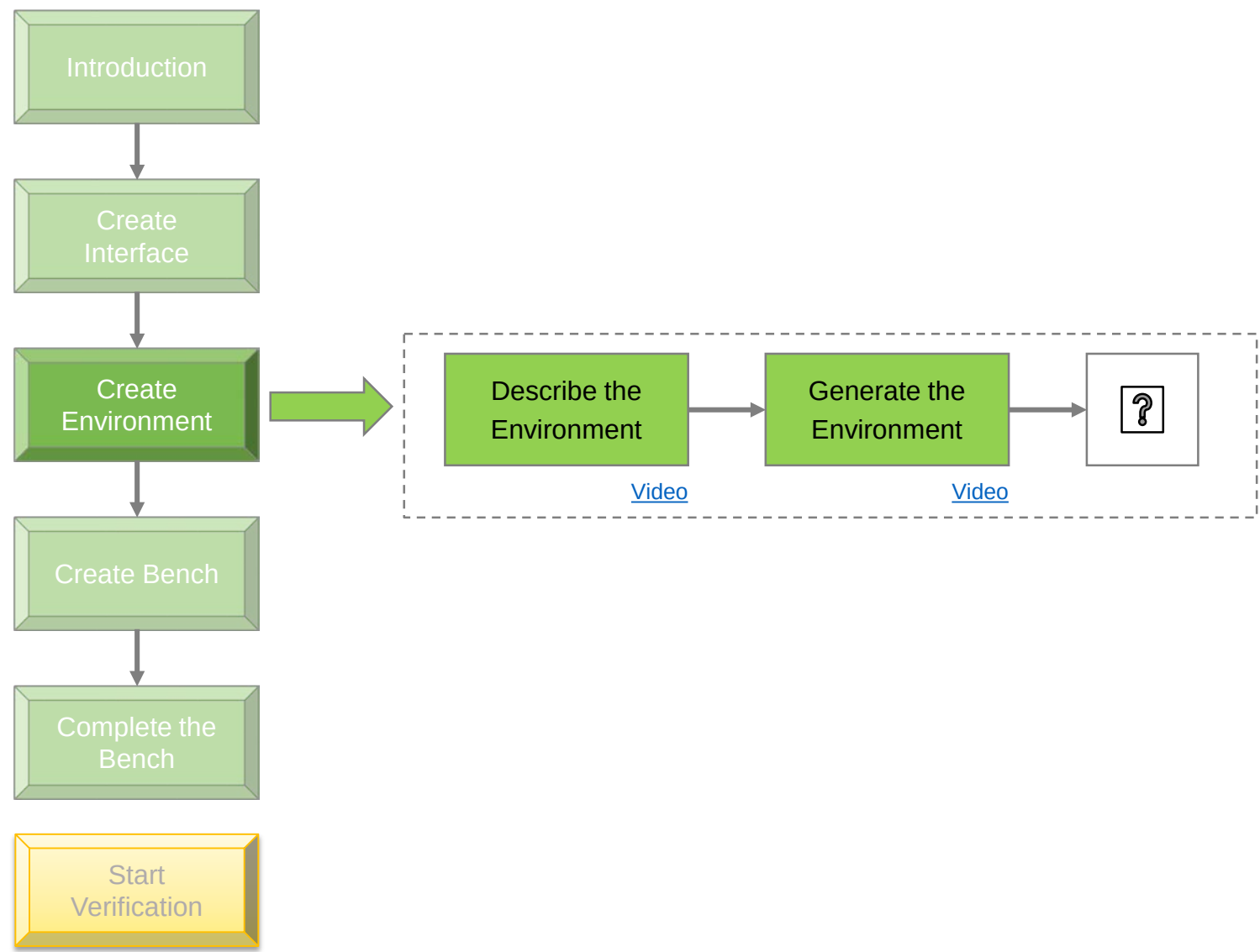
1: Introduction



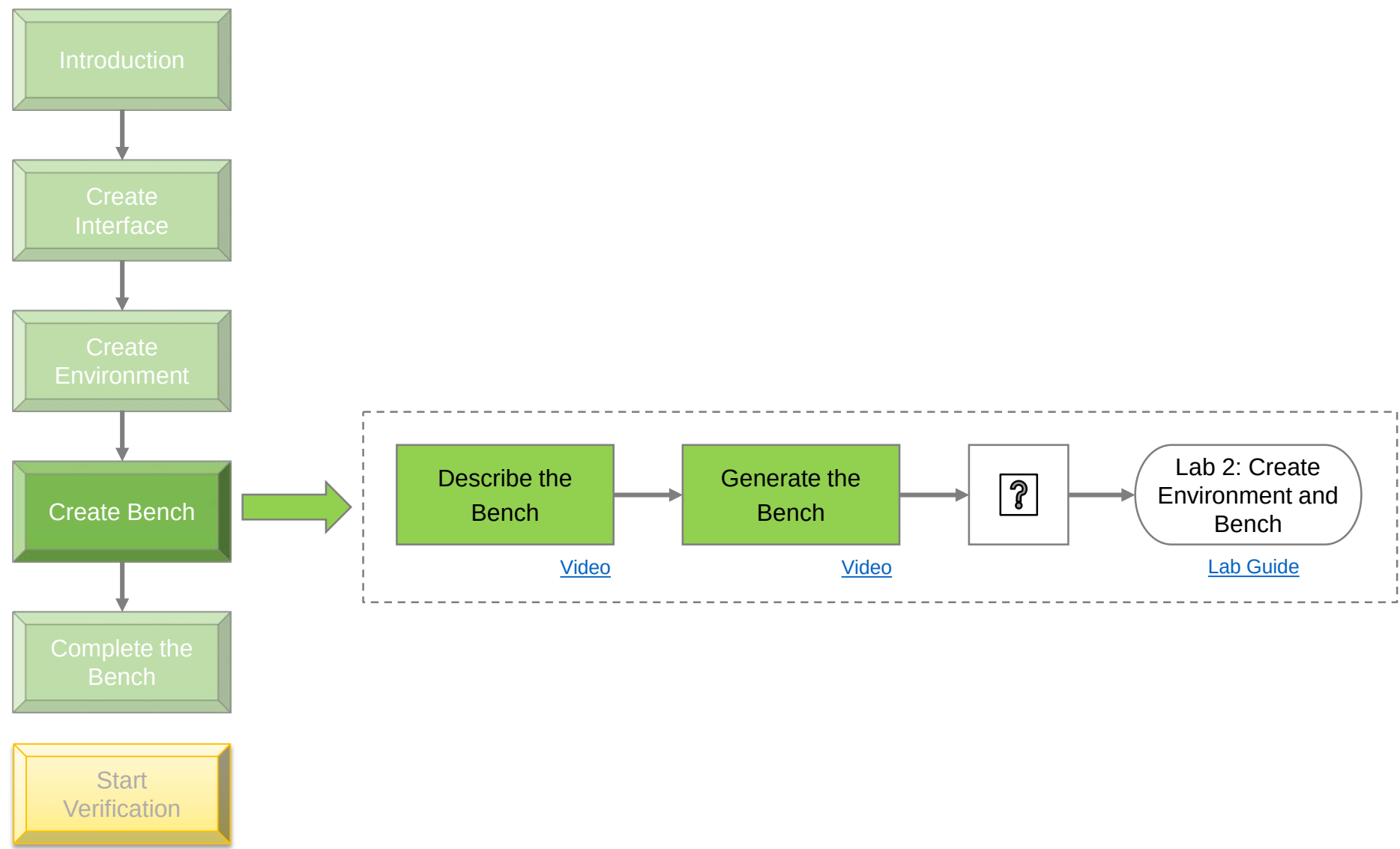
2: Create the Protocol Interface



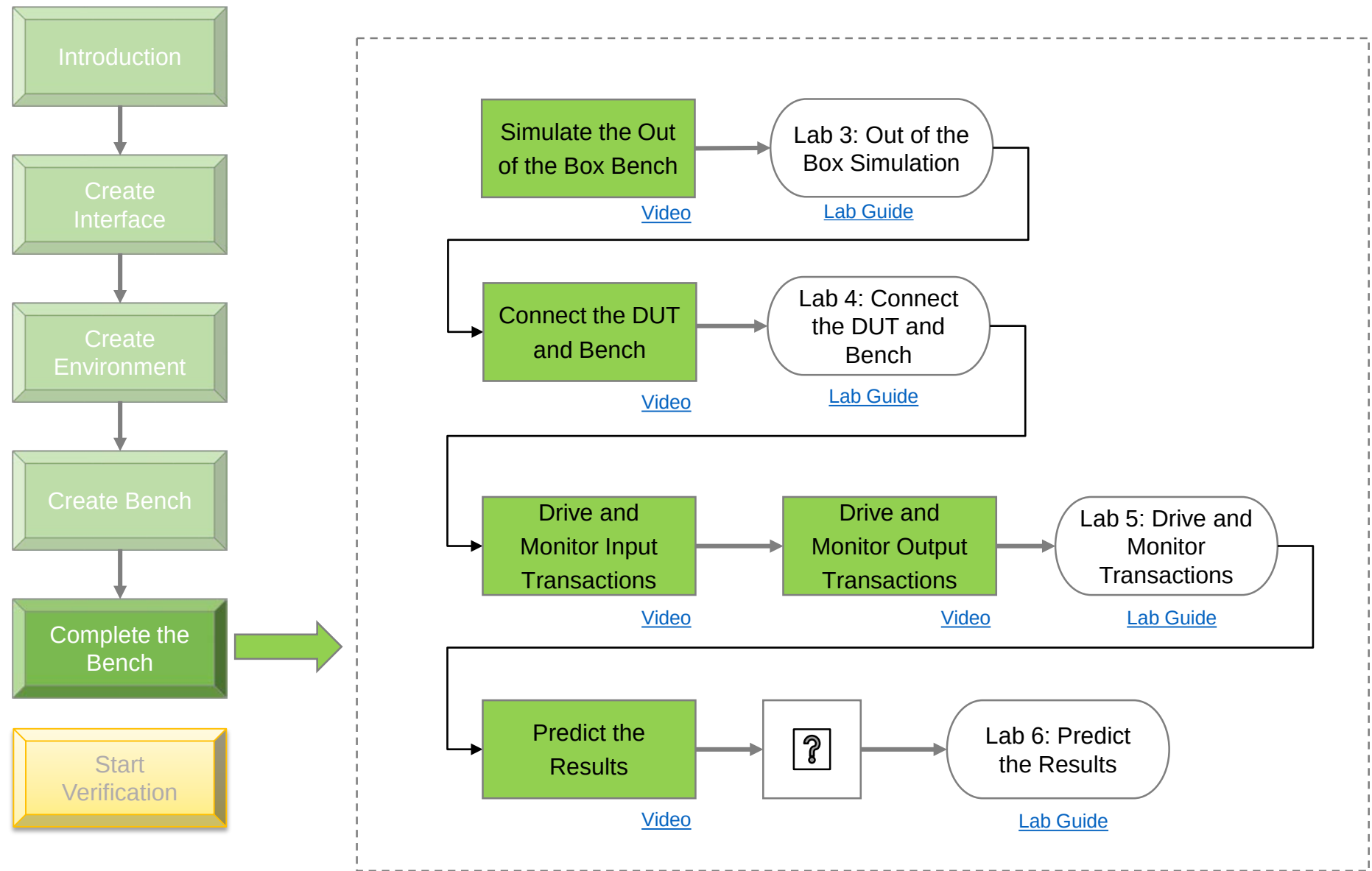
3: Create the Environment



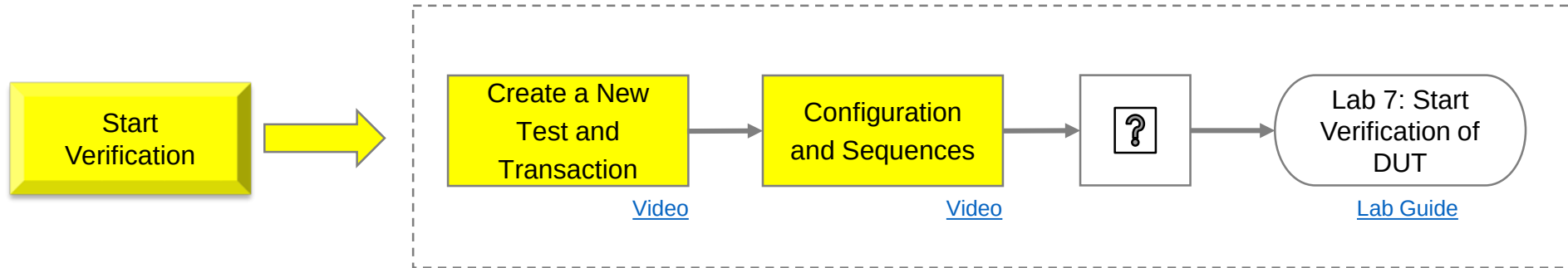
4: Create Bench

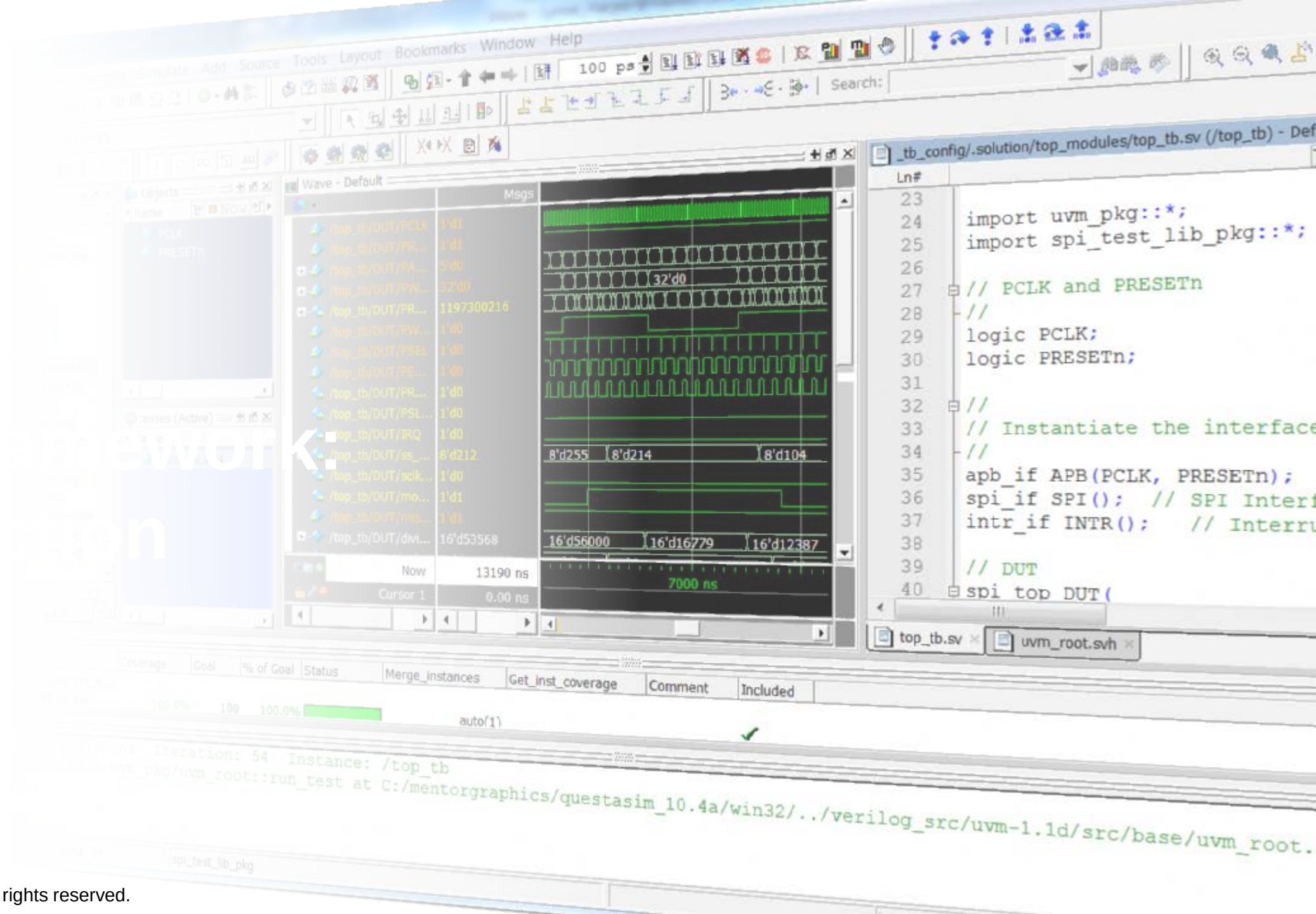


5: Complete the Bench



6: Start Verification





Become Productive Sooner

How can you build a UVM testbench faster?

- UVM Framework accelerates adoption of advanced verification technologies
 - Built on experience from UVM Cookbook
 - Provides a standard architecture, directory structure, package structure & Makefiles
 - Simplify component reuse block-to-top and across projects
 - Veloce compliant, emulation friendly
 - QVIP, VM integration
 - Standard SystemVerilog for all simulators
- Jumpstart UVM with best practices, built from experience
 - Better environments in less time
 - More time to write tests

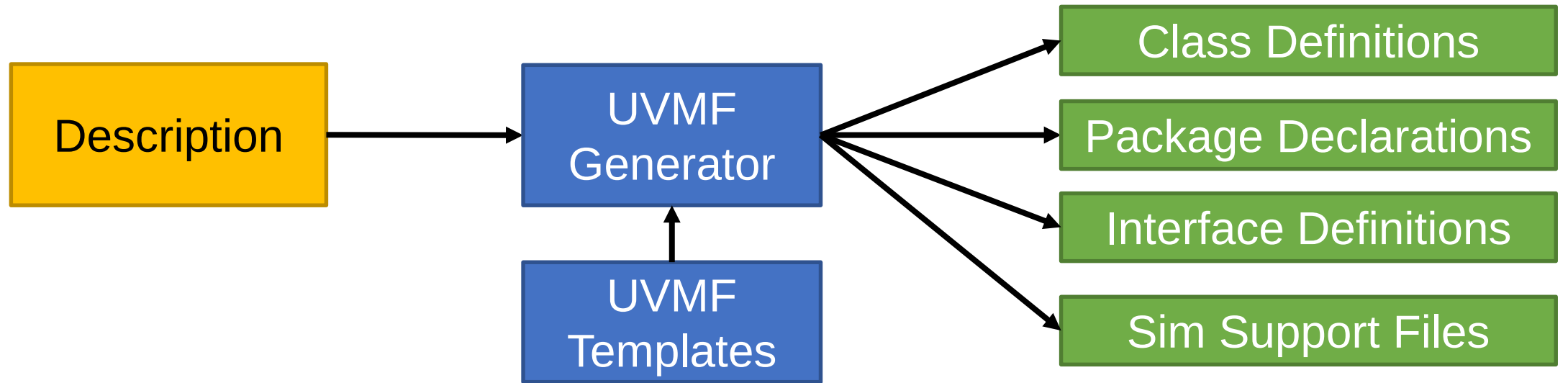


UVM Framework Contents

- Class library
 - Defines reuse methodology
 - Component base classes
 - Package structure for reuse
- Scripts
 - Auto generation of components and test bench
 - Makefiles with common tool flow operations
- Examples
 - Block and chip level benches
 - Technology integrations
- Documentation
 - Users Guide and Reference Manual

Code Generation

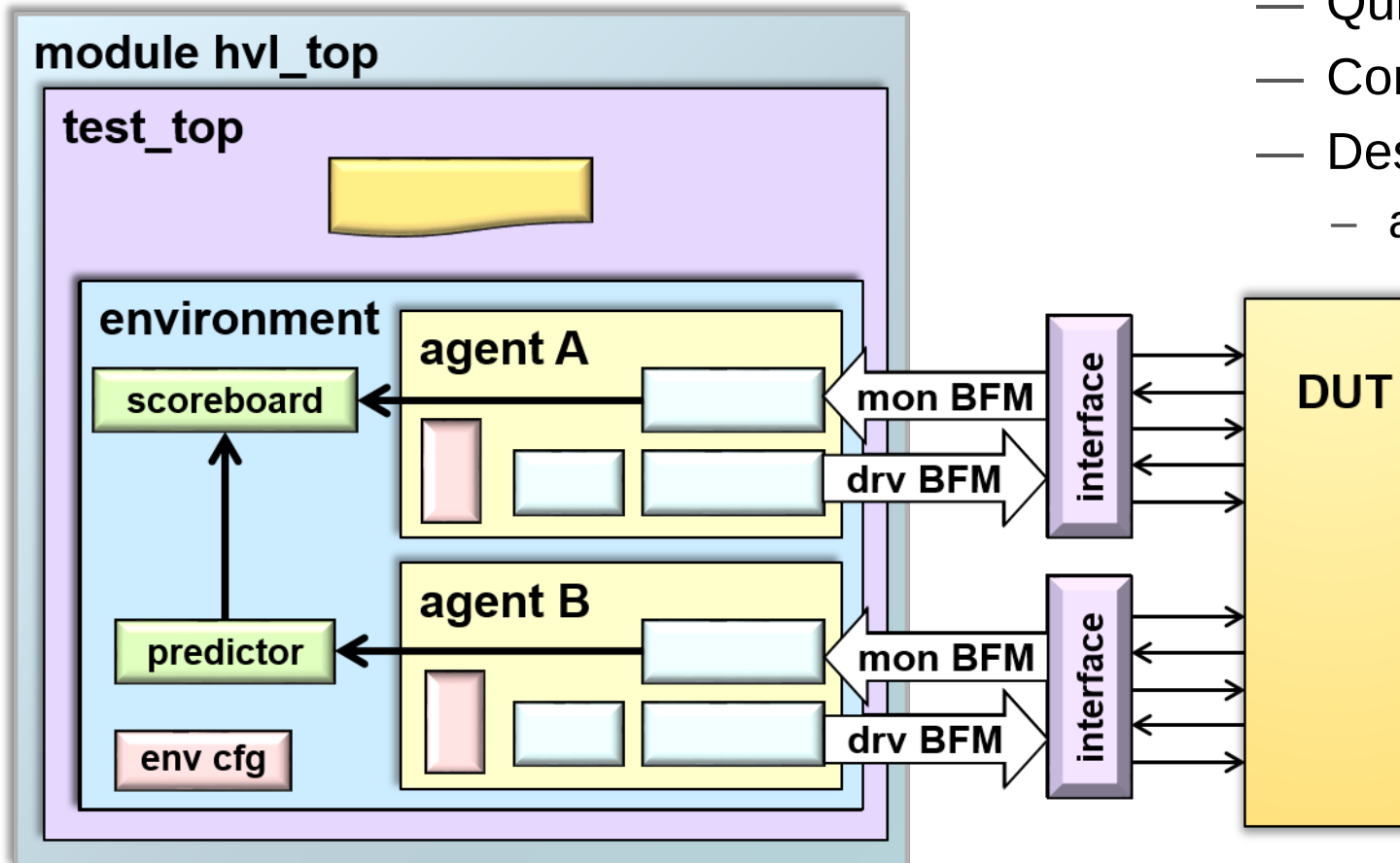
How is the testbench code generated?



Begin Verification Faster

How to build a UVM testbench quickly?

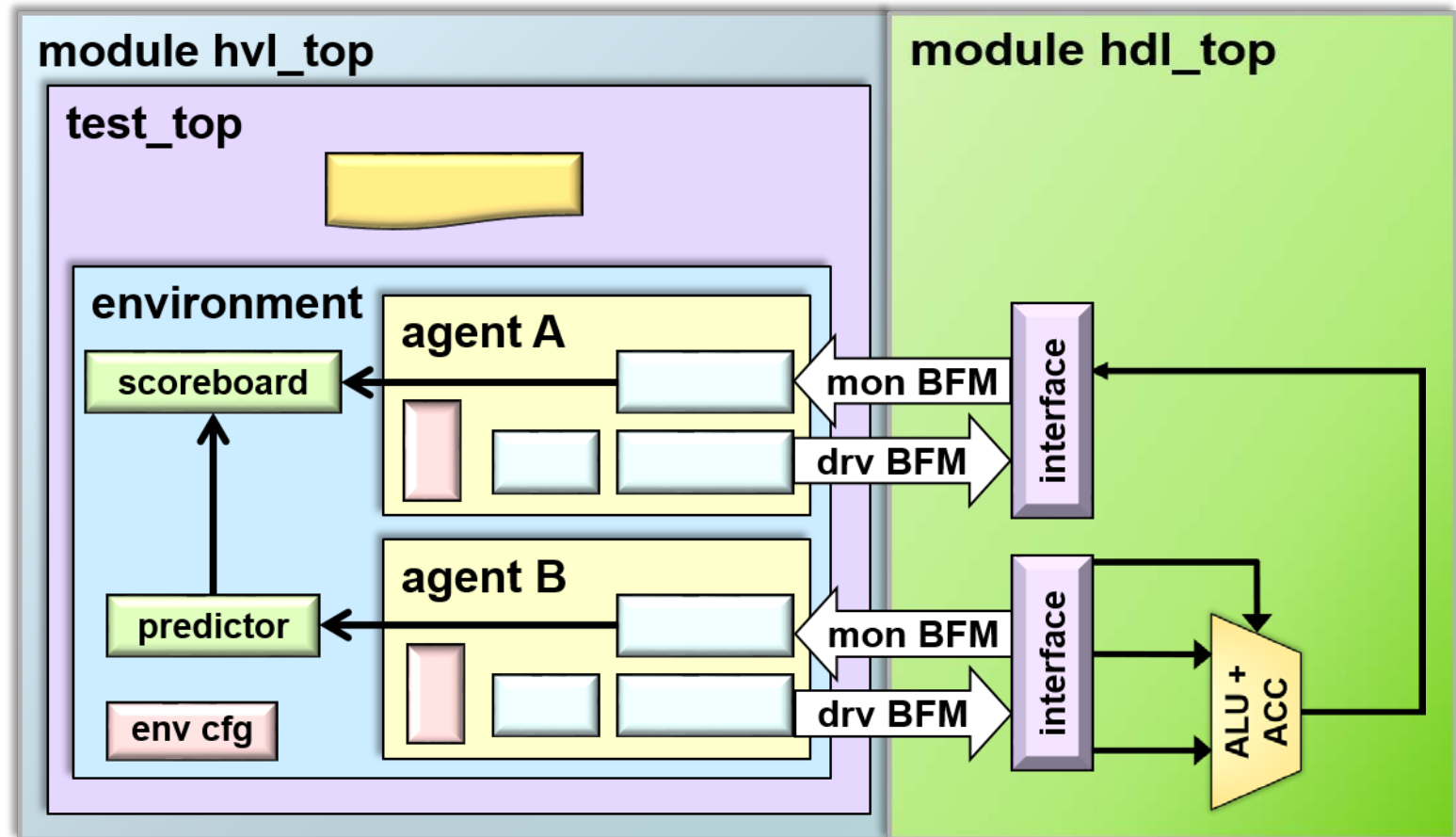
- UVM Framework (UVMF)
 - Generates a complete UVM system
 - Simulate immediately
 - Quickly write sequences
 - Connect to QVIP
 - Designed for block-to-top reuse
 - and future designs too



UVMF Flow

What are the UVMF steps?

0. *Prerequisite: must know UVM*
1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*



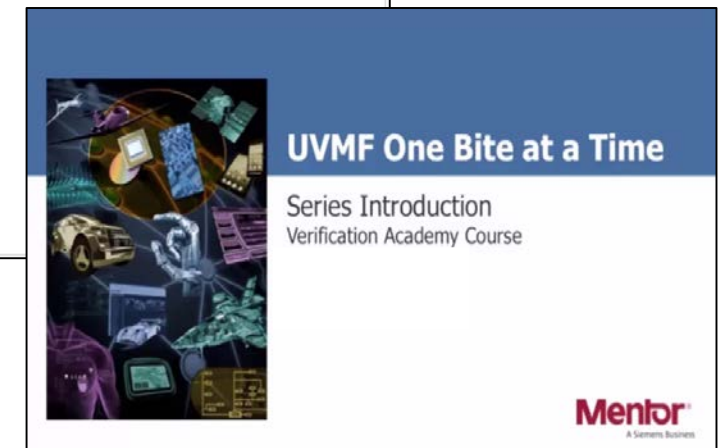
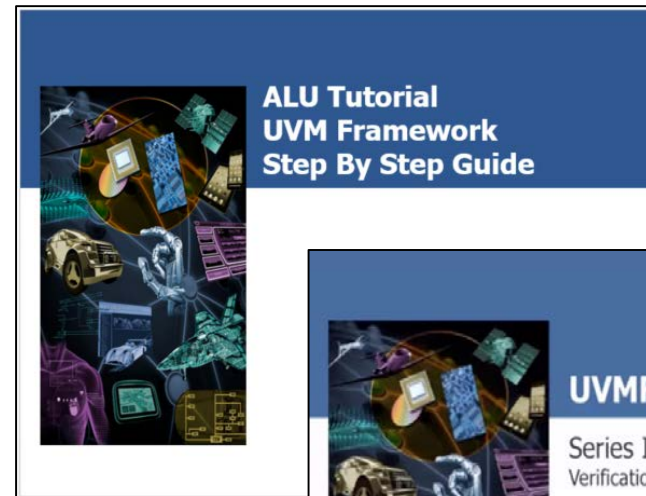
More Details

Where can you learn more?

- Users Guide and more
`$UVMF_HOME/docs`
- ALU tutorial
`$UVMF_HOME/docs/generator_tutorial`
- Examples:
`$UVMF_HOME/base_examples`
`$UVMF_HOME/templates/python/examples`
- [VerificationAcademy.com](https://www.verificationacademy.com)
 - "UVMF One Bite at a Time" videos
- UVMF is a fully supported Mentor tool

UVM Framework
Users Guide

UVM Framework
Code Generator YAML
Reference



Version Updates

- This course developed with UVM Framework 2019.4
 - Some generated methods have changed since 2019.1
 - **driver_bfm::initiate_and_get_response()**
 - Was **do_transfer()**
 - **driver_bfm::respond_and_wait_for_next_transfer()**
 - Was **do_response()**
 - See the Release Notes for more information
- Latest UVMF version on <http://VerificationAcademy.com>

UVM Framework Resources	
Current Releases:	Archived Releases:
+ UVMF - 2019.4	+ UVMF - 3.6h
+ UVMF - 2019.1	+ UVMF - 3.6g
	+ UVMF - 3.6f
	+ UVMF - 3.6e
	+ UVMF - 3.6e

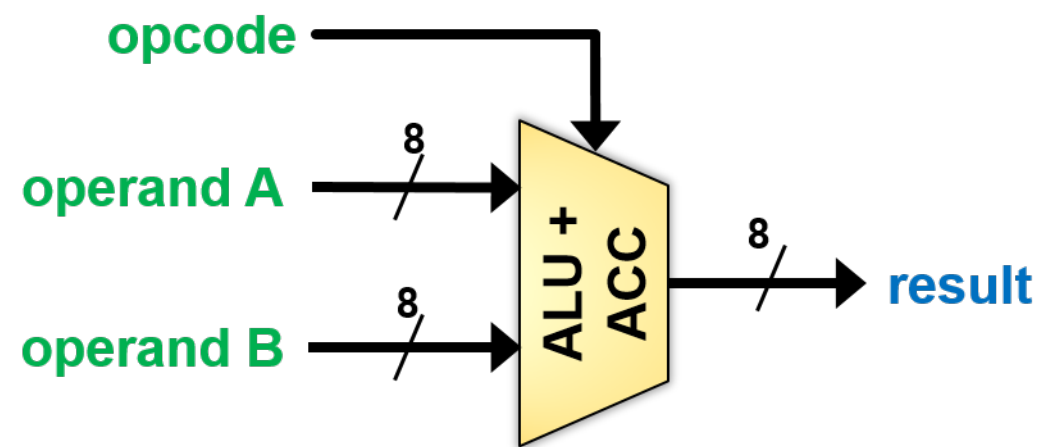
The Design Under Test

1. Plan stimulus and checks

What is the stimulus?

- ALU + Accumulator
 - Design has combinational and sequential logic

```
typedef enum {...} ALU_IN_OP_T;
```

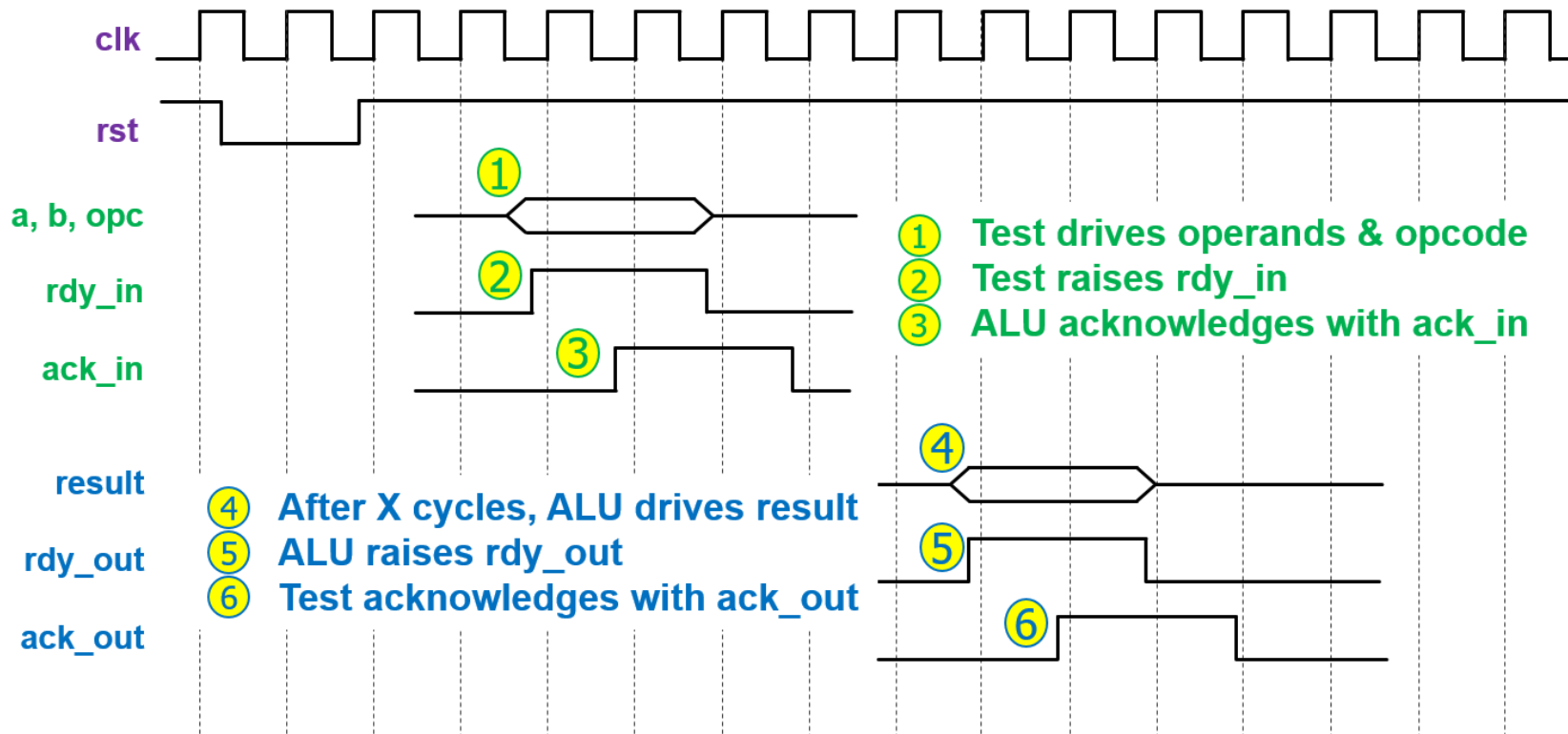
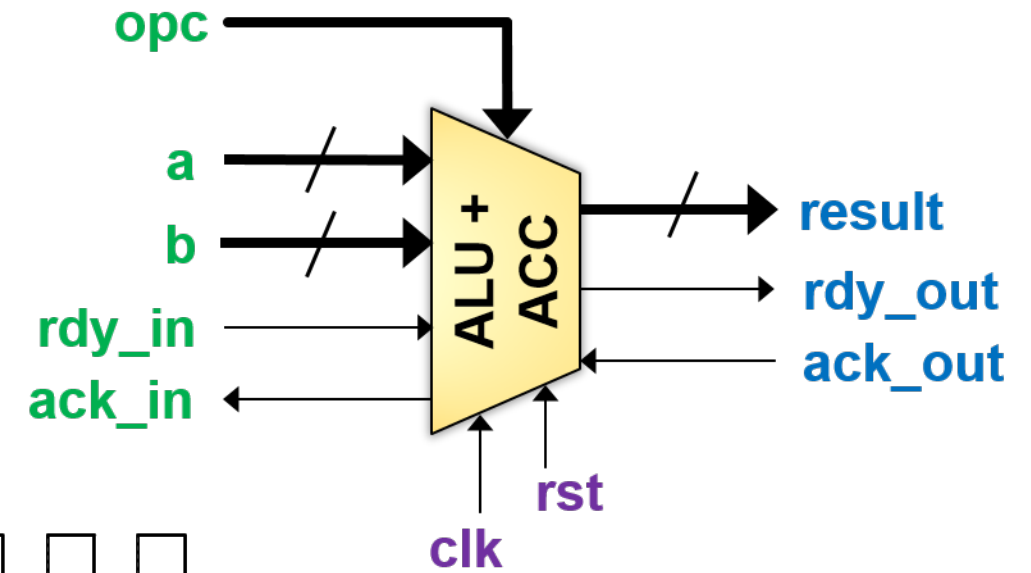


Opcode	Operation	Value
NO_OP	R=0	'b000
AND_OP	R=A&B	'b001
OR_OP	R=A B	'b010
XOR_OP	R=A^B	'b011
ADD_OP	R=A+B	'b100
ADC_OP	R=A+ACC ACC=R	'b101
CLC_OP	R=0 ACC=0	'b110
RSV_OP	<i>reserved</i>	'b111

DUT Timing

When is stimulus sent and received?

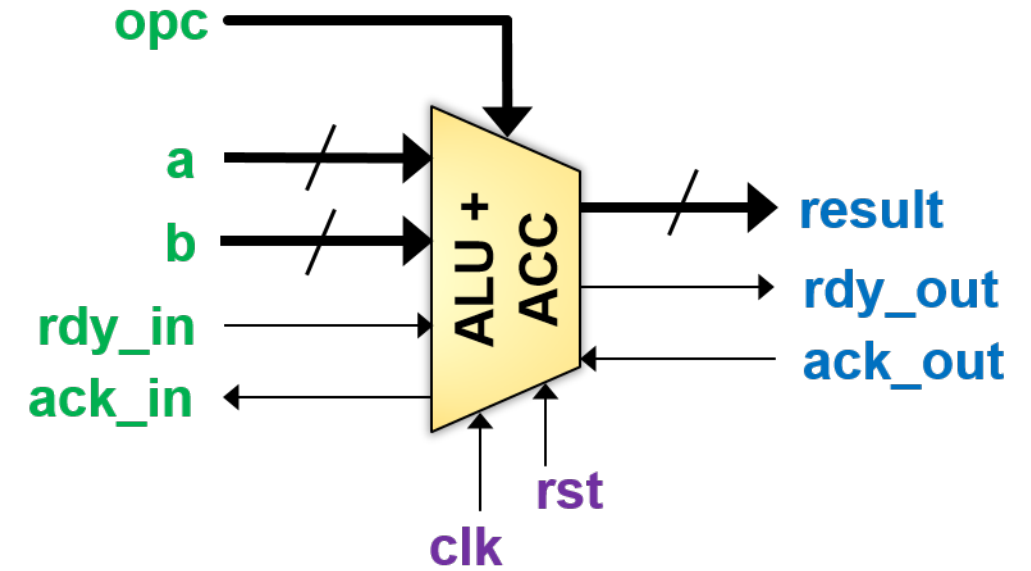
- Two separate protocols, by functionality



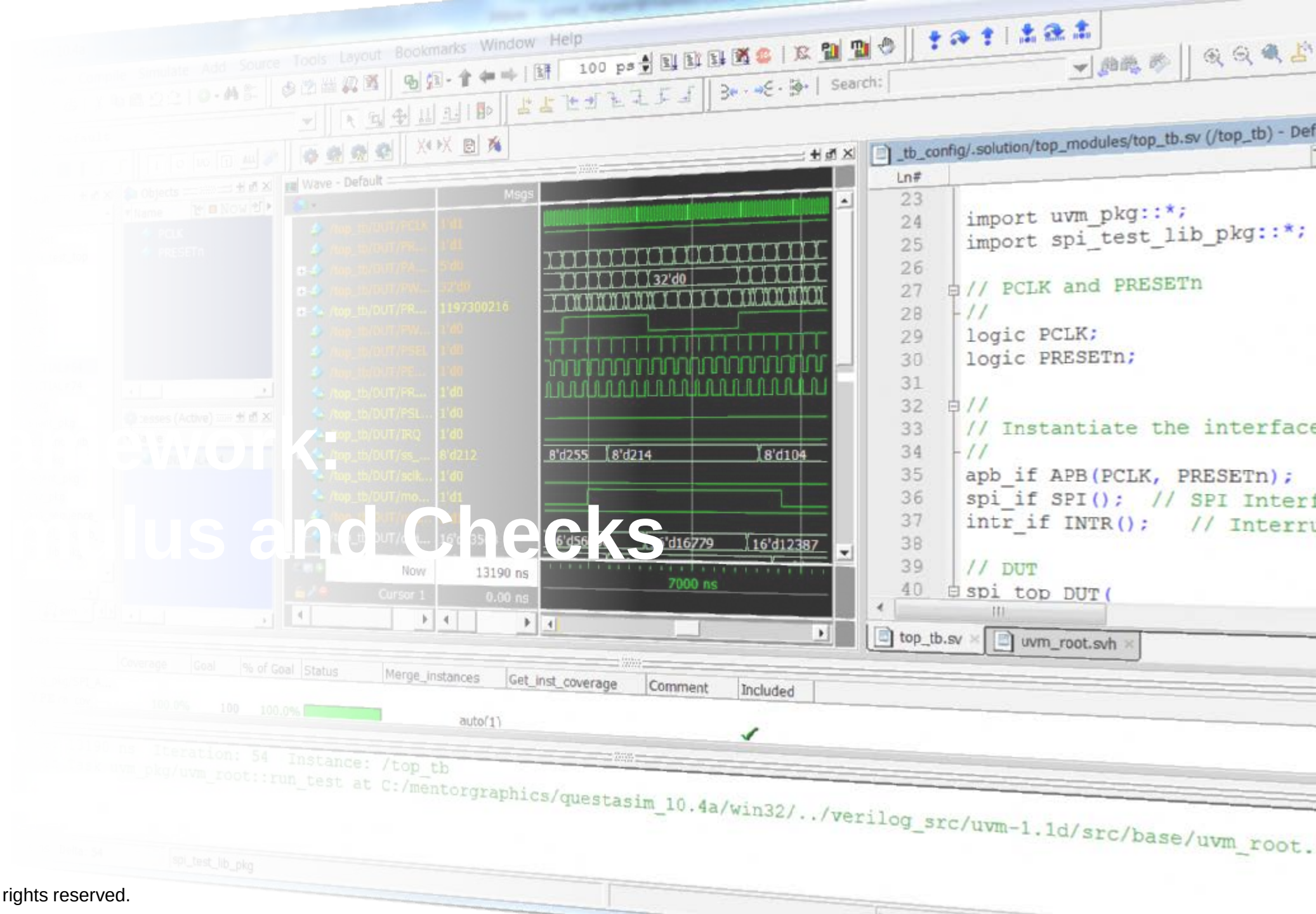
DUT Code

What does the DUT code look like?

```
module alu (input  logic clk, rst,  
  
            input  logic [2:0] opc,  
            input  logic [7:0] a, b,  
            input  logic rdy_in,  
            output logic ack_in,  
  
            output logic [7:0] result,  
            output logic rdy_out,  
            input  logic ack_out);  
  
    // RTL code  
    ...  
endmodule
```



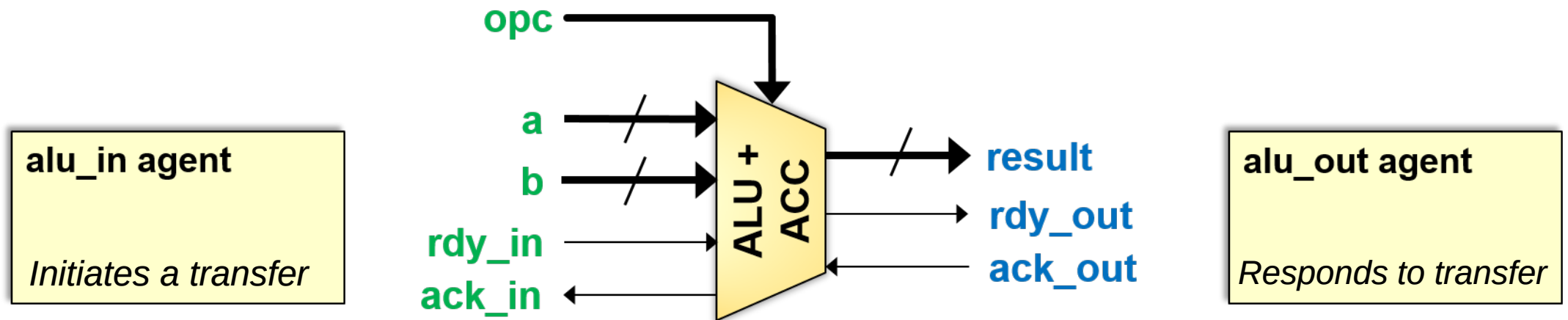
Work: Plus and Checks



Plan Stimulus and Checking

How will the UVMF testbench communicate with the DUT??

- ALU has two interfaces
- Connected to two testbench agents
 - alu_in: initiator
 - alu_out: responder
 - *Both turn transactions into bus ops*
- An active agent includes a driver
- A passive agent has only a monitor



DUT Communication

What are the transactions?

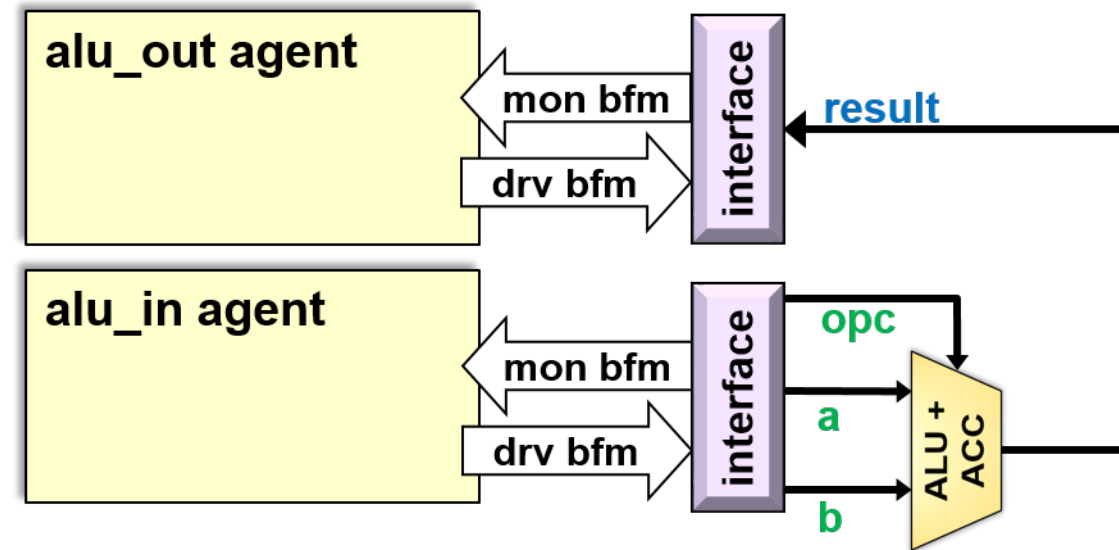
1. Plan stimulus ...



- Transaction class does not include the handshaking signals
- Derived from `uvm_sequence_item`

```
class alu_out_transaction  
    bit [7:0] result;  
    ...
```

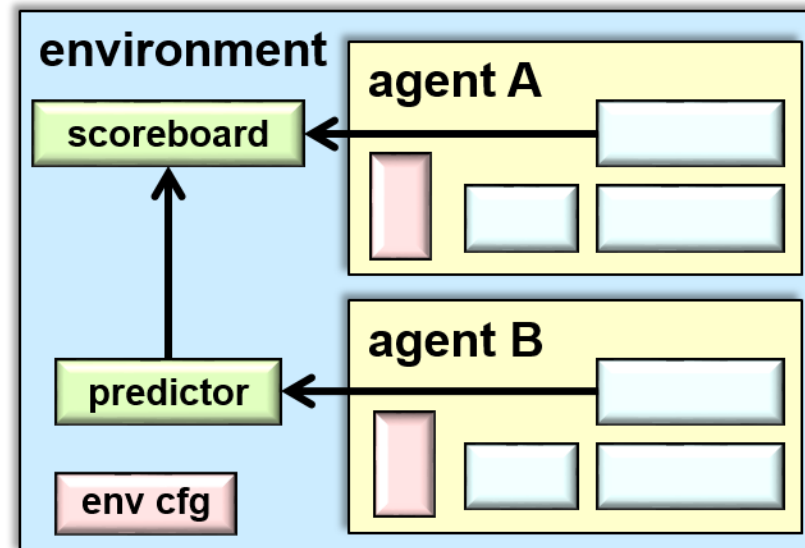
```
class alu_in_transaction  
    rand ALU_IN_OP_T opcode;  
    rand bit [7:0] a, b;  
    ...
```



DUT Checking

How to verify a generic DUT with UVMF?

- UVMF environment topology
 - Reusable across multiple designs
- Predictor driven by inputs
 - Turn input transactions into output transactions
- Scoreboard compares
 - Expected vs. actual
 - In-order vs. out-of-order
 - Number of inputs



DUT Checking

How to verify the ALU with UVMF?

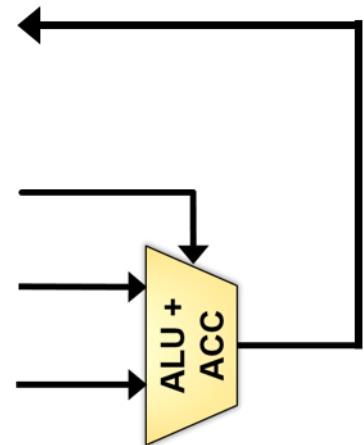
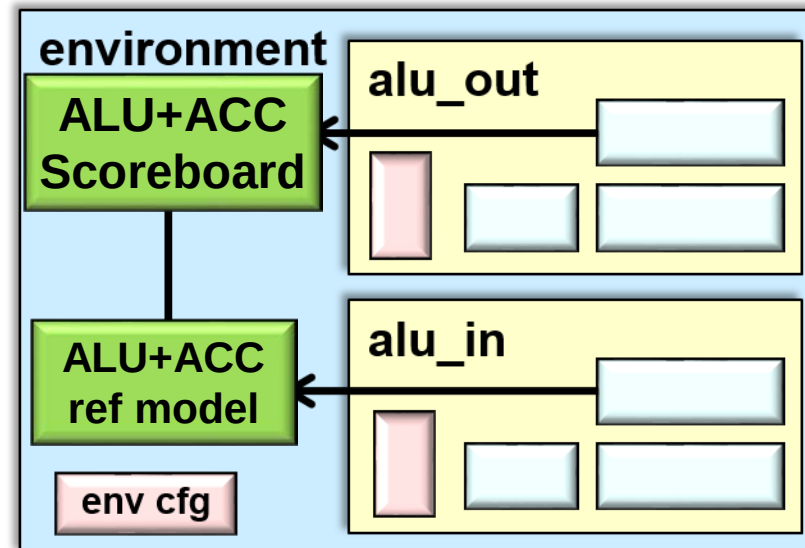
- Typical UVMF environment topology
 - Reusable across multiple designs

1. Plan stimulus and checking



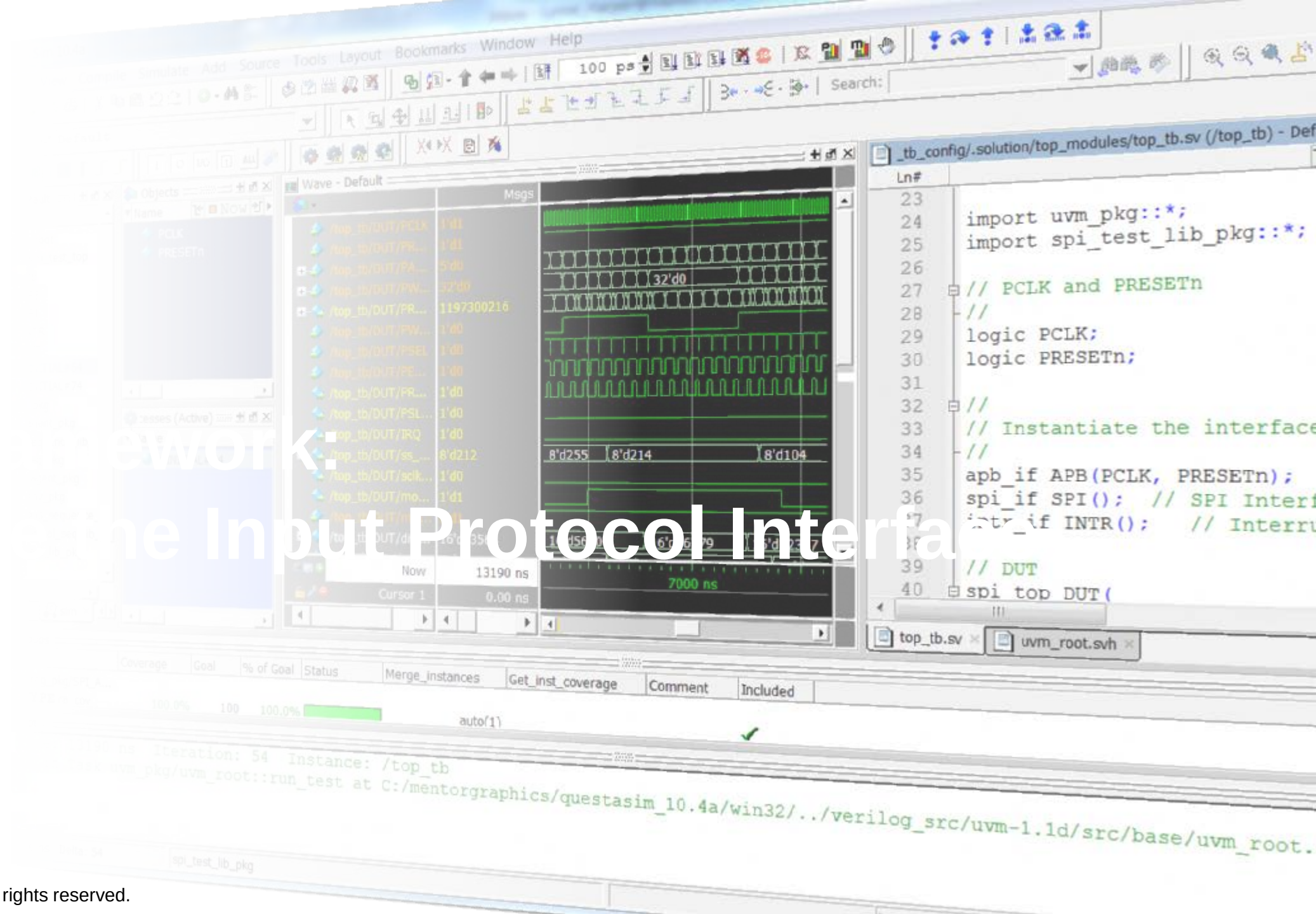
- Predictor driven by inputs
 - Turn input transactions into output transactions

- Scoreboard compares
 - Expected vs. actual
 - In-order vs. out-of-order
 - Number of inputs = 2



Knowledge Check - Introduction

- Which of these is **not** a goal for UVM Framework?
- Which are true about the design under test?

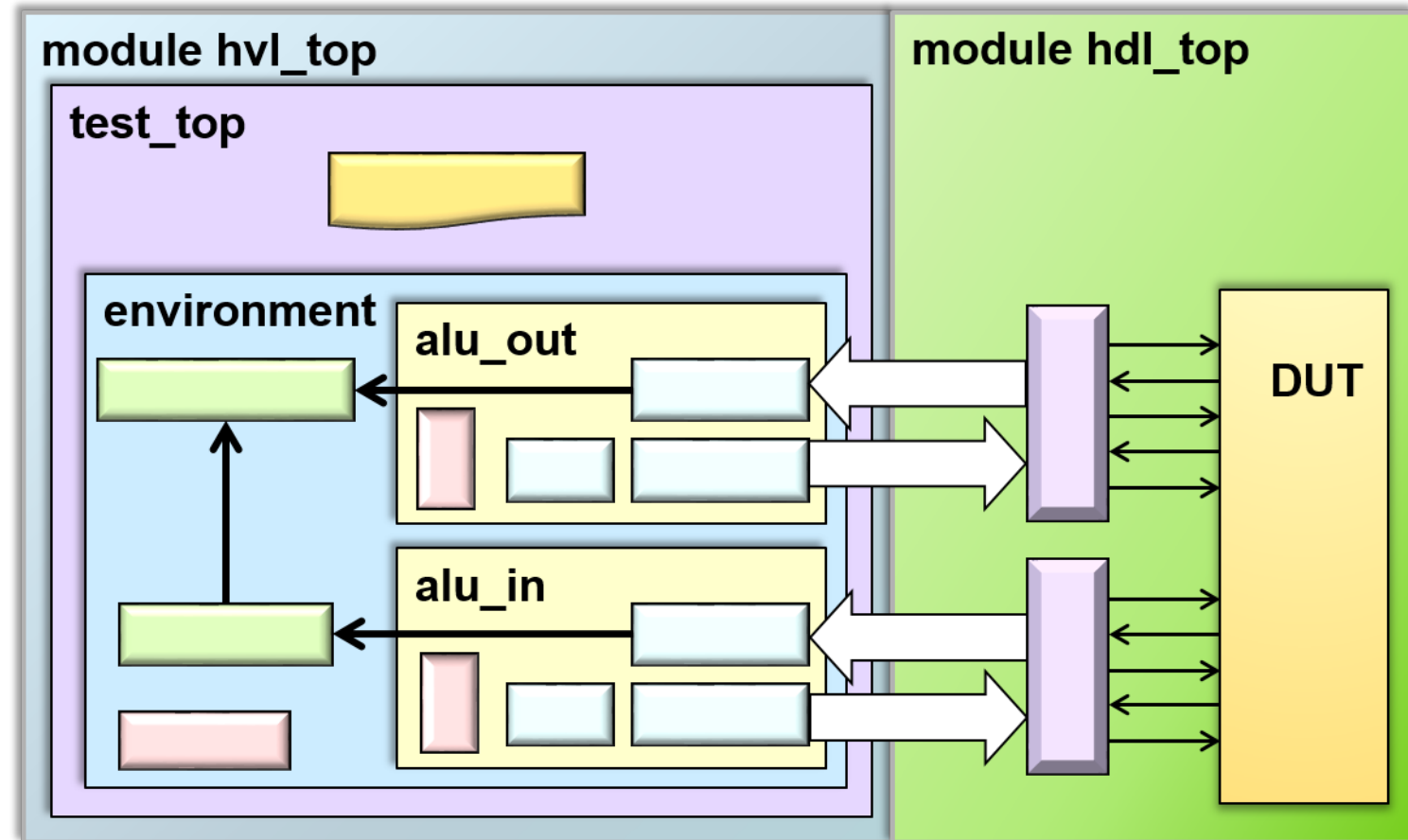


UVM Input Protocol Interface

Course Flow

What are the UVMF steps?

1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*

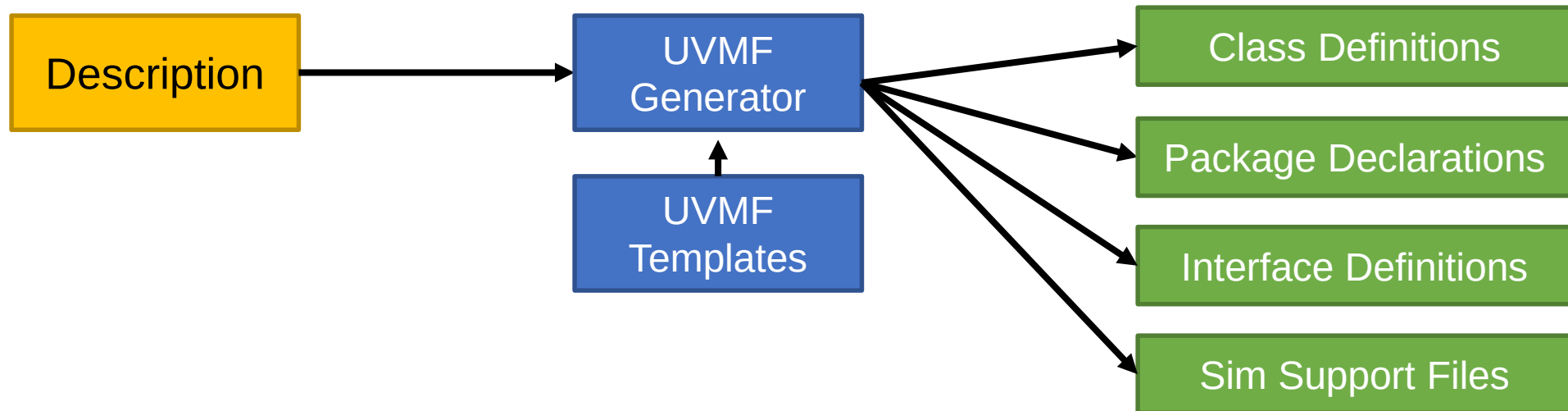


Design Description

How is the testbench code generated?

2. *Create the DUT connections*

- a) Describe the transactions and signals
- b) Run the UVMF generator
- c) Check the results and update the description as needed



Design Description

How is the system described?

- With YAML
 - *YAML Ain't a Markup Language*
`$UVMF_HOME/docs`
www.yaml.org
- Classic python flow still supported

UVM Framework
Code Generator YAML
Reference
Revision 3.6h

UVM Generation Flow

```
# Describe UVMF structures
uvmf:
  interfaces:
    interface_in:
      properties
    interface_out:
      properties
```

```
uvmf:
  util_components:
    util_component:
      properties
```

```
uvmf:
  environments:
    env_name:
      properties
```

```
uvmf:
  benches:
    bench_name:
      properties
```

```
% yaml2uvmf.py *.yaml
```

UVMF
Templates

Package Declarations

Class Definitions

Interface Definitions

Sim Support Files

- Iterative process
- Script does not overwrite existing files!

Interface Description

How to describe a protocol?

- For each interface
 - clock and reset (required)

Indent consistently
2 spaces per level
No tabs

```
# alu_in_interface.yaml
```

```
uvmf:
```

```
  interfaces:
```

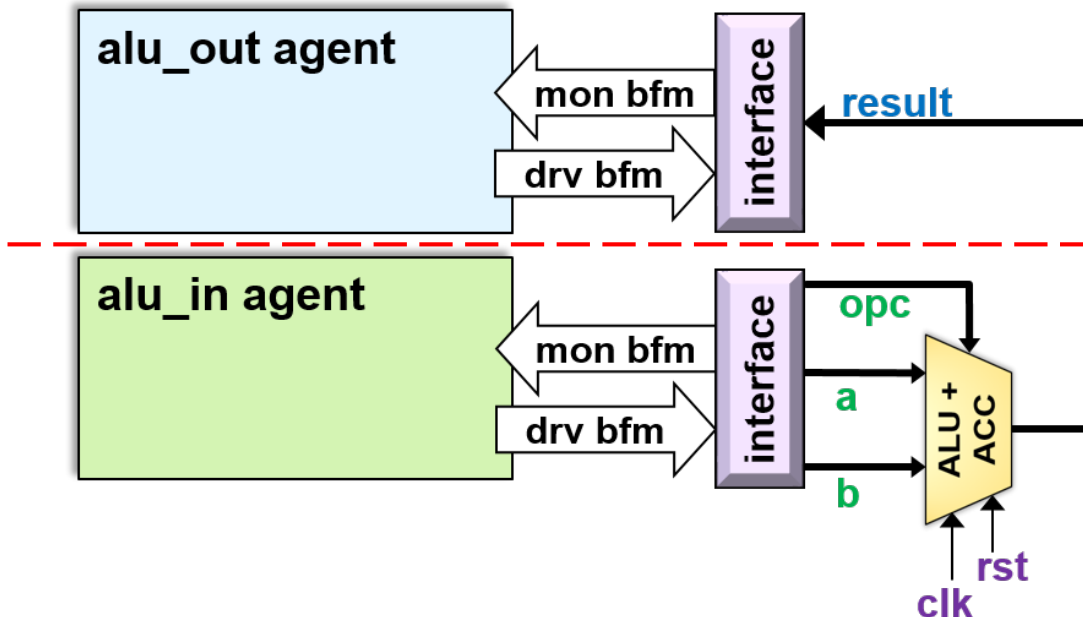
```
    alu_in:
```

```
      clock: clk
```

```
      reset: rst
```

```
      reset_assertion_level: "False"
```

```
      ...
```



Interface Description

How to describe a protocol?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters

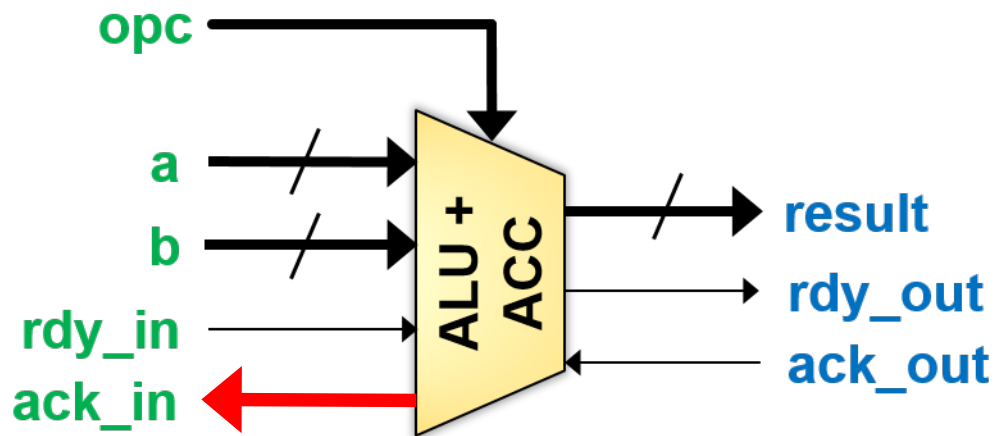
Unused YAML parameter:
omit, or use []

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      ...
      hdl_typedefs:
        - name: ALU_IN_OP_T
          type: "enum bit [2:0] {
            NO_OP = 3'b000, AND_OP = 3'b001,
            OR_OP = 3'b010, XOR_OP = 3'b011,
            ADD_OP = 3'b100, ADC_OP = 3'b101,
            CLC_OP = 3'b110, RSV_OP = 3'b111}"
      hvl_typedefs: []
      parameters: []
      ...
```

Interface Description

How to describe a protocol?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters
 - Ports



```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      ...
      ports:
        - name: opc
          dir: output
          width: "3"
          reset_value: "0"
        - name: a
          dir: output
          width: "8"
          reset_value: "0"
        - dir: output
          width: "8"
          name: b
          reset_value: "0"
        - name: rdy_in
          dir: output
          width: "1"
          reset_value: "0"
        - name: ack_in
          dir: input
          width: "1"
      ...
```

alu_in is an Initiator
dir: relative to TB

Properties in
any order

Interface Description

How to describe a protocol?

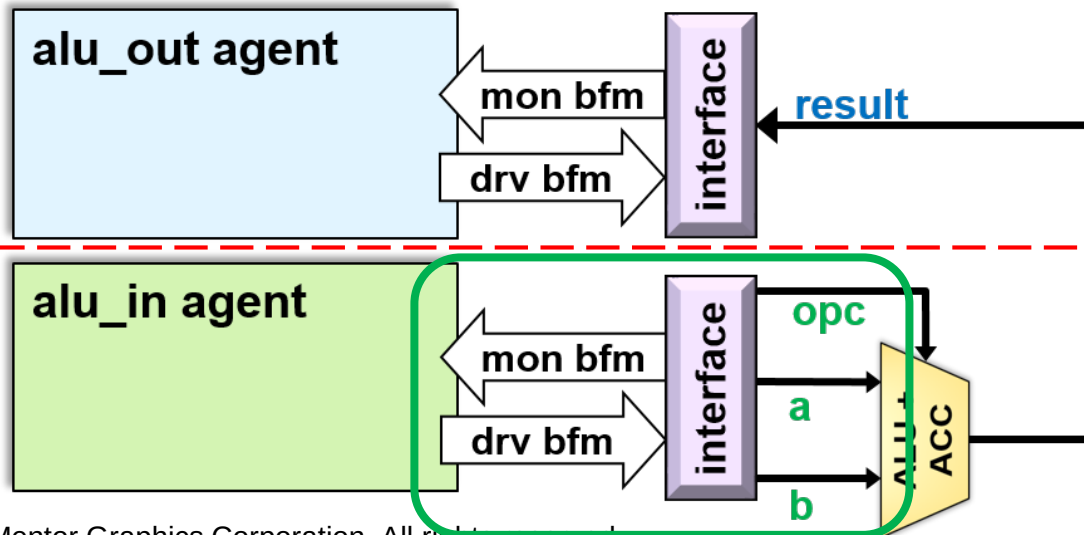
- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters
 - Ports
 - Transactions
 - and constraints

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      ...
      transaction_vars:
        - name: opc
          type: ALU_IN_OP_T
          isrand: "True"
          iscompare: "False"
        - name: a
          type: bit [7:0]
          isrand: "True"
          iscompare: "False"
        - name: b
          type: bit [7:0]
          isrand: "True"
          iscompare: "False"
      transaction_constraints:
        - name: valid_opc_c
          value: "{soft opc inside {
                    NO_OP, AND_OP, OR_OP, XOR_OP,
                    ADD_OP, ADC_OP, CLC_OP}};"
```

Interface Description

How to describe a protocol?

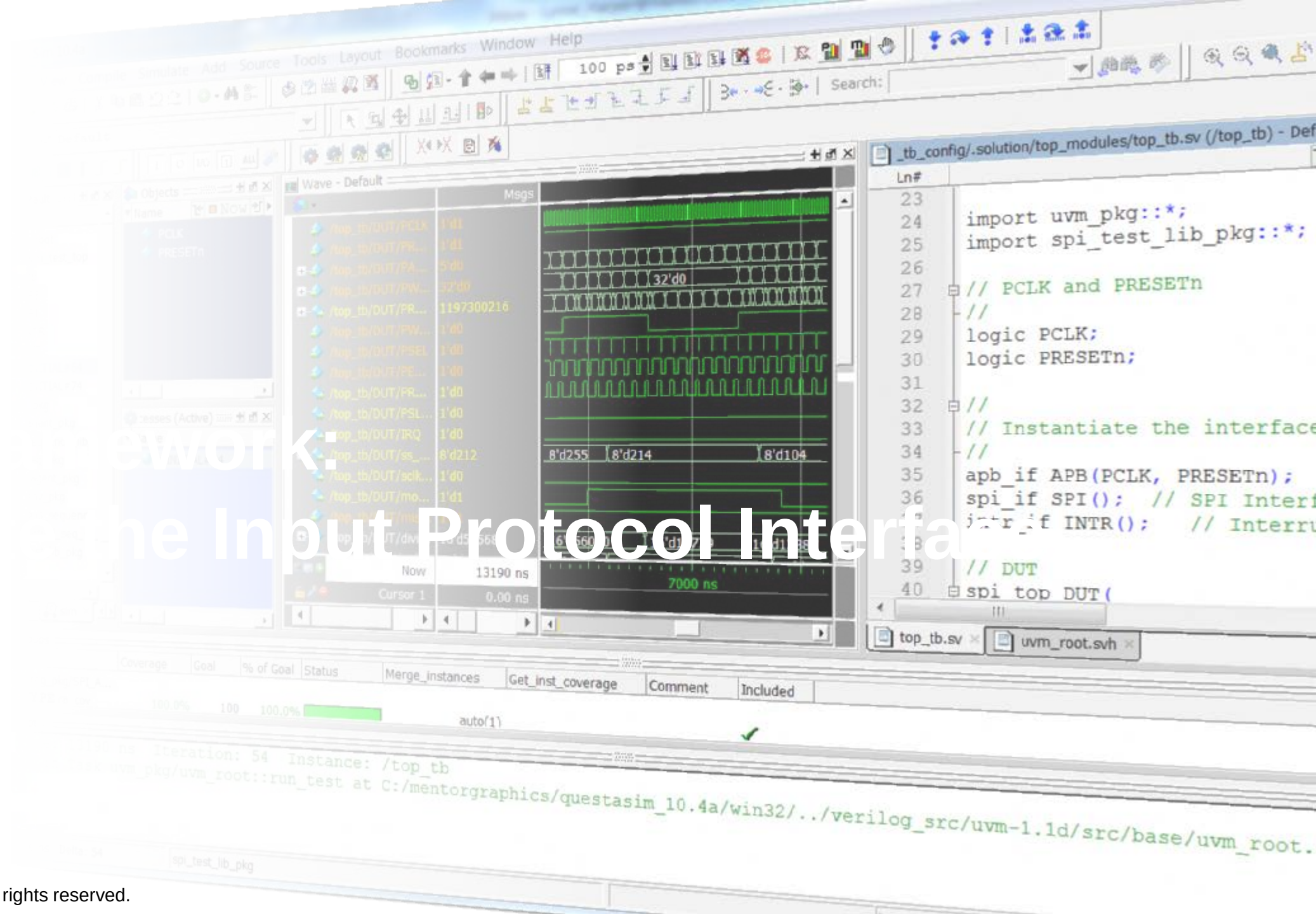
- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters
 - Ports
 - Transactions
 - and constraints



```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      clock: ...
      reset: ...
      reset_assertion_level: ...
      hdl_typedefs:
        ...
      hvl_typedefs:
        ...
      parameters:
        ...
      ports:
        ...
      transaction_vars:
        ...
      transaction_constraints:
        ...
```



UVM Work: The Input Protocol Interface



Input Interface Generation

What is created from the YAML description?

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      ...
```

```
% yaml2uvmf.py \
  alu_in_interface.yaml
```

UVMF
Templates

uvmf_template_output

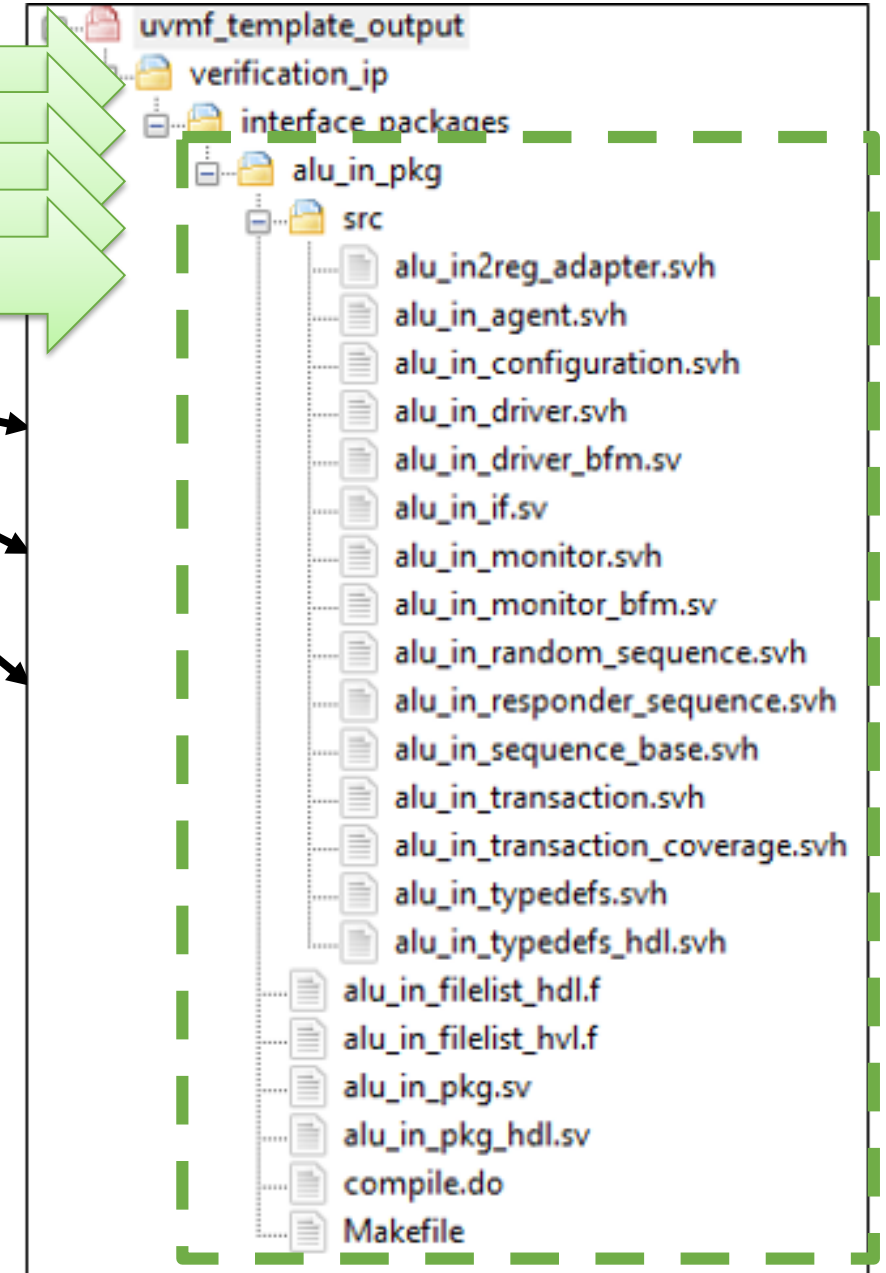
verification_ip

interface_packages

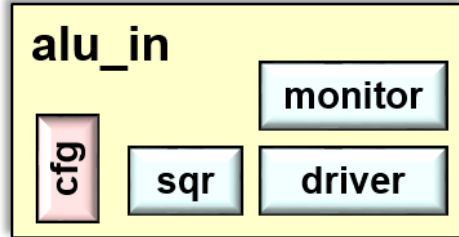
alu_in_pkg

src

alu_in*.svh



Dive into Generated Code

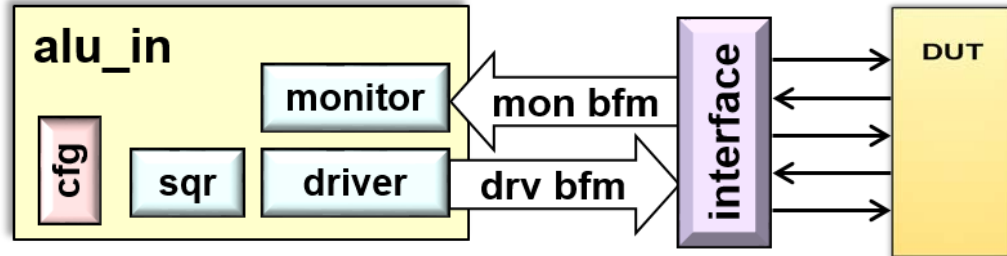


verification_ip/interface_packages/alu_in_pkg

```
package alu_in_pkg;
  import uvm_pkg::*;
  `include "src/alu_in_transaction.svh"
  `include "src/alu_in_configuration.svh"
  `include "src/alu_in_monitor.svh"
  `include "src/alu_in_driver.svh"
  `include "src/alu_in_agent.svh"
  ...
endpackage
```

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      ...
```

Dive into Generated Code



`verification_ip/interface_packages/alu_in_pkg/src`

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      clock: clk
      reset: rst
      ports:
        - name: opc      ...
        - name: a        ...
        - name: b        ...
        - name: rdy_in   ...
        - name: ack_in   ...
```

```
interface alu_in_if (clk, rst, opc, a, b, rdy_in, ack_in);
```

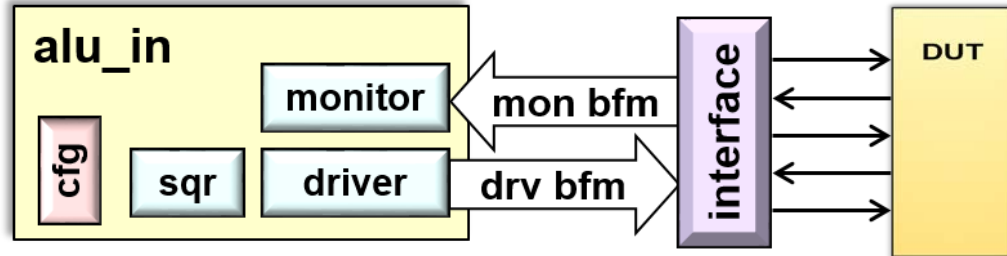
```
interface alu_in_driver_bfm (alu_in_if bus);
```

```
interface alu_in_monitor_bfm (alu_in_if bus);
```



Fill in the details of the specific protocols

Dive into Generated Code



`verification_ip/interface_packages/alu_in_pkg/src`

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      clock: clk
      reset: rst
      ports:
        - name: opc      ...
        - name: a        ...
        - name: b        ...
        - name: rdy_in   ...
        - name: ack_in   ...
```

```
interface alu_in_if (clk, rst, opc, a, b, rdy_in, ack_in);
```

```
interface alu_in_driver_bfm (alu_in_if bus);
  task initiate_and_get_response(...); // New in 2019.4
    // UVMF_CHANGE_ME : ...
    @(posedge clk_i);
    @(posedge clk_i);
    ...
  endtask
endinterface
```



Dive into Generated Code

verification_ip/interface_packages/alu_in_pkg/src

```
# alu_in_interface.yaml
uvmf:
  interfaces:
    alu_in:
      transaction_vars:
        - name: opc
        - name: a
        - name: b
      transaction_constraints:
        - name: valid_opc_c
```

```
class alu_in_transaction extends uvmf_transaction_base;
  rand ALU_IN_OP_T opc;
  rand bit [7:0] a;
  rand bit [7:0] b;
  constraint valid_opc_c {...};
```

```
class alu_in_sequence_base extends uvmf_sequence_base
  #(.REQ(alu_in_transaction), .RSP(alu_in_transaction));
```

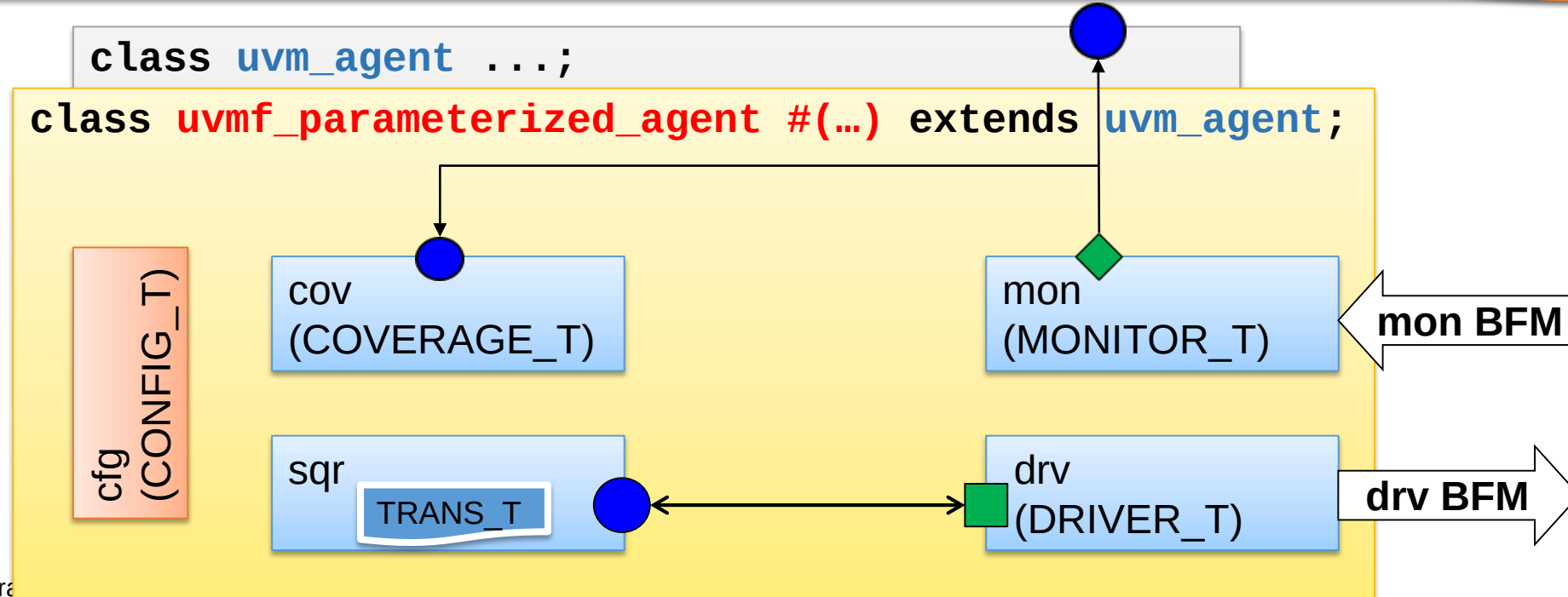
```
class alu_in_random_sequence extends alu_in_sequence_base;
```


Parameterized Base Classes

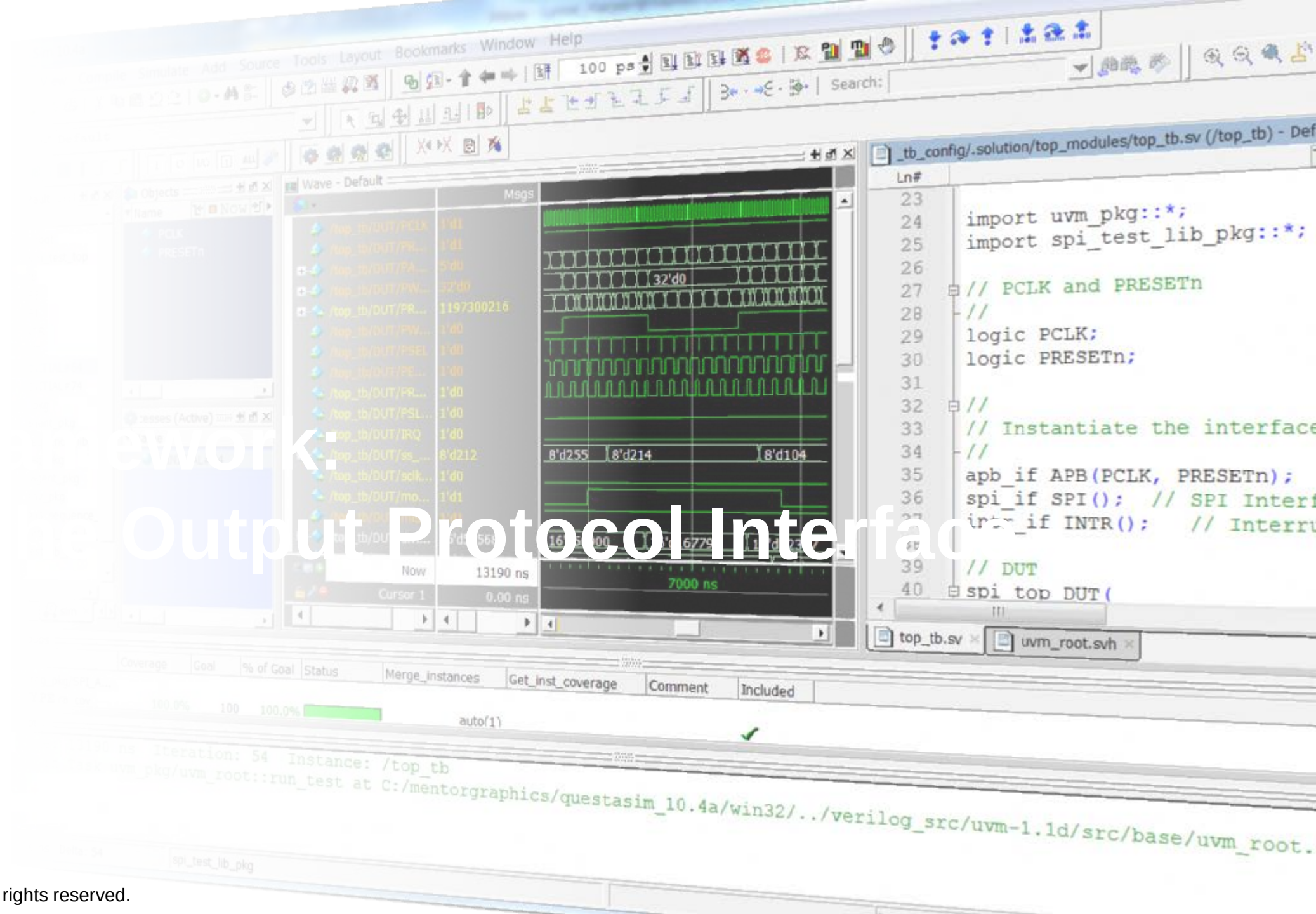
```
class alu_in_agent extends uvmf_parameterized_agent
    #( .CONFIG_T(alu_in_configuration ),
      .DRIVER_T(alu_in_driver ),
      .MONITOR_T(alu_in_monitor ),
      .COVERAGE_T(alu_in_transaction_coverage ),
      .TRANS_T(alu_in_transaction ) );

`uvm_component_utils( alu_in_agent )
function new(...); ...
endclass
```

Where are build_phase()
and connect_phase() ?



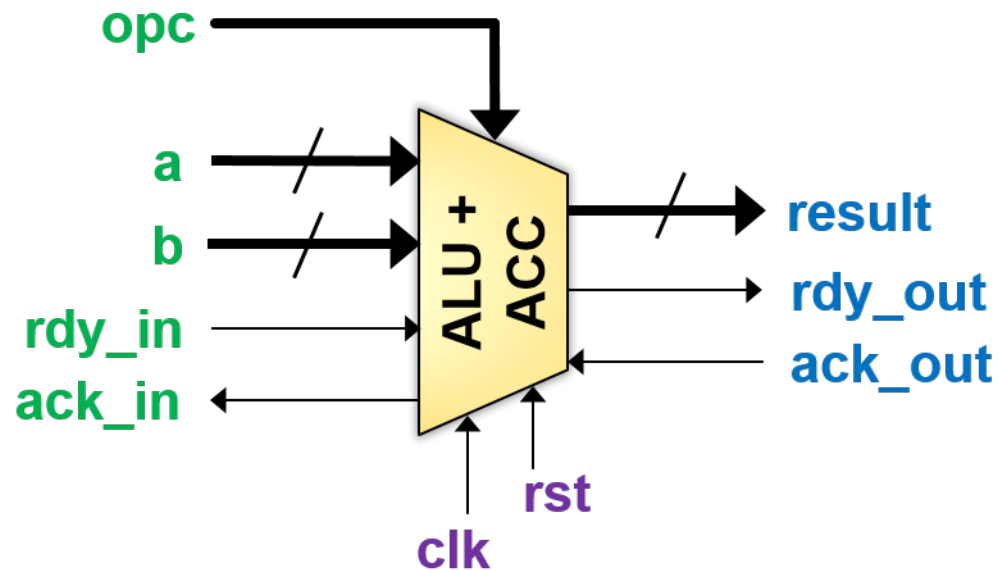
UVM Work: Output Protocol Interface



DUT Output Description

How to describe the ALU output?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters



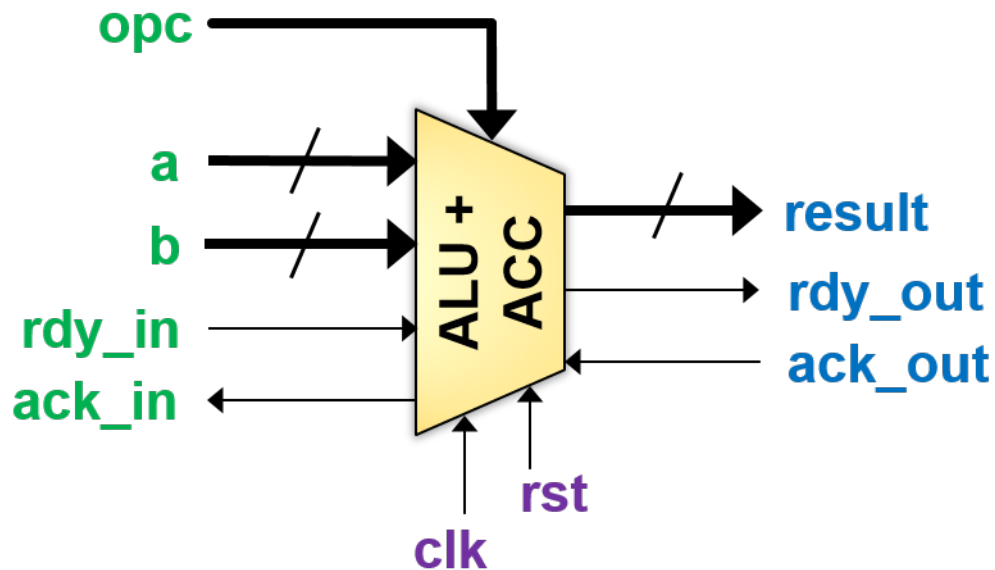
```
# alu_out_interface.yaml
uvmf:
  interfaces:
    alu_out:
      clock: clk
      reset: rst
      reset_assertion_level: "False"

      hdl_typedefs: []
      hvl_typedefs: []
      parameters: []
      ...
```

DUT Output Description

How to describe the ALU output?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - SystemVerilog parameters
 - Ports



```
# alu_out_interface.yaml
uvmf:
  interfaces:
    alu_out:
      ...
      ports:
        - name: result
          dir: input
          width: "8"
          reset_value: "0"
        - name: rdy_out
          dir: input
          width: "1"
          reset_value: "0"
        - name: ack_out
          dir: output
          width: "1"
          reset_value: "0"
      ...
```

relative to testbench

DUT Output Description

How to describe the ALU output?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - Parameters
 - Ports
 - Transactions
- Initiator or Responder? Specified at environment level

```
# alu_out_interface.yaml
uvmf:
  interfaces:
    alu_out:
      ...
      transaction_vars:
        - name: result
          type: bit [7:0]
          iscompare: "True"
          isrand: "False"
```

UVMF Generator

What is created from this description?

```
# alu_out_interface.yaml
uvmf:
  interfaces:
    alu_out:
      ...
```

```
% yam2uvmf.py \
  alu_out_interface.yaml
```

UVMF
Templates

uvmf_template_output

verification_ip

interface_packages

alu_out_pkg

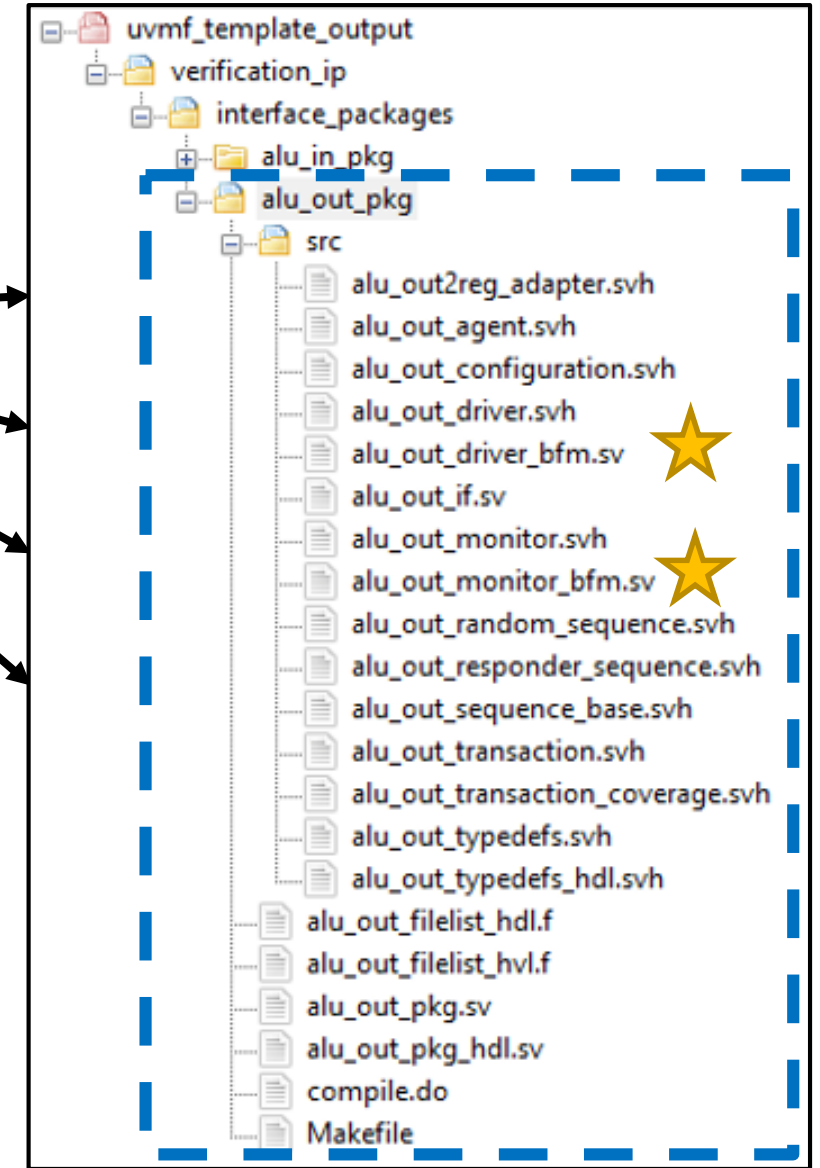
alu_in_pkg

src

src

alu_out*.svh

alu_in*.svh

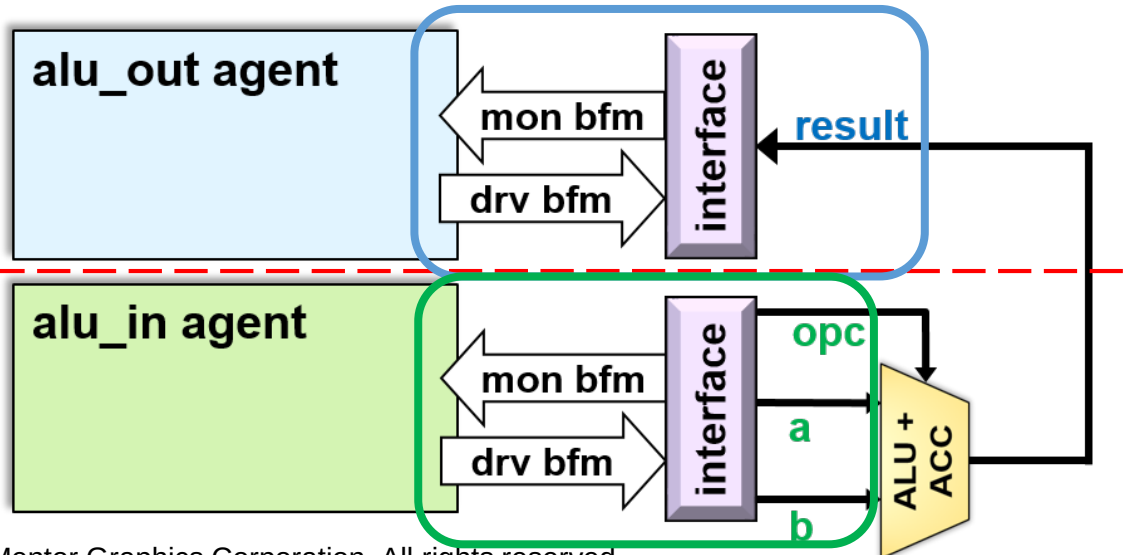


DUT Input and Output Descriptions

How to describe the ALU connections?

- For each interface
 - clock and reset (required)
 - HDL & HVL types
 - Parameters
 - Ports
 - Transactions
 - and constraints

2. Create the DUT connections



Knowledge Check – Create Interface

- A YAML interface description includes all of the following except:
- What is the top directory created by the UVMF generator?

Lab 1 – Protocol Interface Generation

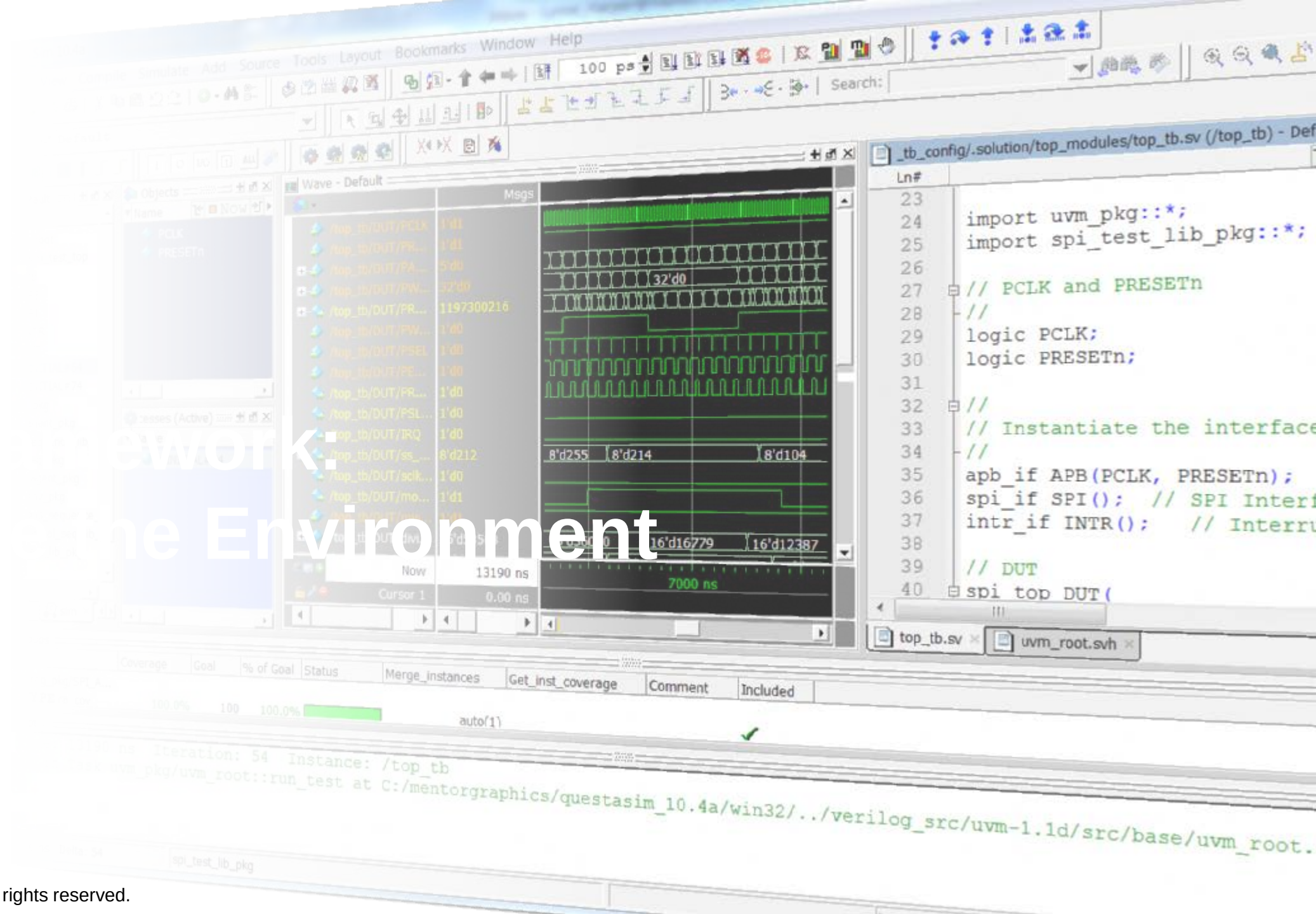


Goal: Create a protocol interface description

- Create **alu_in_interface.yaml** and **alu_out_interface.yaml** descriptions
- Generate and explore the resulting code



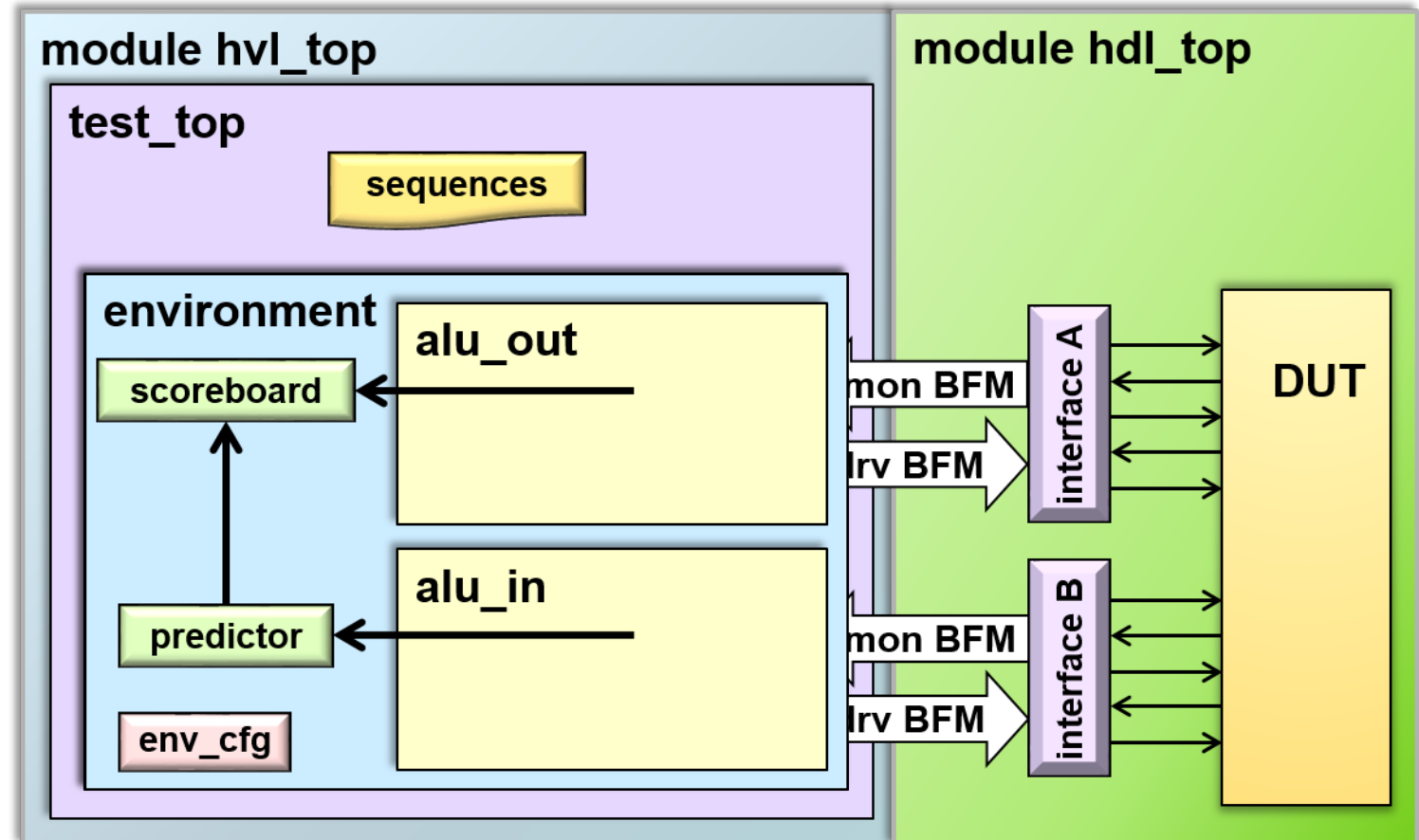
UVM Work: The Environment



Course Flow

What are the UVMF steps?

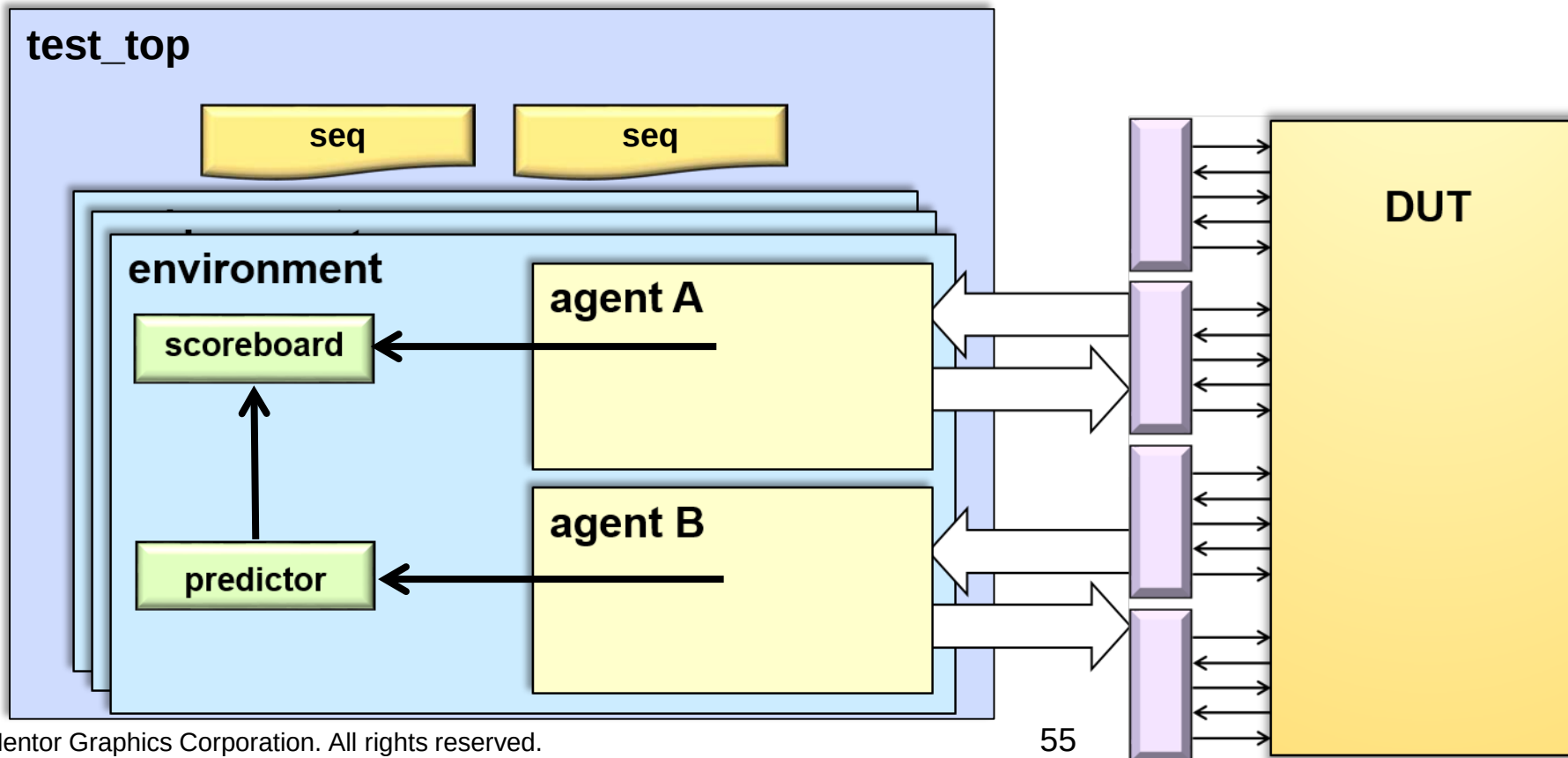
1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*



Refresher: UVM Environment

How to combine stimulus and checks?

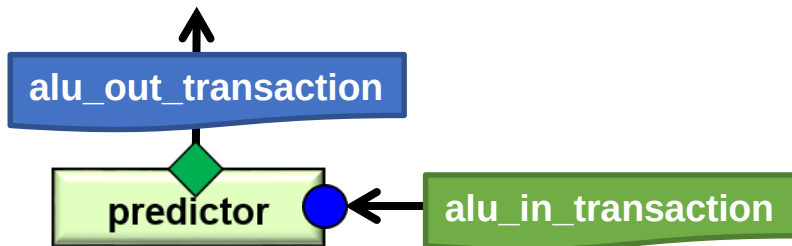
- Creates a self-checking unit
 - Reusable at higher levels too



Predictor

How to turn inputs into outputs?

- Describe a predictor component
 - Input: analysis exports
 - Output: analysis ports
 - Need to fill in **write*()** method



```
# alu_predictor.yaml
uvmf:
  util_components:
    alu_predictor:
      type: predictor
      ● analysis_exports:
        - name: alu_in_agent_ae
          type: "alu_in_transaction"
      ◆ analysis_ports:
        - name: alu_sb_ap
          type: "alu_out_transaction"
```

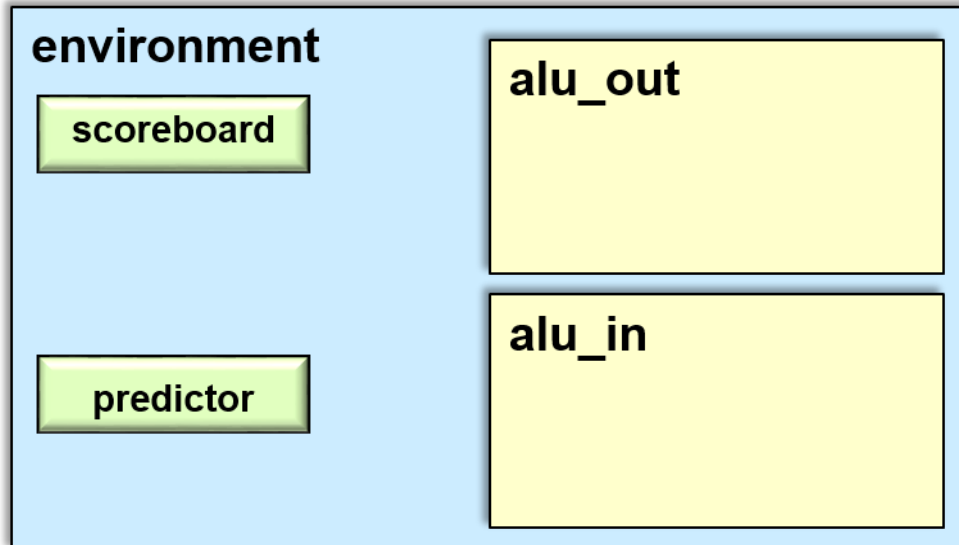


Can have multiple
analysis connections

Environment

How to describe for UVMF?

- Instantiate components
 - Agents
 - Predictor
 - Scoreboard
 - in order
 - out of order



```
# alu_environment.yaml
uvmf:
  environments:
    alu:
      agents:
        - name: alu_in_agent
          type: alu_in
          initiator_responder: "INITIATOR"
        - name: alu_out_agent
          type: alu_out
          initiator_responder: "RESPONDER"

      analysis_components:
        - name: alu_pred
          type: alu_predictor

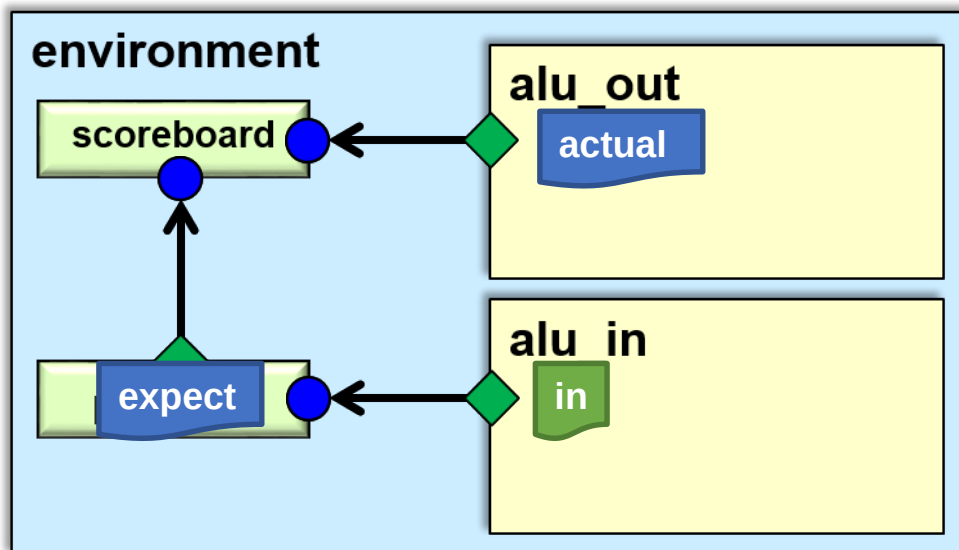
      scoreboards:
        - name: alu_sb
          sb_type: uvmf_in_order_race_scoreboard
          trans_type: alu_out_transaction
        ...
```

Environment

How to describe in YAML?

- Instantiate components
- Describe connections

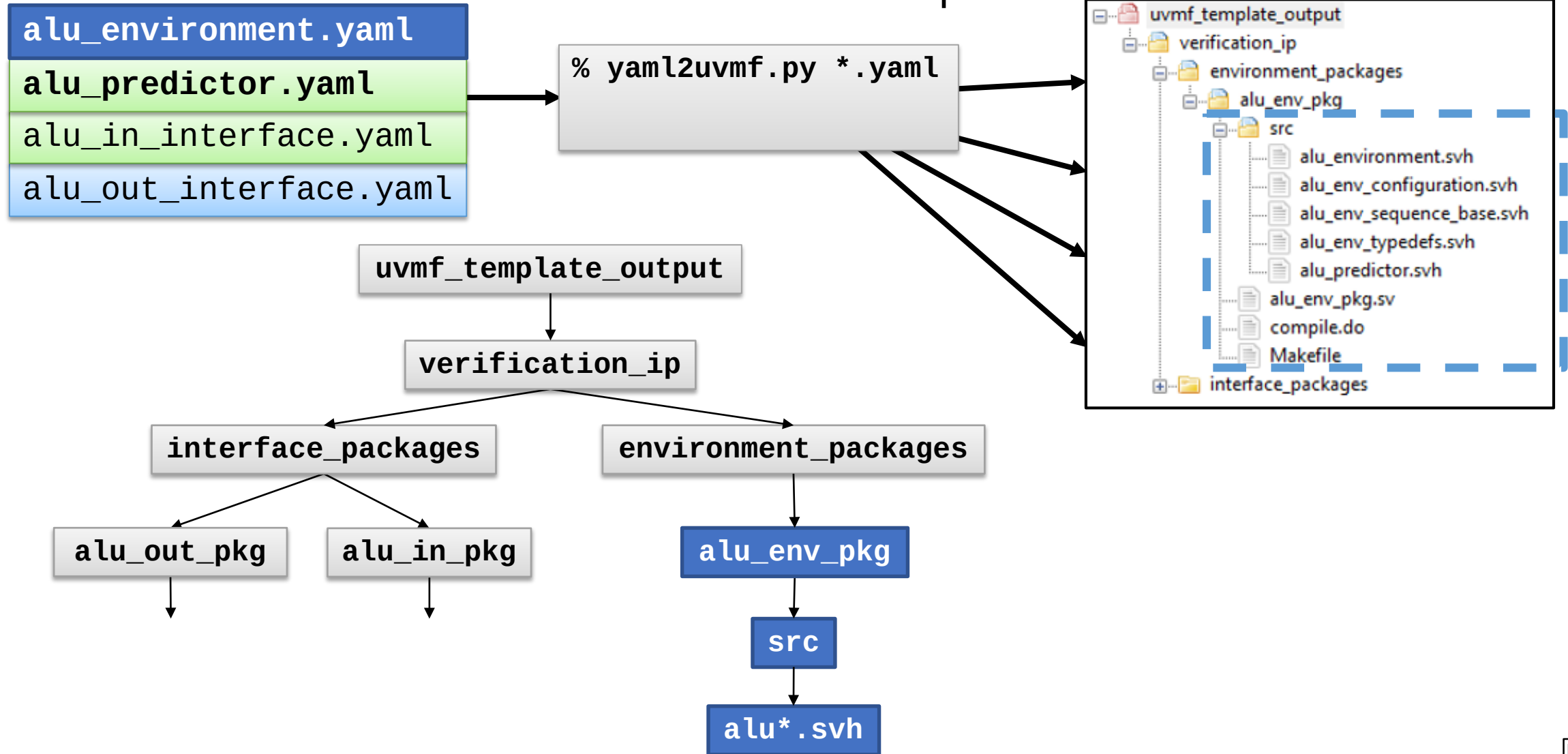
```
# alu_environment.yaml
uvmf:
  environments:
    alu:
      ...
      tlm_connections:
        - driver: alu_in_agent.monitored_ap
          receiver: alu_pred.alu_in_agent_ae
        - driver: alu_pred.alu_sb_ap
          receiver: alu_sb.expected_analysis_export
        - driver: alu_out_agent.monitored_ap
          receiver: alu_sb.actual_analysis_export
```



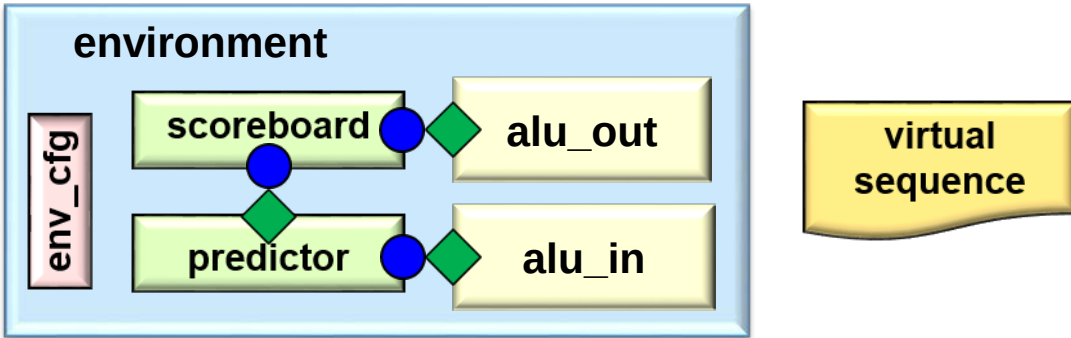
SWORK: The Environment

Environment Generation

What is created from the environment description?



Dive into Generated Code



```
# alu_environment.yaml
uvmf:
  environments:
    alu:
      agents: ...
      analysis_components: ...
      scoreboards: ...
      tlm_connections: ...
```

`verification_ip/environment_packages/alu_env_pkg/src`

```
class alu_env_configuration extends uvmf_environment_configuration_base;
```

```
class alu_env_sequence_base extends uvmf_sequence_base #(uvm_sequence_item);
```

```
class alu_environment extends uvmf_environment_base #(.CONFIG_T(alu_env_configuration));
```

```
class alu_predictor extends uvm_component;
```



Fill in the DUT details

Dive into Generated Files

What is the default predictor code?

- Describe a predictor component
 - Input: analysis export
 - Output: analysis port
 - Need to fill in **write*()** method

```
# alu_predictor.yaml
```

```
uvmf:
```

```
  util_components:
```

```
    alu_predictor:
```

```
      type: predictor
```

```
      analysis_exports:
```

```
- name: alu_in_agent_ae
```

```
  type: "alu_in_transaction"
```

```
      analysis_ports:
```

```
- name: alu_sb_ap
```

```
  type: "alu_out_transaction"
```



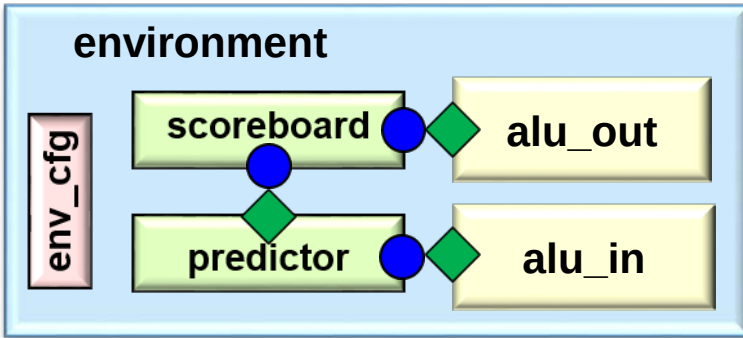
```
class alu_predictor extends uvm_component;
  uvm_analysis_port #(alu_out_transaction) alu_sb_ap;

  virtual function void write_alu_in_agent_ae(alu_in_transaction t);
    alu_sb_ap_output_transaction = alu_out_transaction::type_id::create("...");

    // UVMF_CHANGE_ME: Implement predictor model here.
    `uvm_info("UNIMPLEMENTED_PREDICTOR_MODEL", ... UVM_NONE)

    alu_sb_ap.write(alu_sb_ap_output_transaction);
  endfunction
endclass
```

Dive into Generated Code



```
# alu_environment.yaml
uvmf:
  environments:
    alu:
      agents: ...
      analysis_components: ...
      scoreboards: ...
      tlm_connections: ...
```

verification_ip/environment_packages/alu_env_pkg/src

```
class alu_environment extends uvmf_environment_base #(.CONFIG_T(alu_env_configuration));

function build_phase(
  super.build_phase(
    alu_in_agent = ....
  )
endfunction

function connect_phase(...);
  super.connect_phase(...);
  alu_in_agent.monitored_ap.connect(alu_pred.alu_in_agent_ae);
endfunction
endclass
```

```
class uvmf_environment_base #(type CONFIG_T ...)
  CONFIG_T configuration;
endclass
```

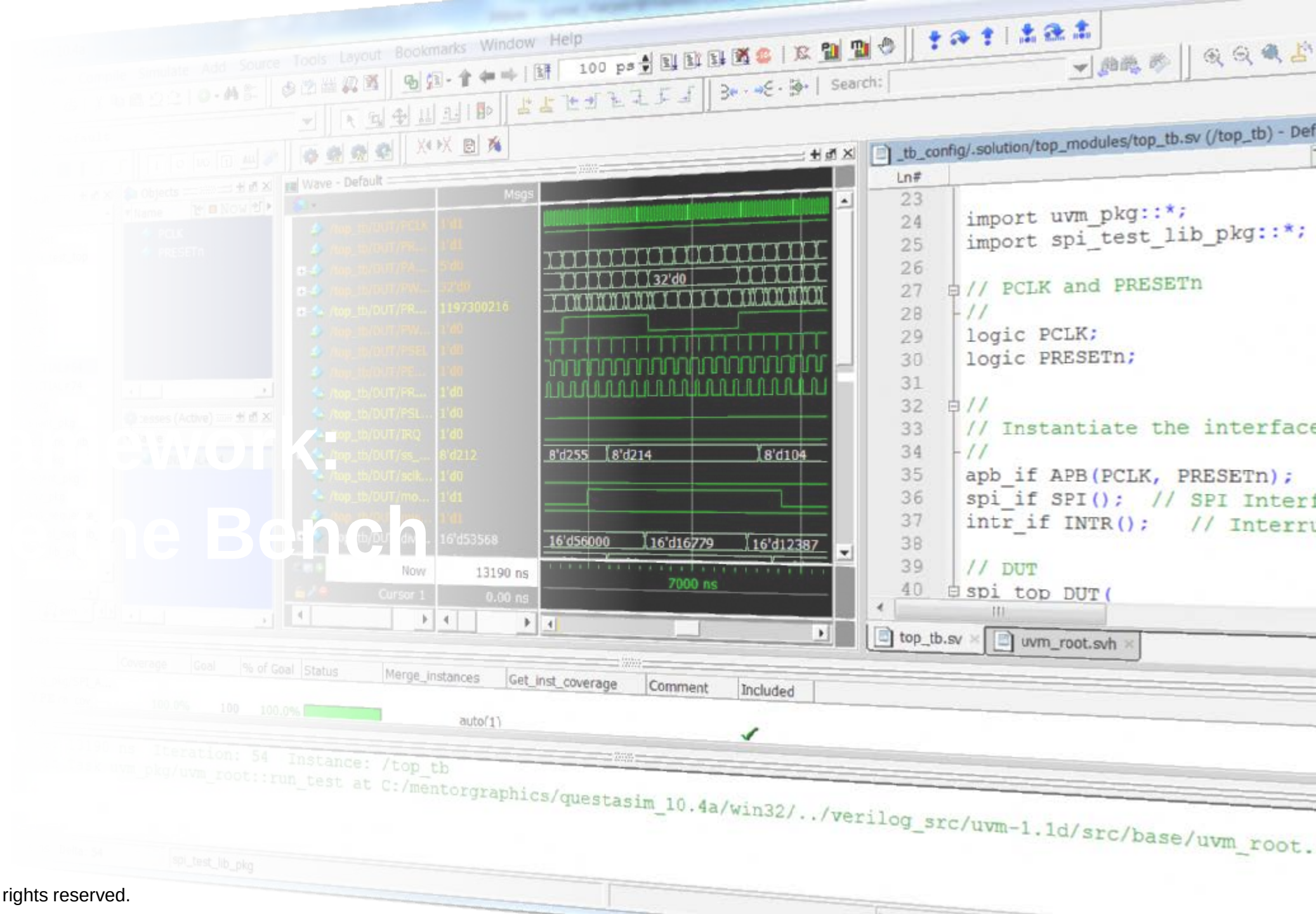
Environment

Knowledge Check – Create Environment



- What are characteristics of a UVM environment?
- Where is an agent specified as an initiator or responder?

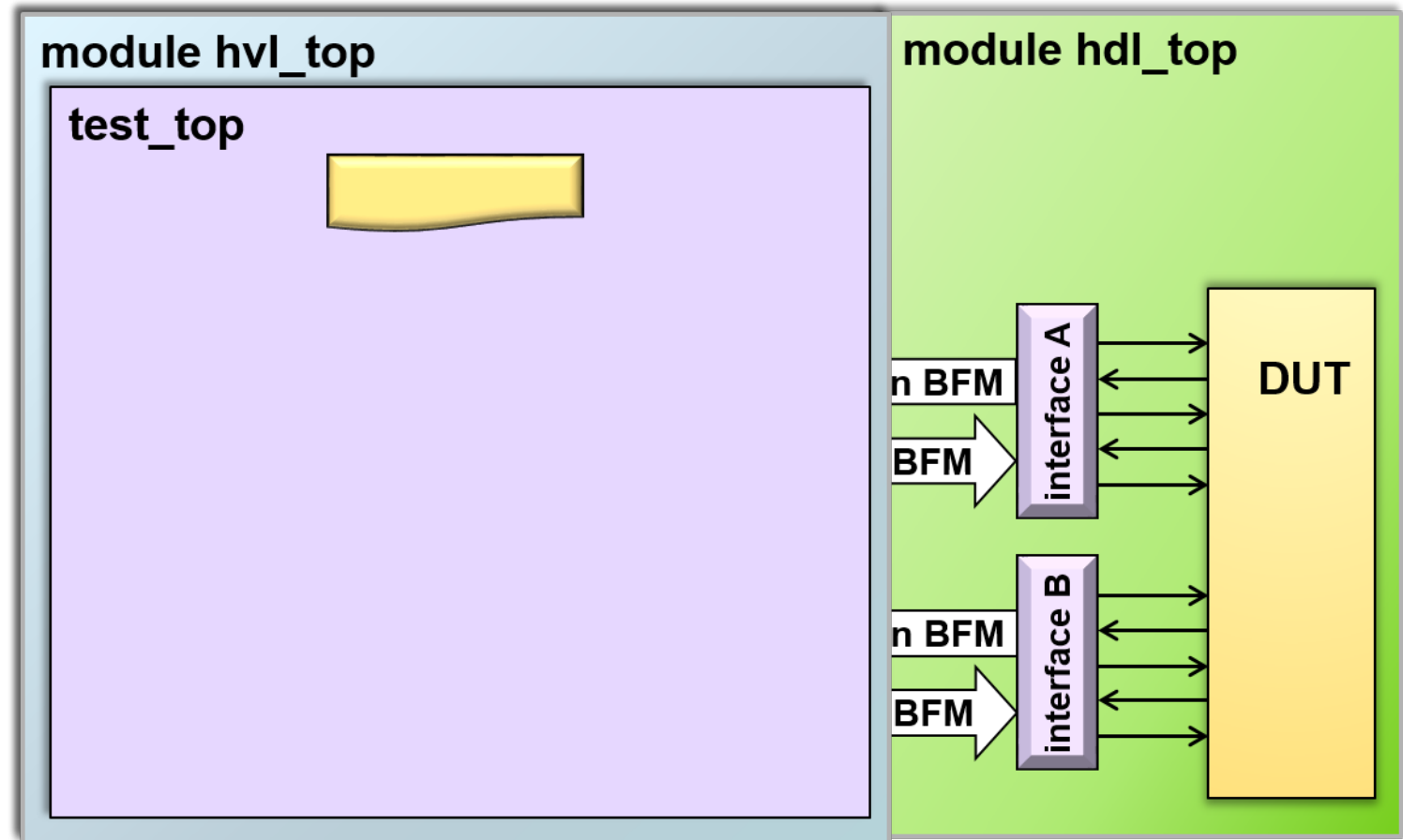




Course Flow

What are the UVMF steps?

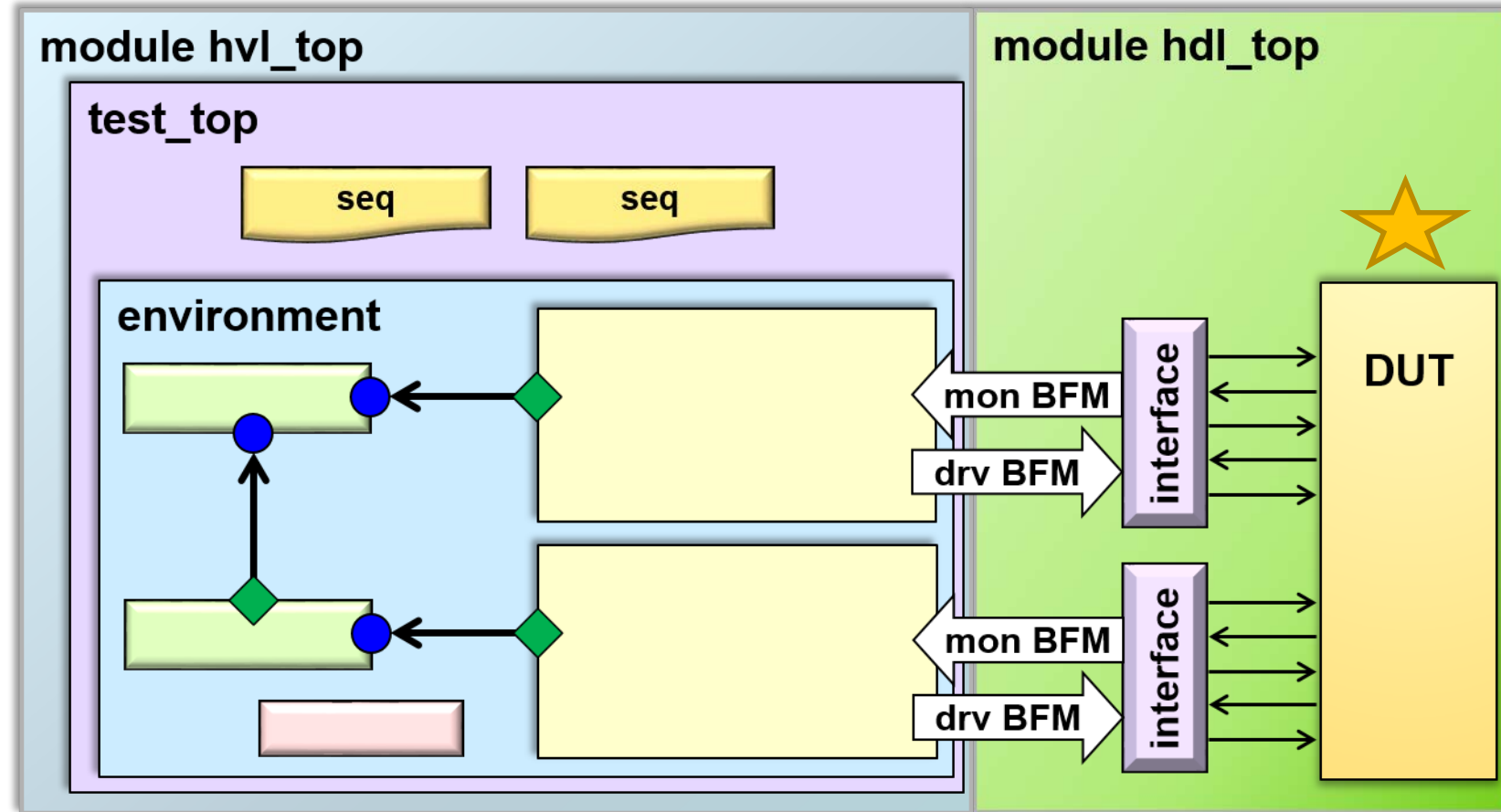
1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*



The Testbench

What is the top of the system?

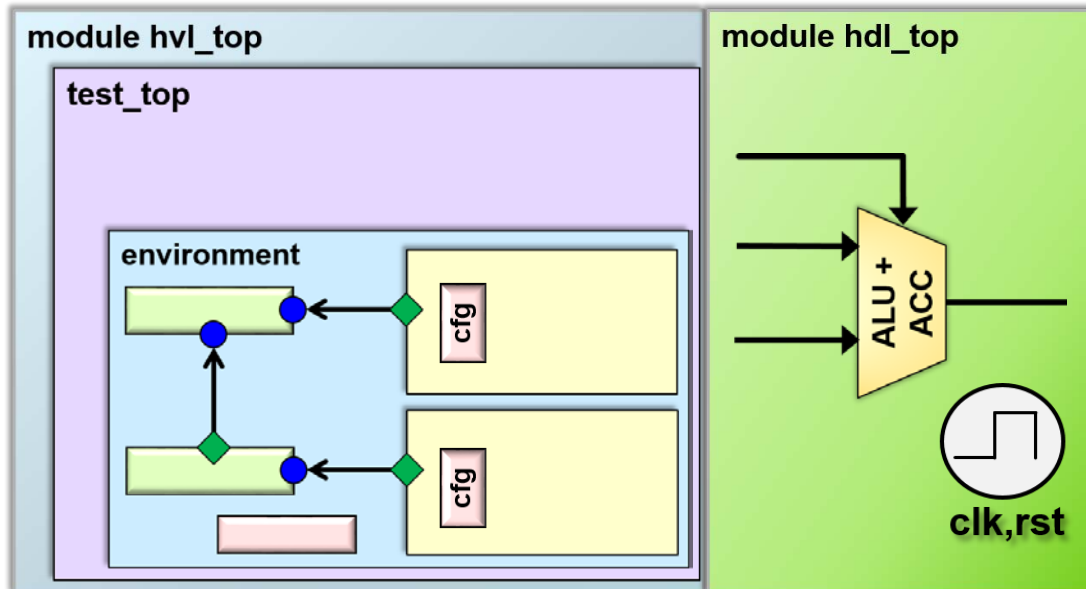
- "UVMF **bench**":
everything above the env
- The test instantiates the env
 - Starts sequences
- All verification code in hvl_top
- All HDL code in hdl_top
 - DUT
 - interfaces
 - BFM
- Some edits required



Bench

How to describe in YAML?

- Clocks & reset
- Environment
- Agent setting



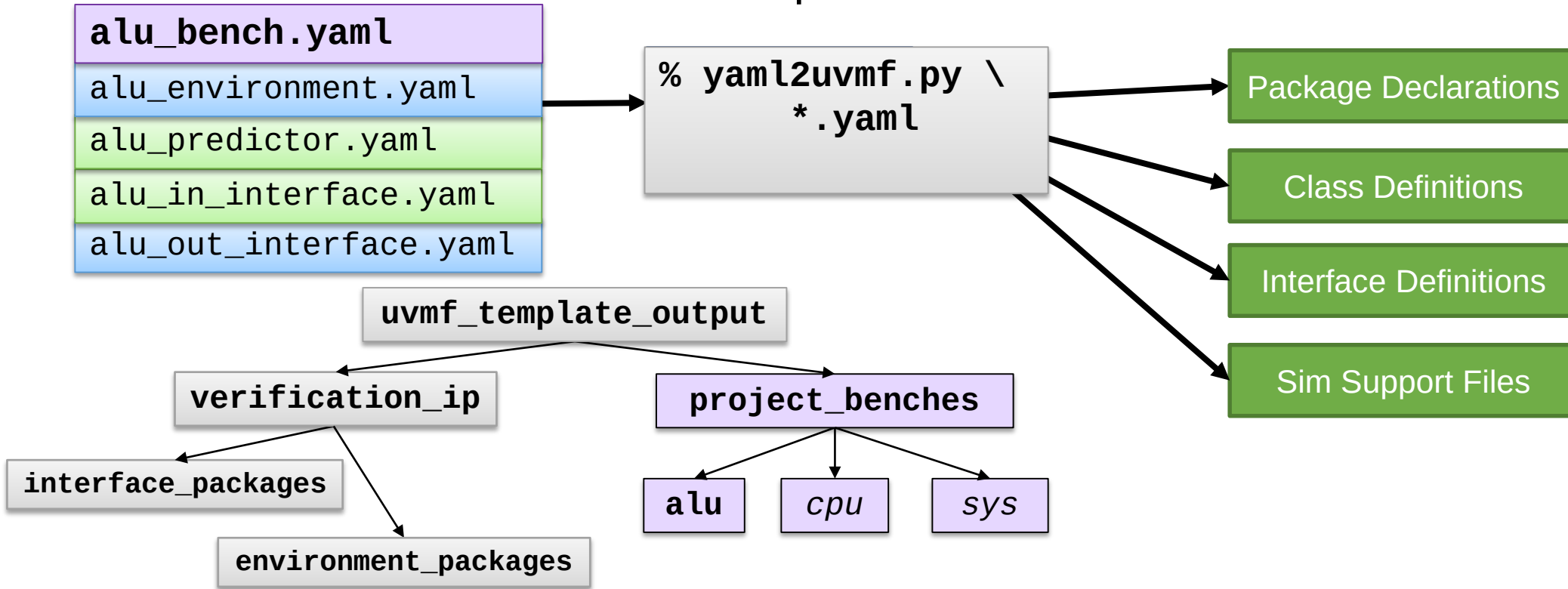
```
# alu_bench.yaml
uvmf:
  benches:
    alu:
      clock_half_period: 5ns
      clock_phase_offset: 9ns
      reset_assertion_level: "False"
      reset_duration: 200ns

  top_env: alu

  active_passive:
    - bfm_name: alu_in_agent
      value: ACTIVE
    - bfm_name: alu_out_agent
      value: ACTIVE
```


UVMF Generator

What is created from this description?

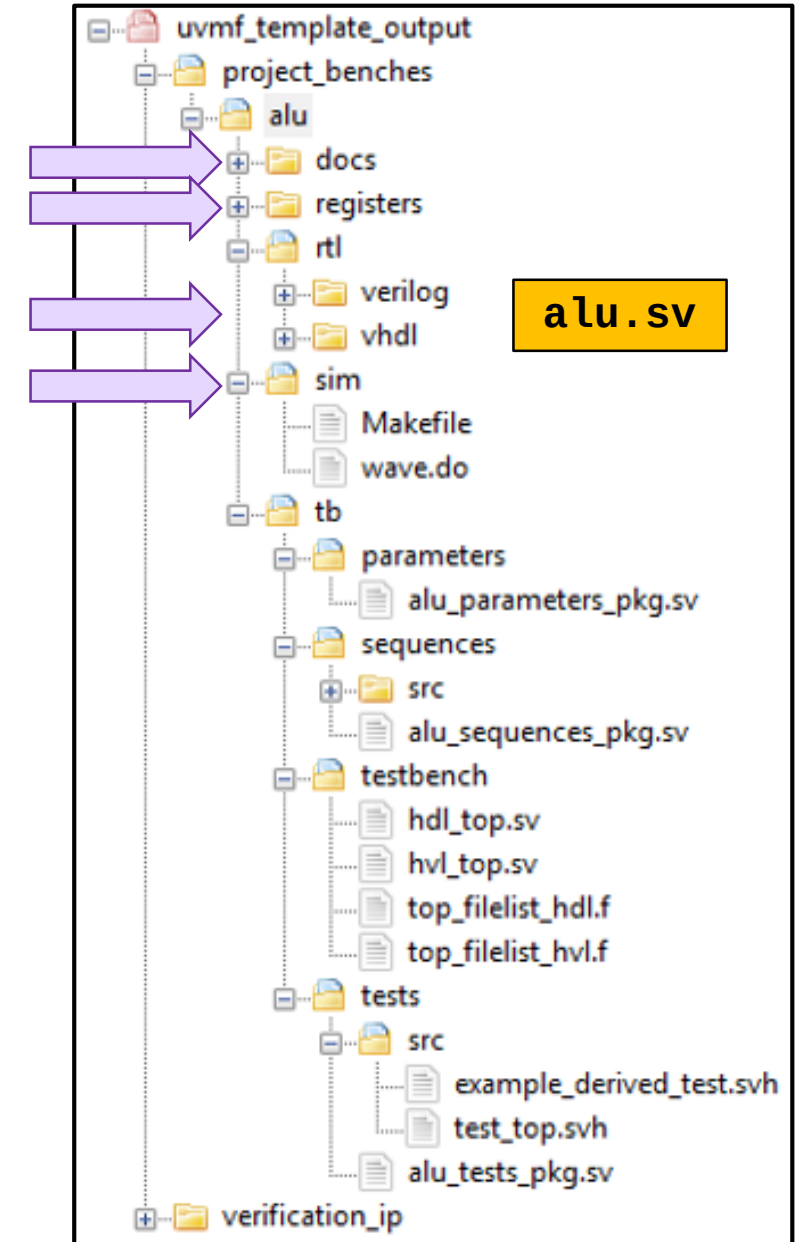


Generated Bench Files

What is created from the bench description?

uvmf_template_output/project_benches/alu

- docs: User created documentation
- registers: UVM Register Layer code
- rtl: RTL, both Verilog and VHDL
- sim: Simulation specific code

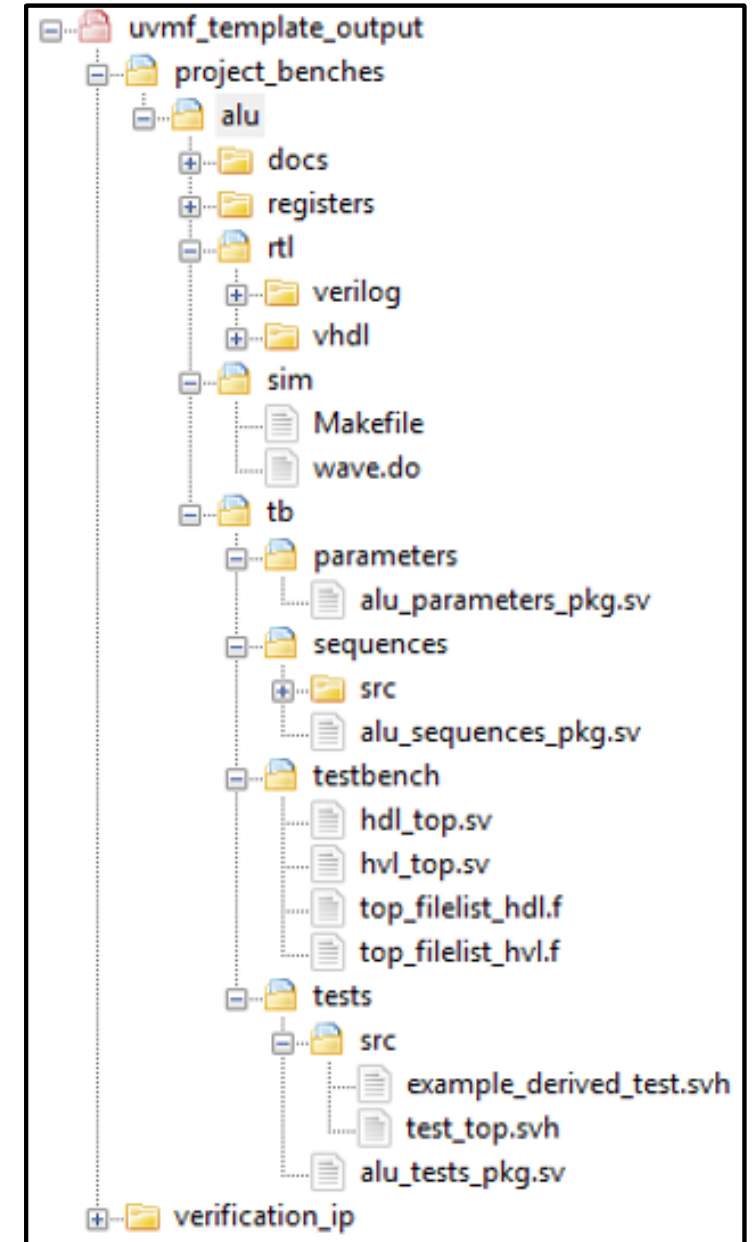


Generated Bench Files

What is created from the bench description?

uvmf_template_output/project_benches/alu

- docs: User created documentation
- registers: UVM Register Layer code
- rtl: RTL, both Verilog and VHDL
- sim: Simulation specific code
- tb:
 - parameters
 - sequences
 - src/alu_bench_sequence_base.svh
 - testbench
 - hdl_top.sv
 - hvl_top.sv
 - tests
 - src/test_top.sv



Default Test

project_benches/alu/tb/tests/src/test_top.svh

- Extended from **uvmf_test_base**
- Parameterized with config, env, and sequence
- Configuration & environment created in **super.build_phase()**
- Configuration initialized in **build_phase()**
- Included in **project_benches/alu/tb/tests/alu_test_pkg.sv**

```
27 class test_top extends uvmf_test_base #(.CONFIG_T(alu_env_configuration_t),
28                                     .ENV_T(alu_environment_t),
29                                     .TOP_LEVEL_SEQ_T(alu_bench_sequence_base));
30     `uvm_component_utils( test_top );
31
63 virtual function void build_phase(uvm_phase phase);
64     super.build_phase(phase);
65     configuration.initialize(BLOCK, "uvm_test_top.environment", interface_names, null, interface_activities);
66 endfunction
67
68 endclass
```


Default Test-Level Virtual Sequence

project_benches/alu/tb/sequences/src/alu_bench_sequence_base.svh

```
23 class alu_bench_sequence_base extends uvmf_sequence_base #(uvm_sequence_item);
24
74 virtual task body();
75     // Construct sequences here
76     alu_in_agent_random_seq      = alu_in_agent_random_seq_t::type_id::create("alu_in_agent_random_seq");
77     alu_out_agent_responder_seq  = alu_out_agent_responder_seq_t::type_id::create("alu_out_agent_responder_seq");
78     fork
79         alu_in_agent_config.wait_for_reset();
80         alu_out_agent_config.wait_for_reset();
81     join
82     // Start RESPONDER sequences here
83     fork
84         alu_out_agent_responder_seq.start(alu_out_agent_sequencer);
85     join_none
86     // Start INITIATOR sequences here
87     fork
88         repeat (25) alu_in_agent_random_seq.start(alu_in_agent_sequencer);
89     join
90     // UVMF_CHANGE_ME : Extend the simulation XXX number of clocks after
91     // the last sequence to allow for the last sequence item to flow
92     // through the design.
93     fork
94         alu_in_agent_config.wait_for_num_clocks(400);
95         alu_out_agent_config.wait_for_num_clocks(400);
96     join
97     endtask
98
99 endclass
```

**Synchronize w/DUT
through agent config**

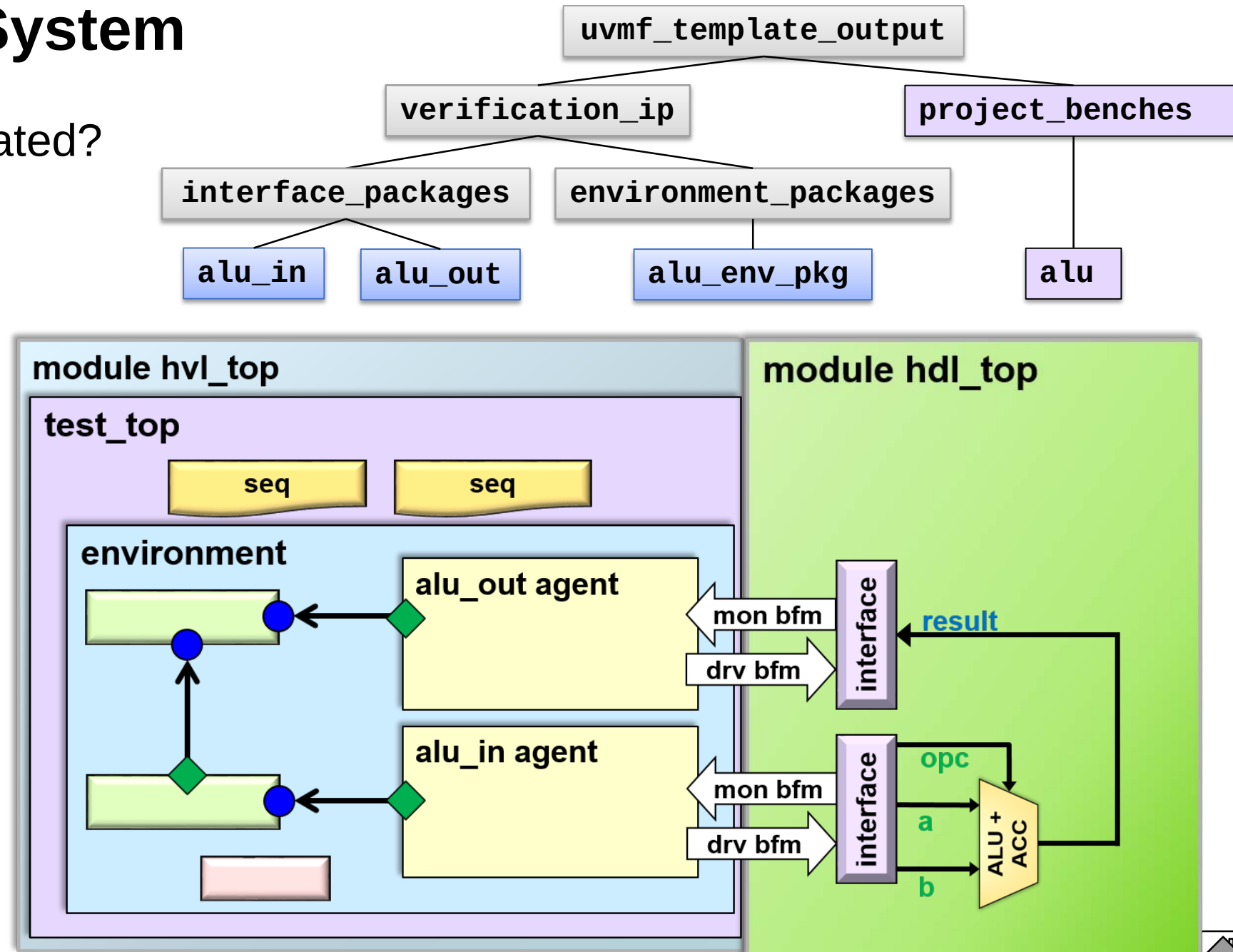


UVM Framework System

What has been generated?

- Protocol interfaces
- Environment
- Bench

4. Create testbench



Knowledge Check – Create Bench

- What is the name of the module which holds the RTL code??
- Which of these have virtual interface variables?

Lab 2 – Create the Environment and Bench

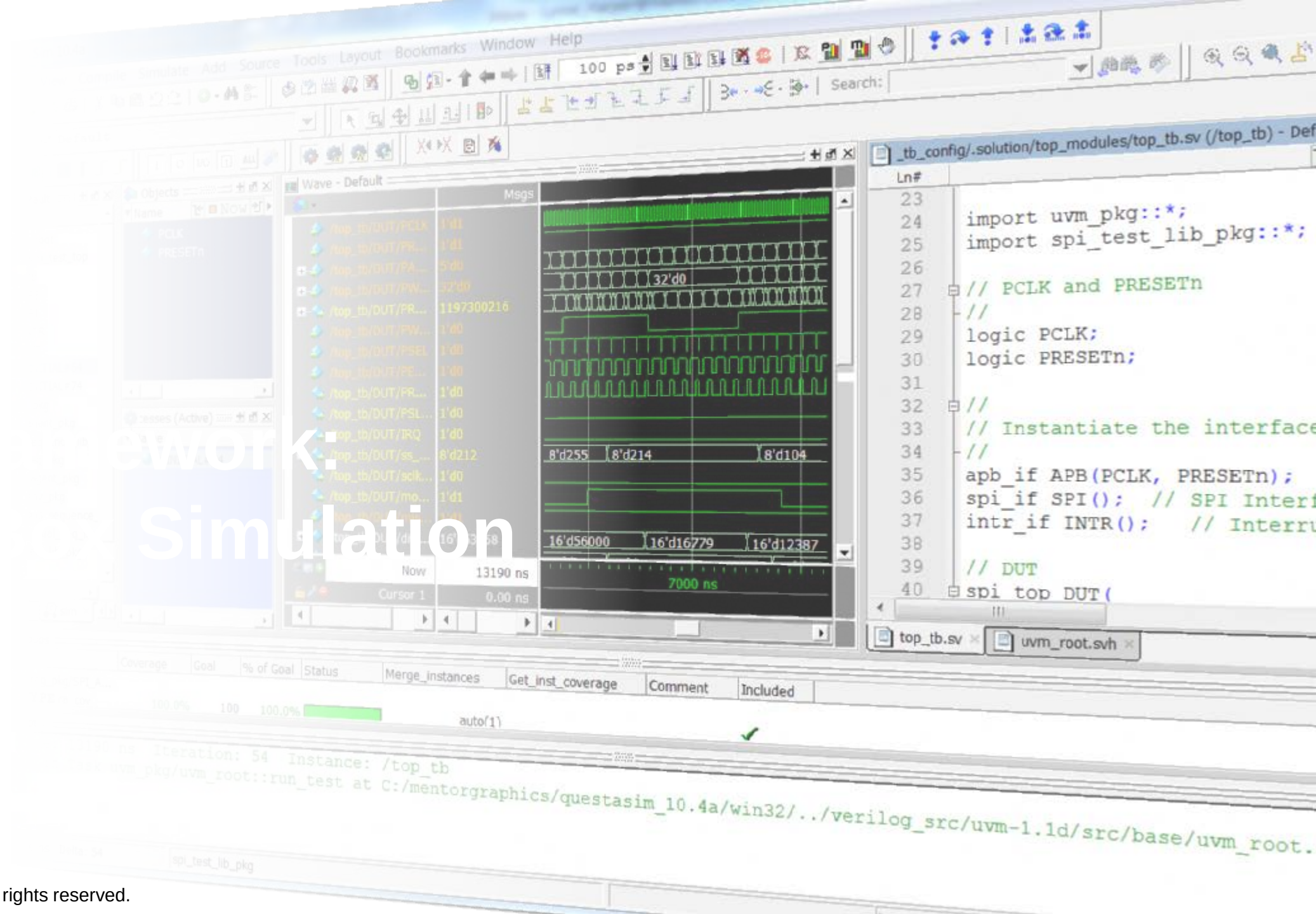


Goal: Create environment and bench description

- Create **alu_predictor.yaml** and **alu_environment.yaml**
- Create **alu_bench.yaml**
- Generate and explore resulting code



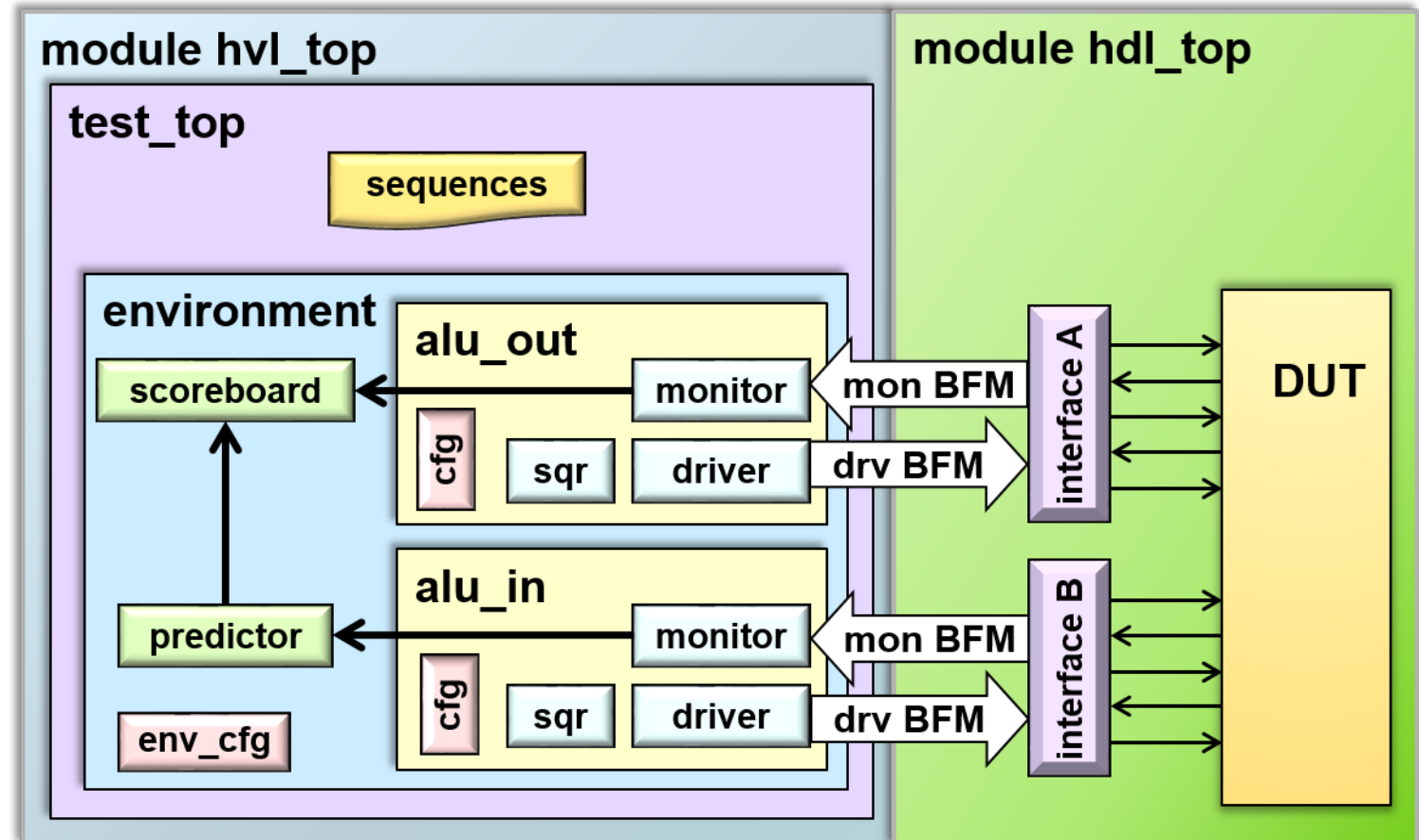
UVM Work: Simulation



Course Flow

What are the UVMF steps?

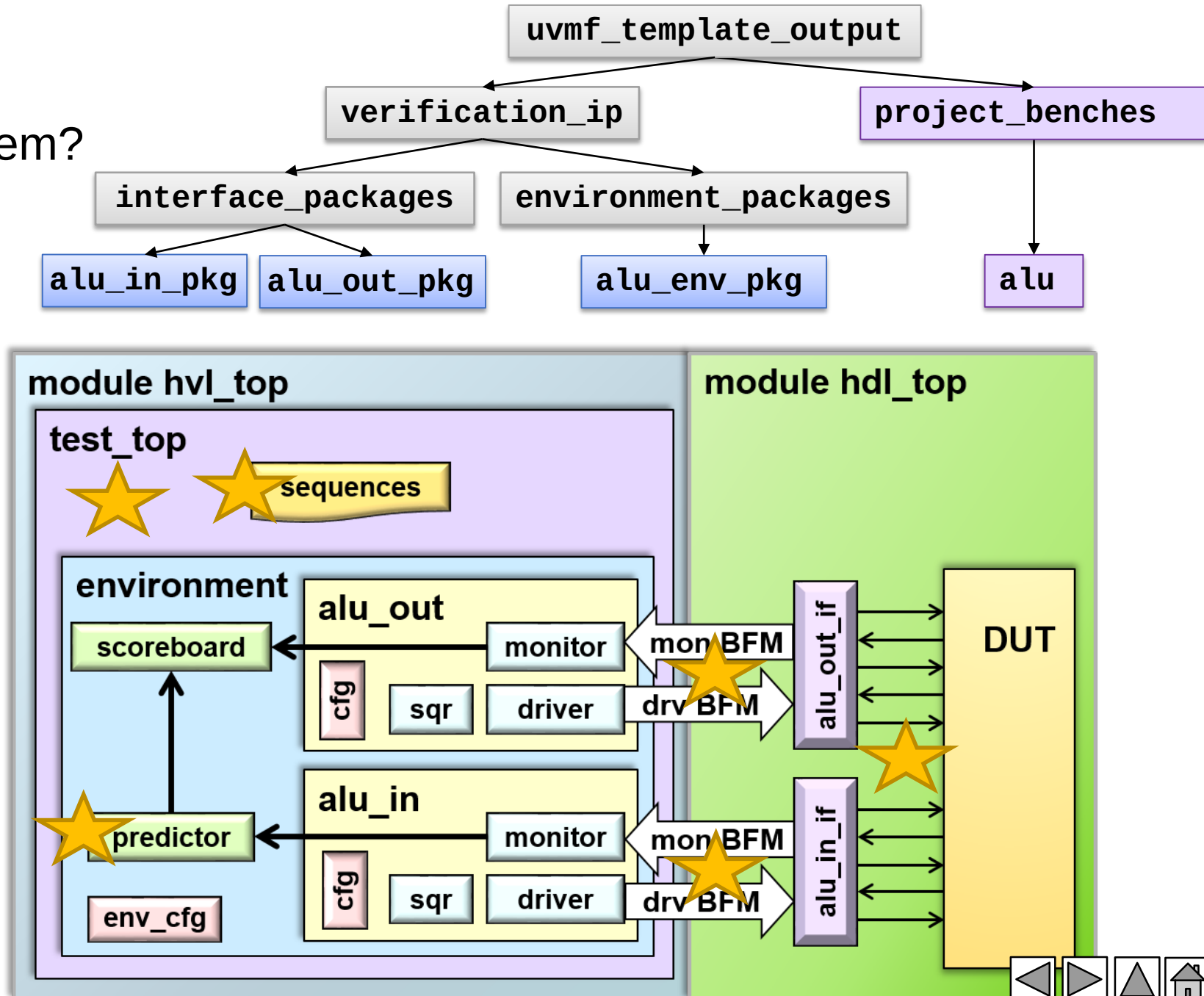
1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*



Next Steps

How to complete the system?

- Generated code
- Modify
 - Connect DUT
 - BFM
 - Predictor
- Add new
 - Tests
 - Sequences
- Work incrementally



Out of the Box Simulation

How to complete the testbench?

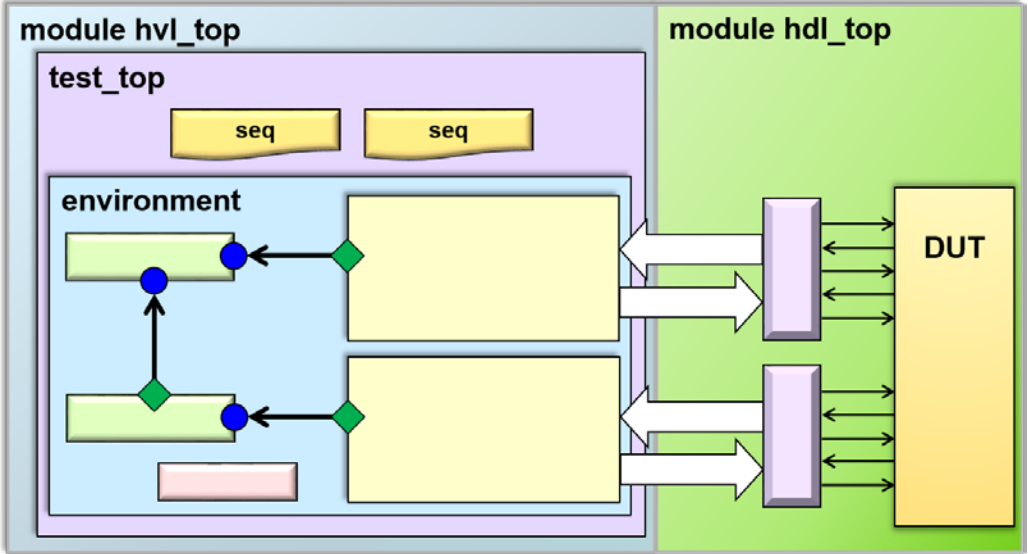
- Demonstrate the code generated by UVM Framework, with no modifications



Out of the Box Simulation

What is simulated by default?

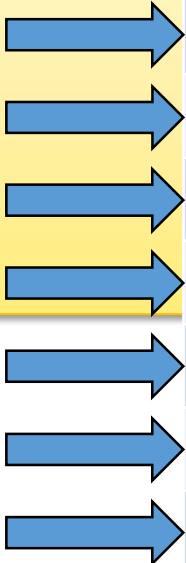
- Complete system compilation
- Simulate with Questa GUI:



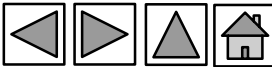
```
% cd uvmf_template_output/project_benches/alu/sim
```

```
% make debug
```

```
...  
vsim optimized_debug_top_tb \  
+UVM_TESTNAME=test_top \  
-do "run 0 ; do wave.do; ..." \  
-sv_seed random \  
...
```



make target	Action
debug	Compile & run with GUI
run_gui	Run with GUI
cli	Compile & run in batch
run_cli	Run in batch
clean	Clean up files
TEST_SEED= 1	Set random seed= 1
TEST_NAME= test	Run UVM test test



UVMF Generator

How to control the generated files?

- For more details on YAML and generator
\$UVMF_HOME/doc

UVM Framework
Code Generator YAML
Reference

UVMF_Code_Generator_YAML_Reference.pdf

UVMF App Note

Refactoring and Reusing Code Using the Generator

App_Note_Regeneration.pdf

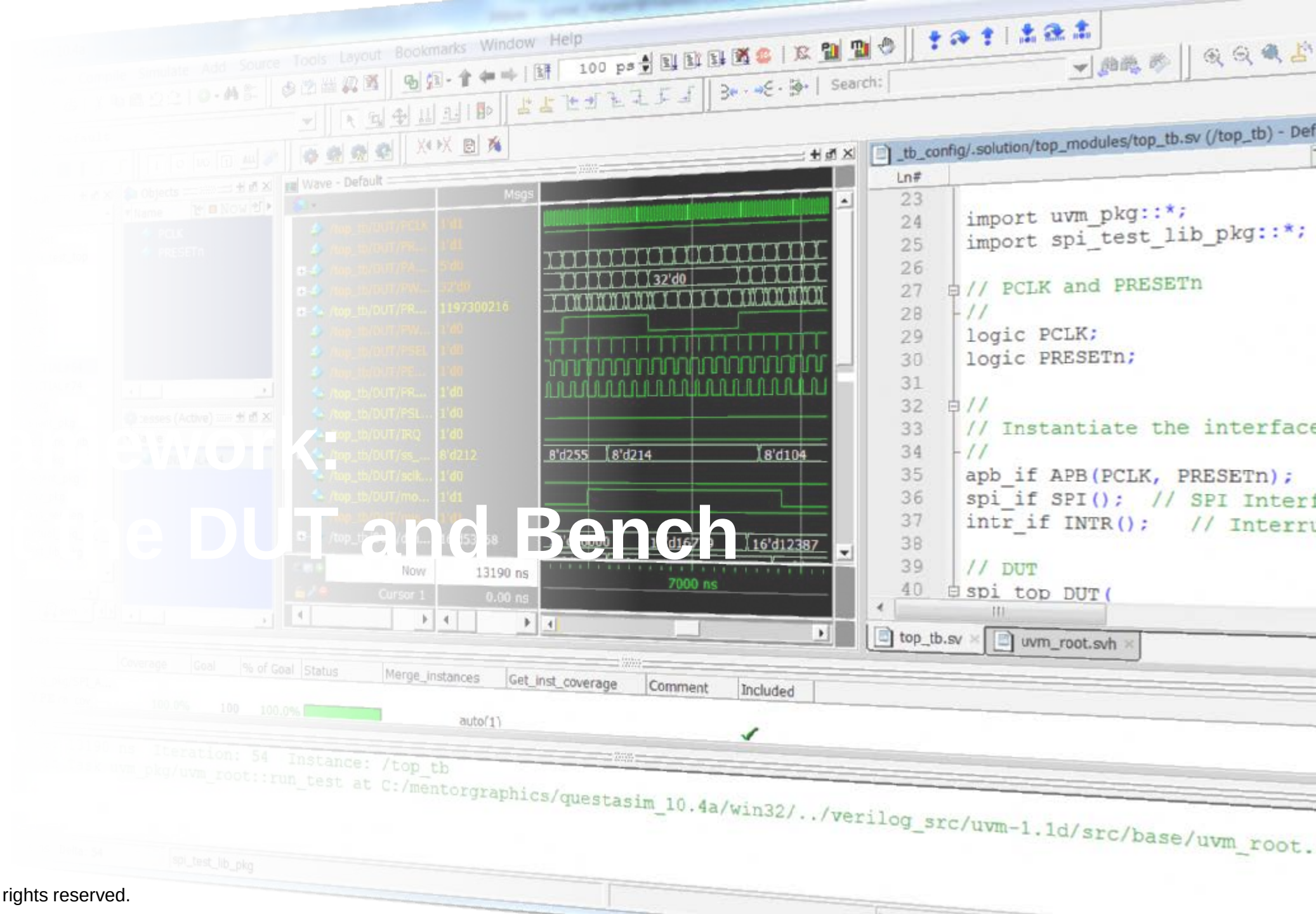
Lab 3 – Out of the Box Simulation



Goal: Generate and simulate the default UVMF verification system

- Compile the system from Lab 3 with Questa, looking for warnings and errors
- Simulate the system and explore the components

UVM Work: The DUT and Bench



Steps to Connect DUT and Bench

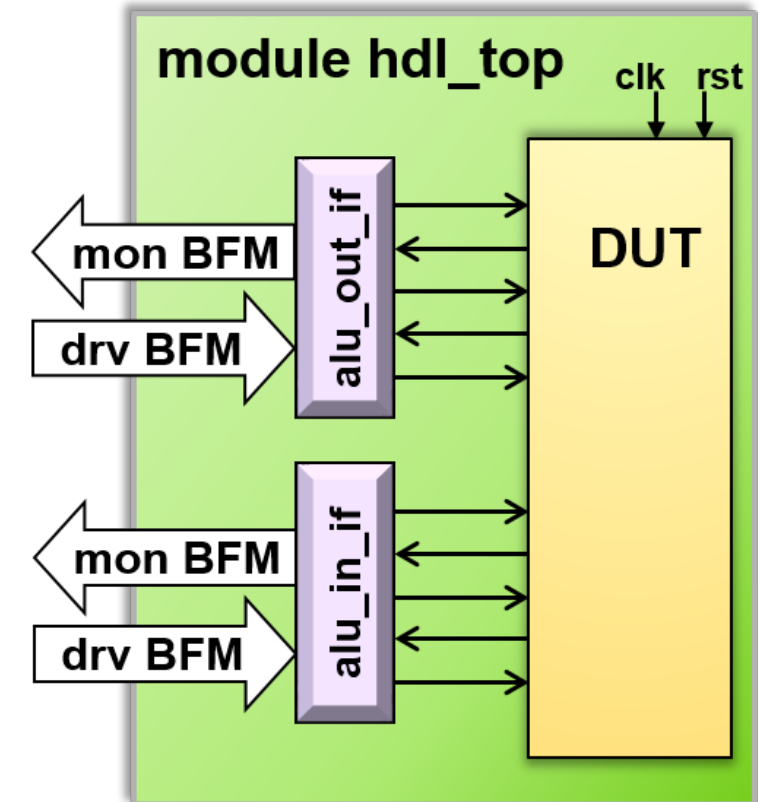
What modifications are needed to the generated UVMF files?

1. Update **hdl_top.sv**
2. Update **Makefile**
3. Compile & run
4. Check the results
 - Look for warnings or errors
 - Observe signal values

Instantiate and Connect the DUT

project_benches/alu/tb/testbench/hdl_top.sv

```
// UVMF_CHANGE_DONE: Add DUT and connect to signals in _bus interfaces
alu DUT(.clk      (clk),
        .rst      (rst),
        .opc      (alu_in_agent_bus.opc),
        .a        (alu_in_agent_bus.a),
        .b        (alu_in_agent_bus.b),
        .rdy_in   (alu_in_agent_bus.rdy_in),
        .ack_in   (alu_in_agent_bus.ack_in),
        .result   (alu_out_agent_bus.result),
        .rdy_out  (alu_out_agent_bus.rdy_out),
        .ack_out  (alu_out_agent_bus.ack_out));
```



Compile and Simulate with the DUT

project_benches/alu/tb/sim/Makefile

- Update DUT name in the Makefile

```
# UVMF_CHANGE_ME : Reference Verilog DUT source.  
alu_VERILOG_DUT =\  
$(UVMF_PROJECT_DIR)/rtl/verilog/verilog_dut.v
```



```
# UVMF_CHANGE_DONE : Reference Verilog DUT source.  
alu_VERILOG_DUT =\  
$(UVMF_PROJECT_DIR)/rtl/verilog/alu.sv
```

- Remove VHDL target

```
# UVMF_CHANGE_ME : Add make target to compile your dut here  
comp_alu_dut: comp_alu_verilog_dut comp_alu_vhdl_dut
```



```
# UVMF_CHANGE_DONE : Add make target to compile your dut here  
comp_alu_dut: comp_alu_verilog_dut
```

- Change build order for HDL/HVL

```
make_build: comp_alu_dut comp_uvmf_core comp_hvl comp_test_bench
```



```
make_build: comp_uvmf_core comp_hvl comp_alu_dut comp_test_bench
```



Simulation Results

```
% cd uvmf_template_output/project_benches/alu/tb/sim
```

```
% make debug
```

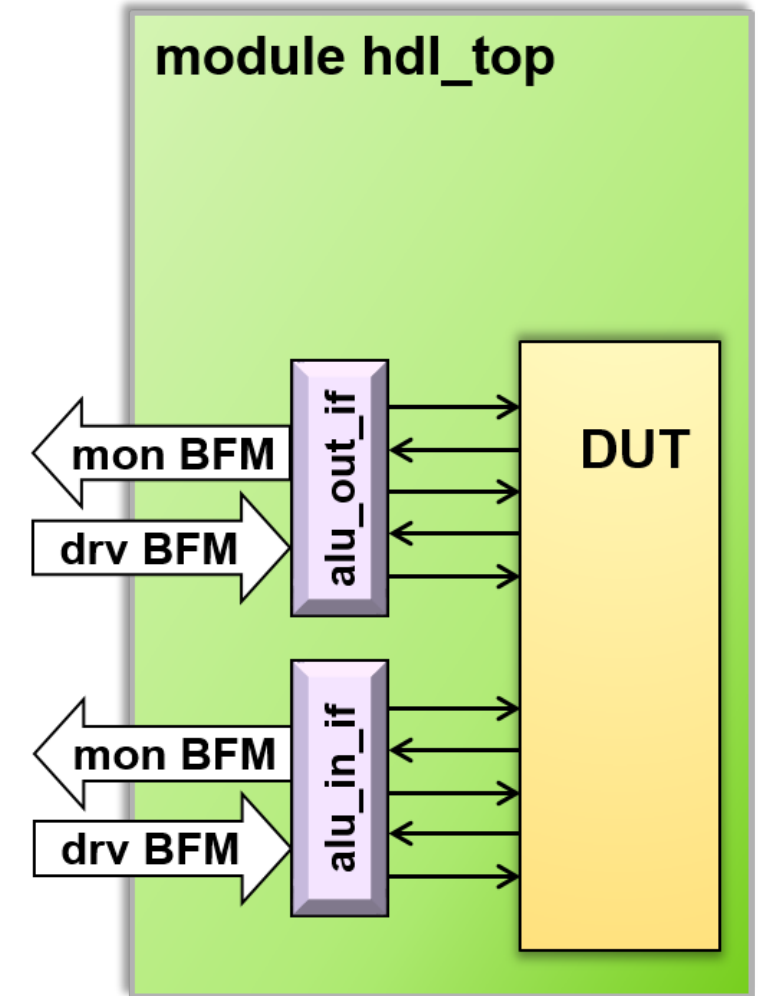
- **alu_in_agent/ack_in** – now driven by ALU DUT
- **alu_out_agent/result** and **rdy_out** – now driven by ALU DUT



Testbench Topology

Why does the DUT instantiation need to be hand-edited?

- **hdl_top.sv** has basic topology with interfaces and BFM
- Is the DUT VHDL, Verilog, or SystemVerilog?
- Name changes across boundaries
- Internal signals need SystemVerilog bind
- Multiple clocks and reset
- Some ports are outside the YAML description



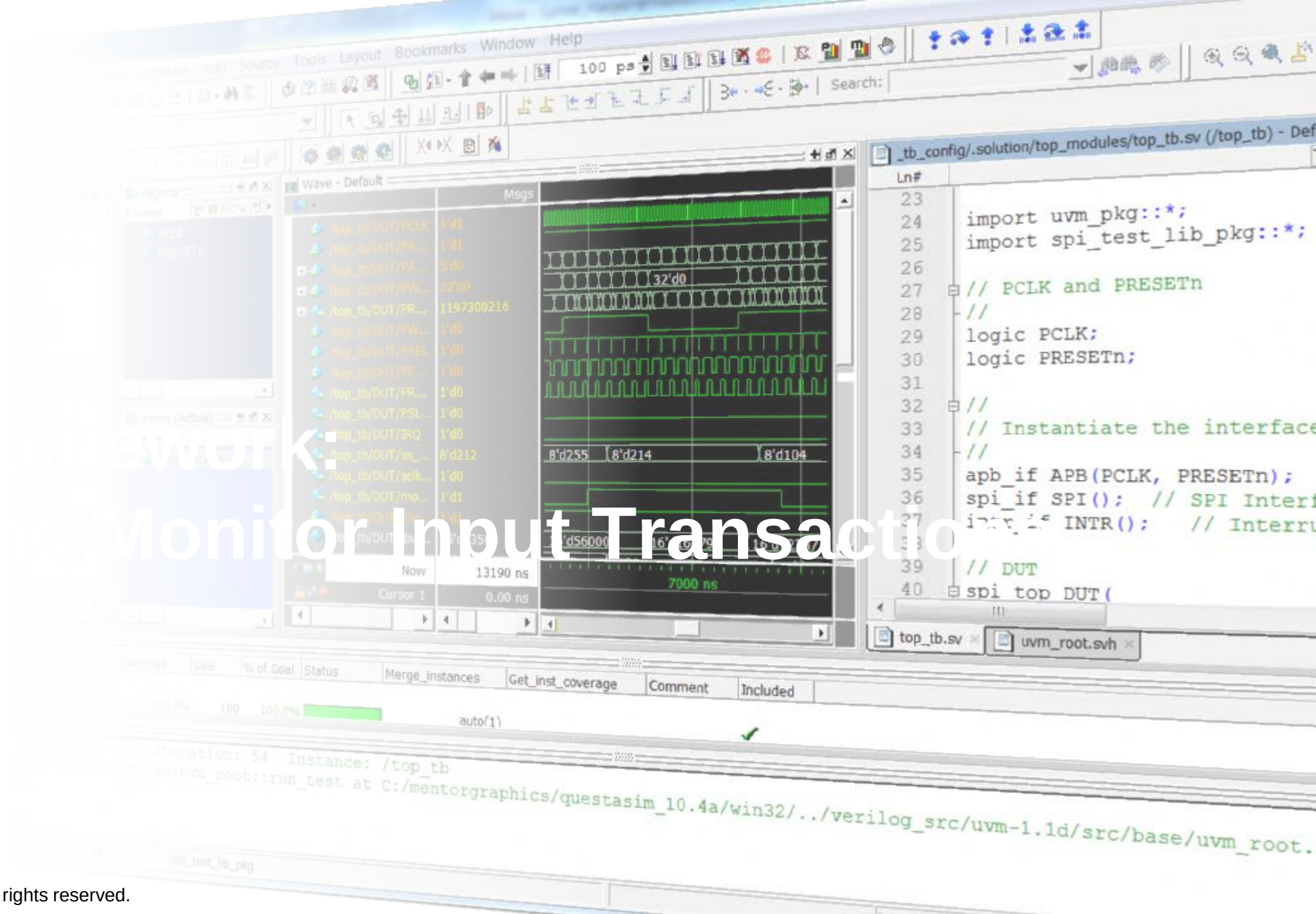
Lab 4 – Connect the DUT and Bench



Goal: Instantiate the DUT in the UVMF testbench

- Edit **hdl_top.sv**
- Edit **Makefile**
- Compile, simulate, and examine signals

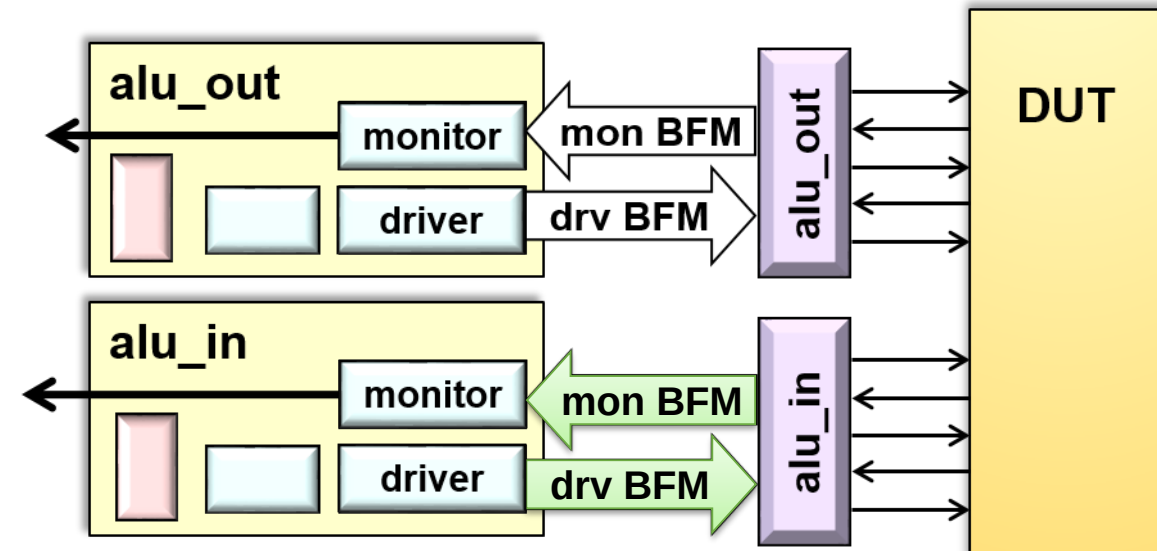
How to Monitor Input Transaction



Drive and Monitor with the Input Protocol

How does a bench send a transaction to the DUT?

- UVM classes access the DUT through BFM's for reusability
 - Never directly access DUT signals
- The BFMs contain drive & monitor methods
 - Fill in the protocol details



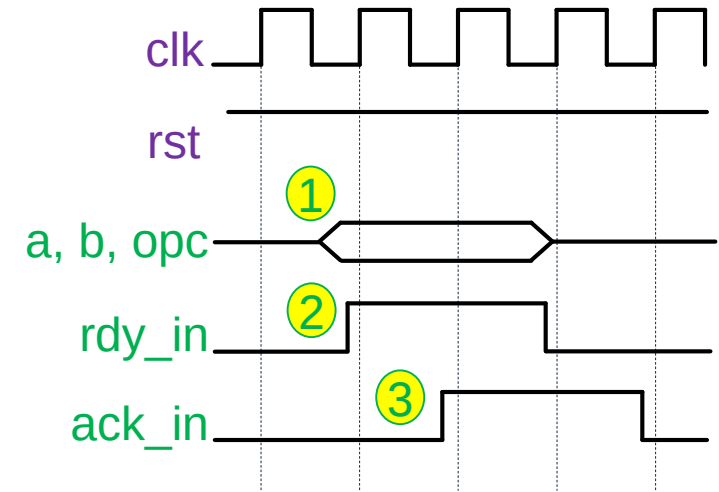
Input Driver: `initiate_and_get_response()`

How to send transactions into the DUT?

`verification_ip/interface_packages/alu_in_pkg/src/alu_in_driver_bfm.sv`



```
task initiate_and_get_response(  
    input alu_in_initiator_s alu_in_initiator_struct,  
    output alu_in_responder_s alu_in_responder_struct);  
  
    @(posedge clk_i); // Always start with a clock edge  
  
    $display("%t alu_in_driver_bfm:initiate_and_get_response %p",  
            $realtime, alu_in_initiator_struct);  
  
    // Wait for the previous transaction to complete  
    while (ack_in_i == 1'b1)  
        @(posedge clk_i);  
  
    // Drive the transaction into the DUT  
    ① opc_o    <= alu_in_initiator_struct.opc;  
    a_o       <= alu_in_initiator_struct.a;  
    b_o       <= alu_in_initiator_struct.b;  
    ② rdy_in_o <= 1'b1;  
  
    // Wait for DUT acknowledge, then complete the handshake  
    ③ do @(posedge clk_i);  
    while (ack_in_i == 1'b0);  
    rdy_in_o <= 1'b0;  
endtask
```



The initiate_and_get_response() task

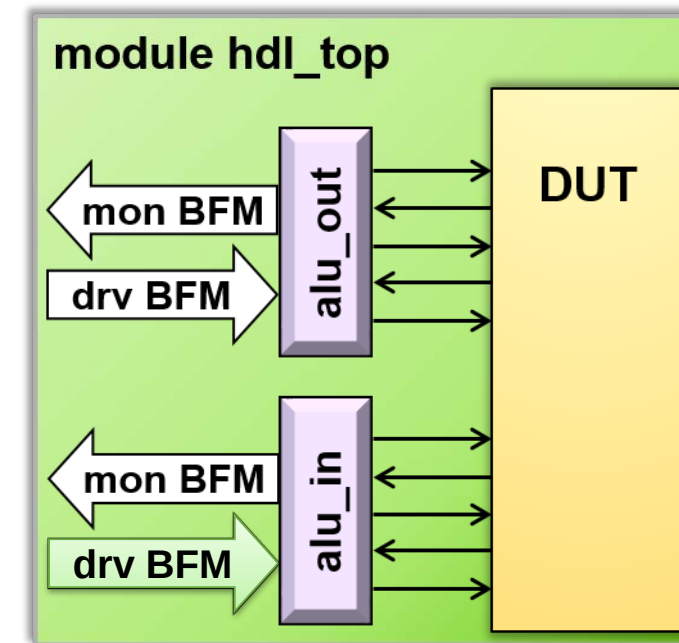
More details on BFM initiator task

verification_ip/interface_packages/alu_in_pkg/src/alu_in_driver_bfm.sv

```
interface alu_in_driver_bfm (alu_in_if bus);
```

```
task initiate_and_get_response(  
    input alu_in_initiator_s alu_in_initiator_struct,  
    output alu_in_responder_s alu_in_responder_struct);  
  
    @(posedge clk_i); // Always start with a clock edge  
  
    $display("%t alu_in_driver_bfm:initiate_and_get_response %p",  
            $realtime, alu_in_initiator_struct);  
    ...  
endtask
```

- XRTL: start by synchronizing to active clock edge
- Call `$display()` in the BFMs, outside of the UVM classes

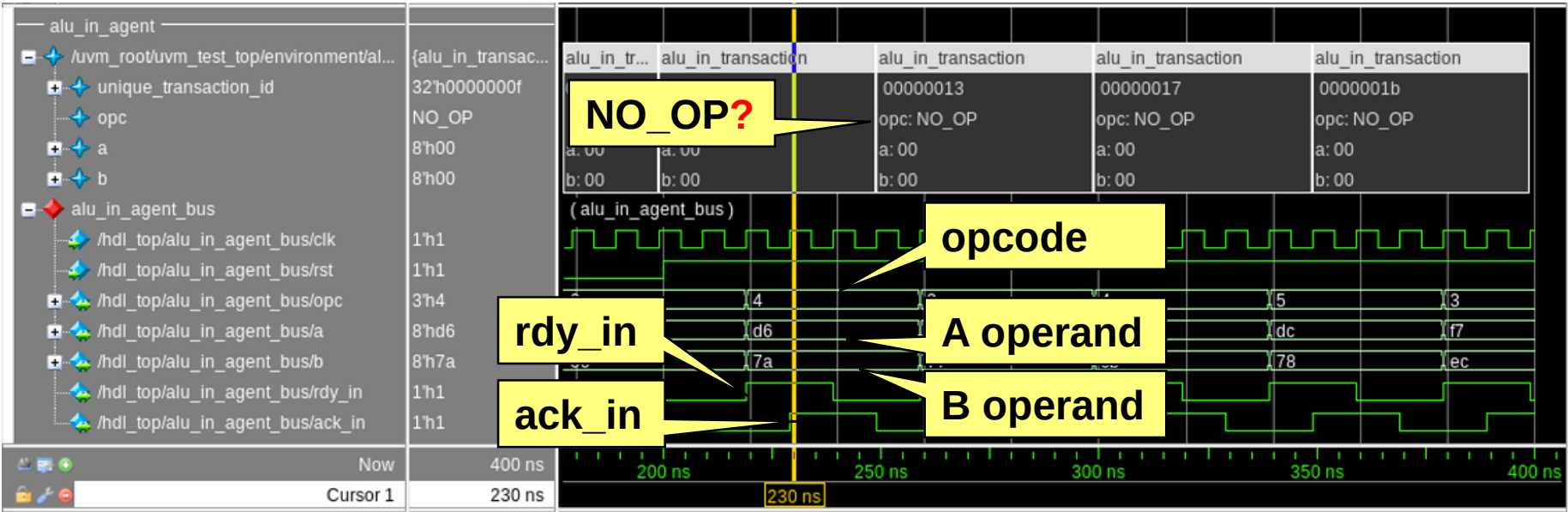


Simulation Results

Testbench drives transactions

```
alu_in_driver_bfm::initiate_and_get_response()
```

```
% make debug TEST_SEED=1
```

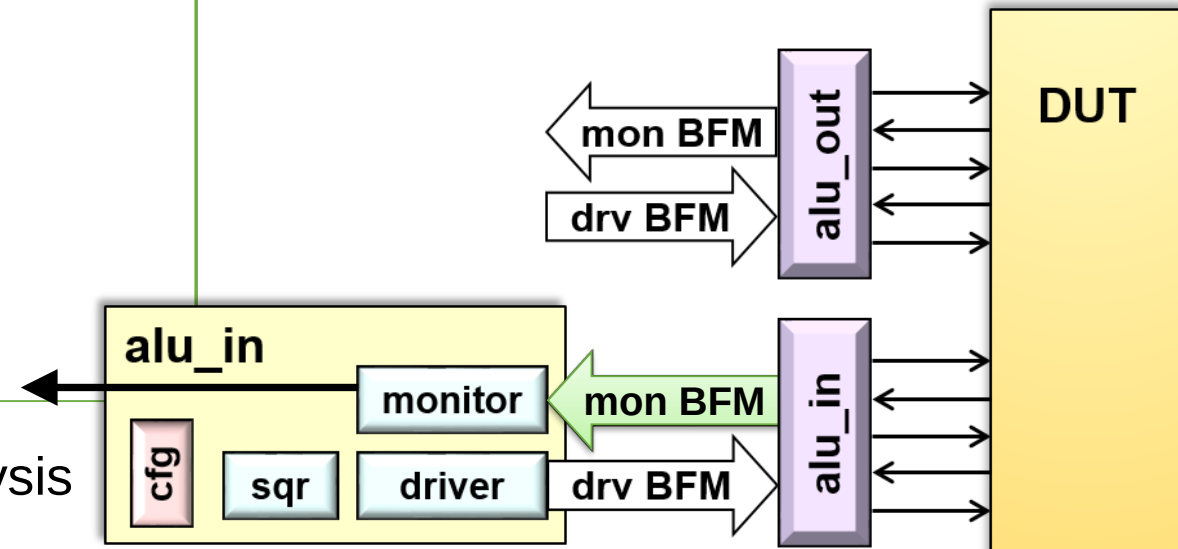


Input Monitor: do_monitor()

How to capture the input transactions for analysis?

verification_ip/interface_packages/alu_in_pkg/src/**alu_in_monitor_bfm.sv**

```
task do_monitor(output alu_in_monitor_s alu_in_monitor_struct);  
  @(posedge clk_i); // Always start with a clock edge  
  
  // Wait for rdy to assert  
  while (rdy_in_i == 1'b0) ←  
    @(posedge clk_i);  
  
  // Capture transaction values from the DUT  
  $cast(alu_in_monitor_struct.opc, opc_i);  
  alu_in_monitor_struct.a = a_i; ←  
  alu_in_monitor_struct.b = b_i;  
  $display("%t alu_in_monitor_bfm:do_monitor() %p",  
    $realtime, alu_in_monitor_struct);  
  
  // Wait for the driver BFM to finish  
  while (rdy_in_i == 1'b1) ←  
    @(posedge clk_i);  
endtask
```



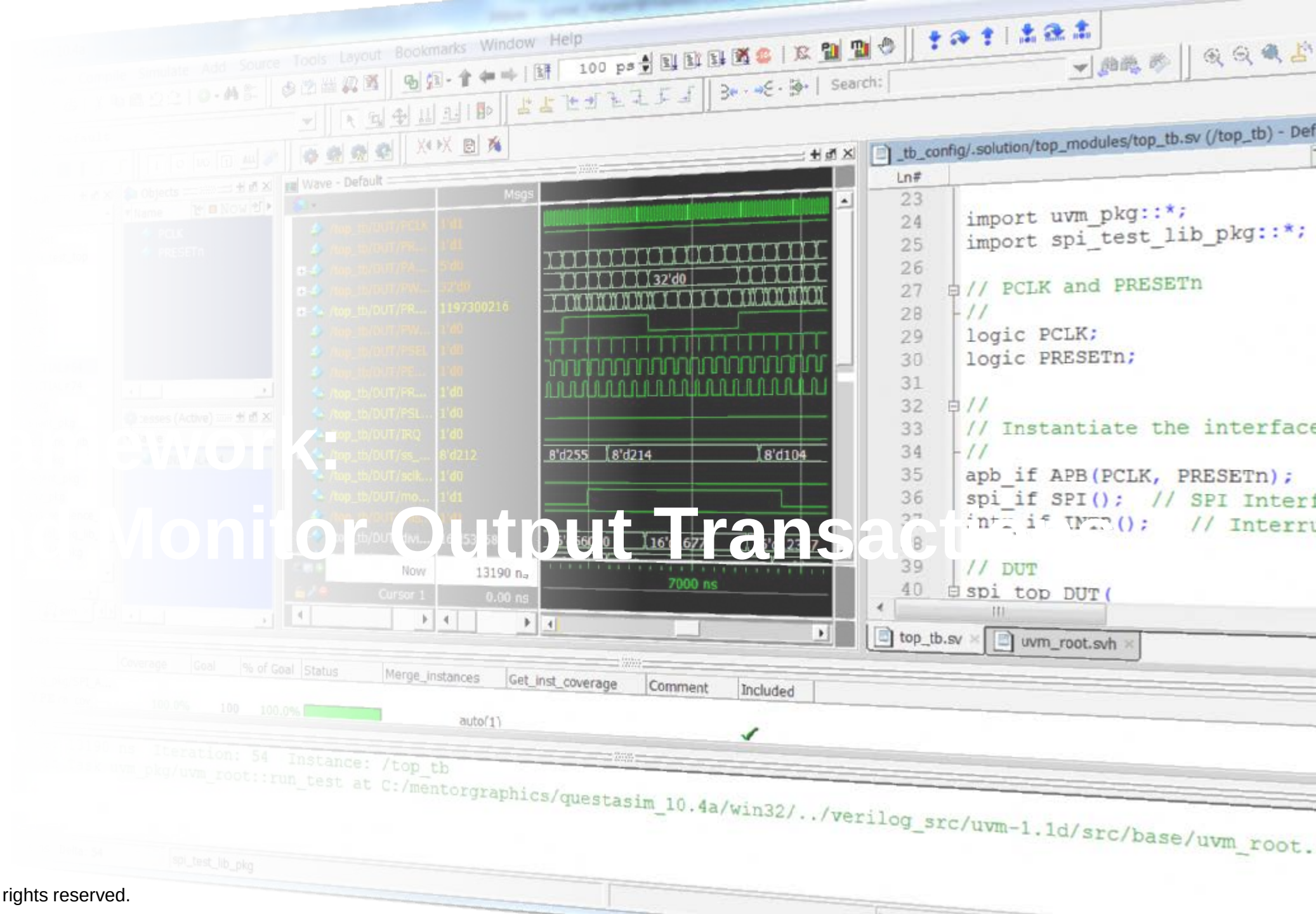
- UVM monitor sends transaction object for analysis

Results

What do the waveforms show?



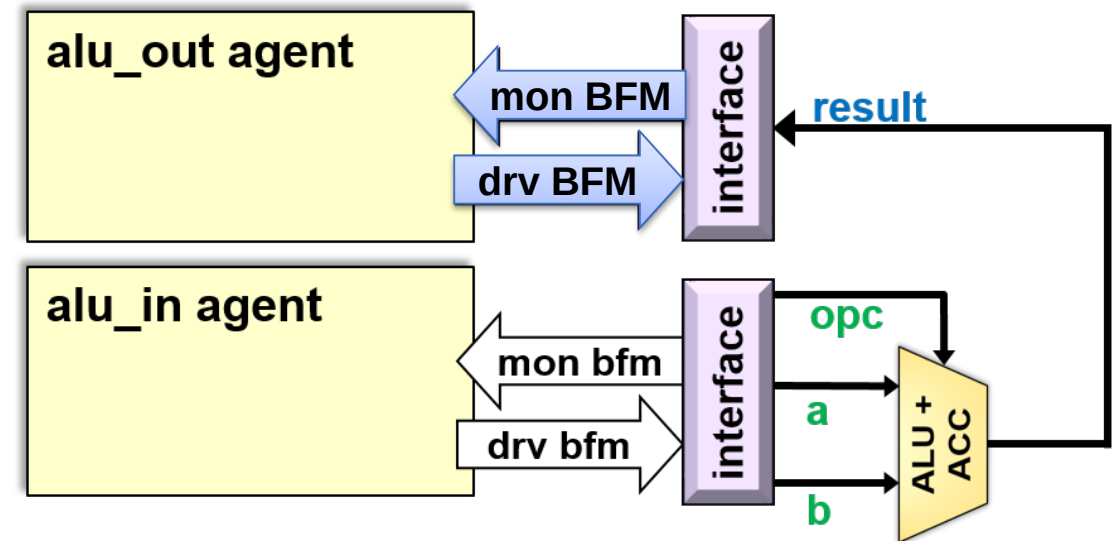
How to Monitor Output Transact



The DUT Output Side

Now update the alu_out BFMs

```
# alu_environment.yaml
uvmf:
  environments:
    alu:
      agents:
        - name: alu_in_agent
          type: alu_in
          initiator_responder: "INITIATOR"
        - name: alu_out_agent
          type: alu_out
          initiator_responder: "RESPONDER"
        ...
```

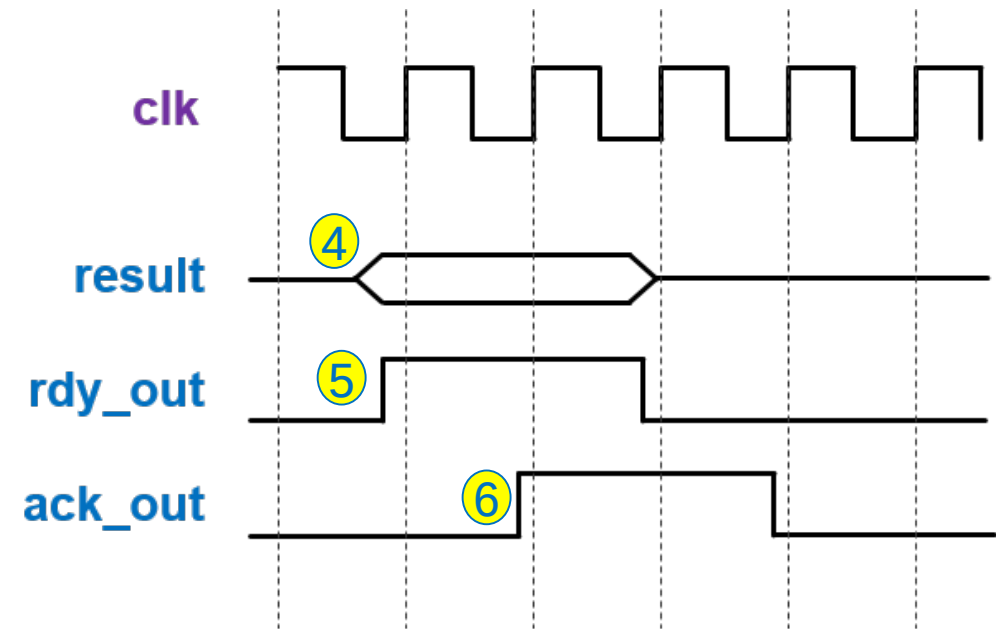


The DUT Output Side

How to respond to the DUT?

verification_ip/interface_packages/alu_out_pkg/src/alu_out_driver_bfm.sv

```
task respond_and_wait_for_next_transfer(  
    output alu_out_initiator_s alu_out_initiator_struct,  
    input alu_out_responder_s alu_out_responder_struct  
);  
    @(posedge clk_i);    // Always start with a clock edge  
  
    // Wait for the result to be ready  
    5 while (rdy_out_i == 1'b0)  
        @(posedge clk_i);  
  
    4 alu_out_initiator_struct.result = result_i;  
    6 ack_out_o <= 1'b1; // Acknowledge the result  
    $display("%t alu_out_driver_bfm:respond_and_wait_for_next_transfer result=0x%x",  
            $realtime, result_i);  
  
    // Wait for the DUT to finish  
    do @(posedge clk_i);  
    while (rdy_out_i == 1'b1);  
  
    ack_out_o <= 1'b0; // Deassert acknowledge  
endtask
```



- Responder waits for DUT to start the transaction
- UVM driver saves **result** in a transaction, sends to the **alu_out** response sequence

The DUT Output Side

verification_ip/interface_packages/alu_out_pkg/src/alu_out_monitor_bfm.sv

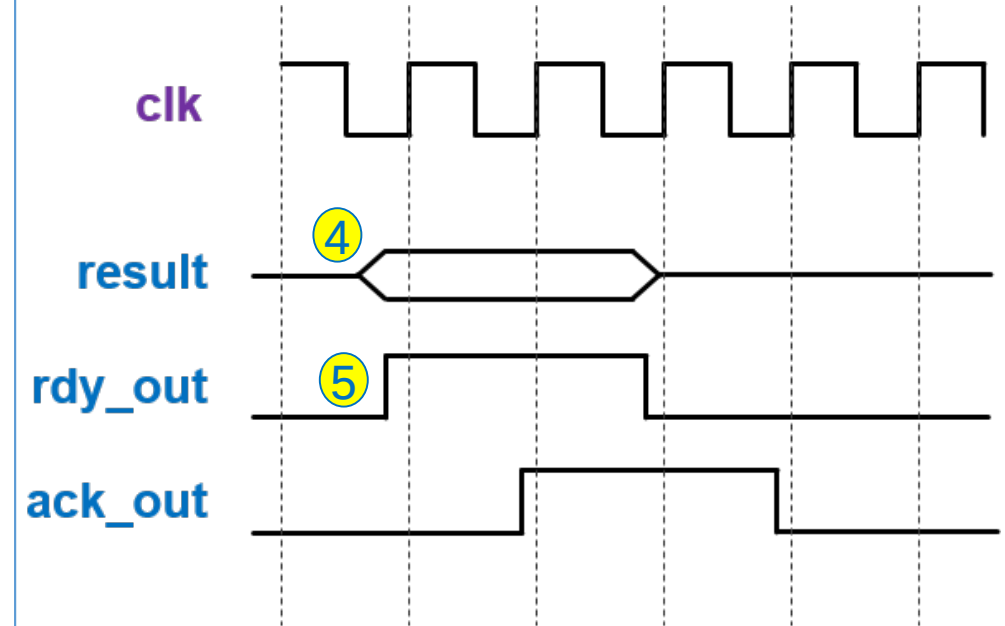
```
task do_monitor(output alu_out_monitor_s alu_out_monitor_struct);
    @(posedge clk_i);    // Always start with a clock edge

    // Wait for the result to be ready
    ⑤ while (rdy_out_i == 1'b0)
        @(posedge clk_i);

    ④ alu_out_monitor_struct.result = result_i; // Sample output result

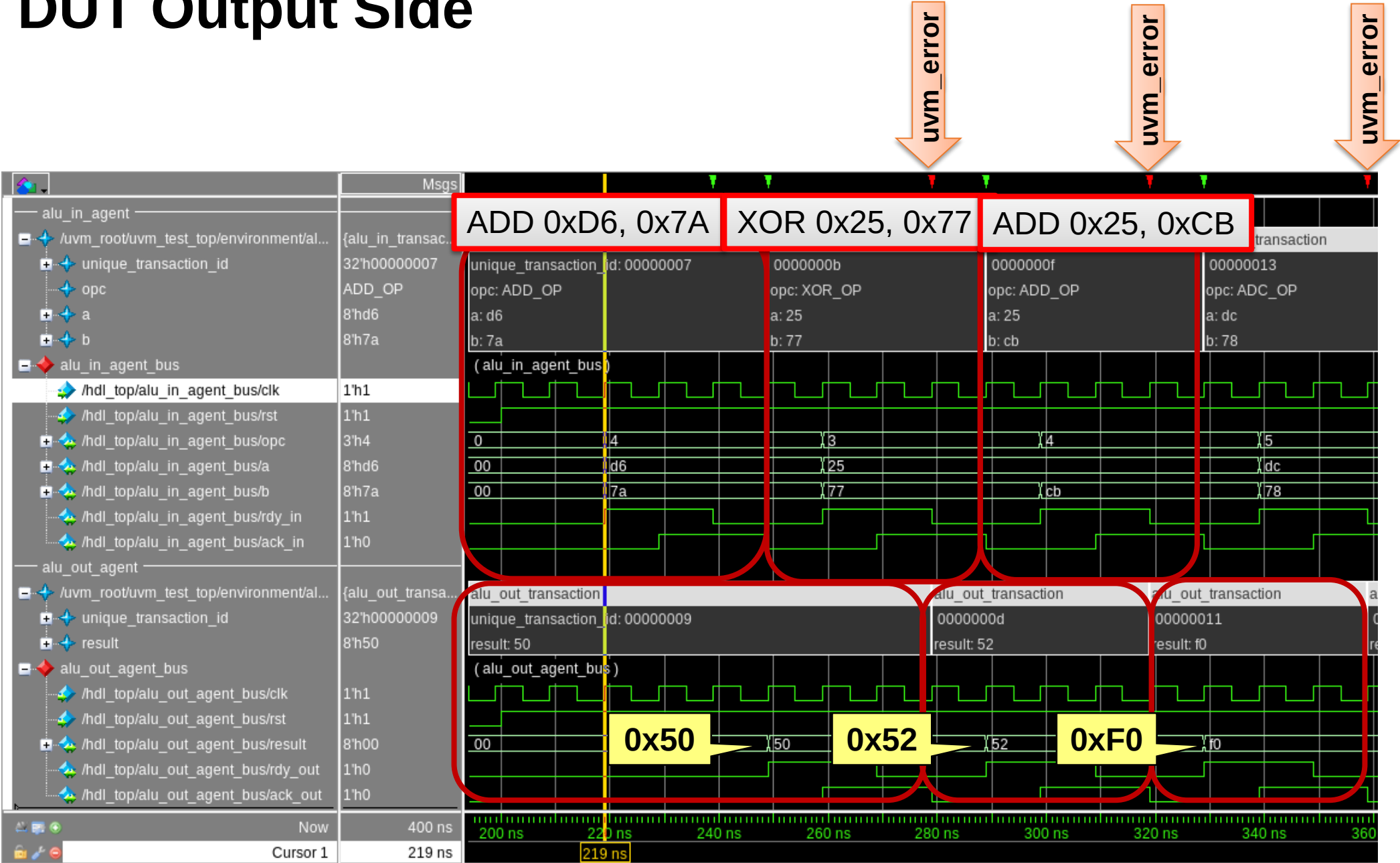
    $display("%t alu_out_monitor_bfm:do_monitor() result=0x%x",
            $realtime, result_i);

    // Wait for the DUT to finish
    do @(posedge clk_i);
    while (rdy_out_i == 1'b1);
endtask
```



- **result** is put inside a transaction object, sent by monitor to scoreboard
- Flag any unexpected X/Z with **\$isunknown()** and **\$error()**
 - Q: Why not call **`uvm_error()** ?
 - A: This is an interface, not a testbench class

The DUT Output Side



Lab 5 – Drive and Monitor Transactions



Goal: Drive and Monitor transactions in and out of the DUT

- Edit **alu_in_driver_bfm.sv** and **alu_in_monitor_bfm.sv** and run simulation
- Edit **alu_out_driver_bfm.sv** and **alu_out_monitor_bfm.sv** and run simulation
- Observe the transactions flowing into and out of the DUT

Check if Test Passed

Simulate with ALU and BFMs for both protocols

- Check the transcript's end

```
# UVM_INFO @ 5199.000ns: uvm_test_top.environment.alu_sb [SCBD] SCOREBOARD_RESULTS:  PREDICTED_TRANSACTIONS=25 MATCHES=7 MISMATCHES=18
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 138
# UVM_WARNING : 0
# UVM_ERROR : 18
# UVM_FATAL : 0
# ** Report counts by id
# [PRED] 25
# [Questa UVM] 2
# [RNTST] 1
# [SCBD] 26
# [SEQ] 25
# [TEST_DONE] 1
# [UNIMPLEMENTED_PREDICTOR_MODEL] 75
```

Mismatches!

Look for errors and fatals

No predictor!

End of Transcript

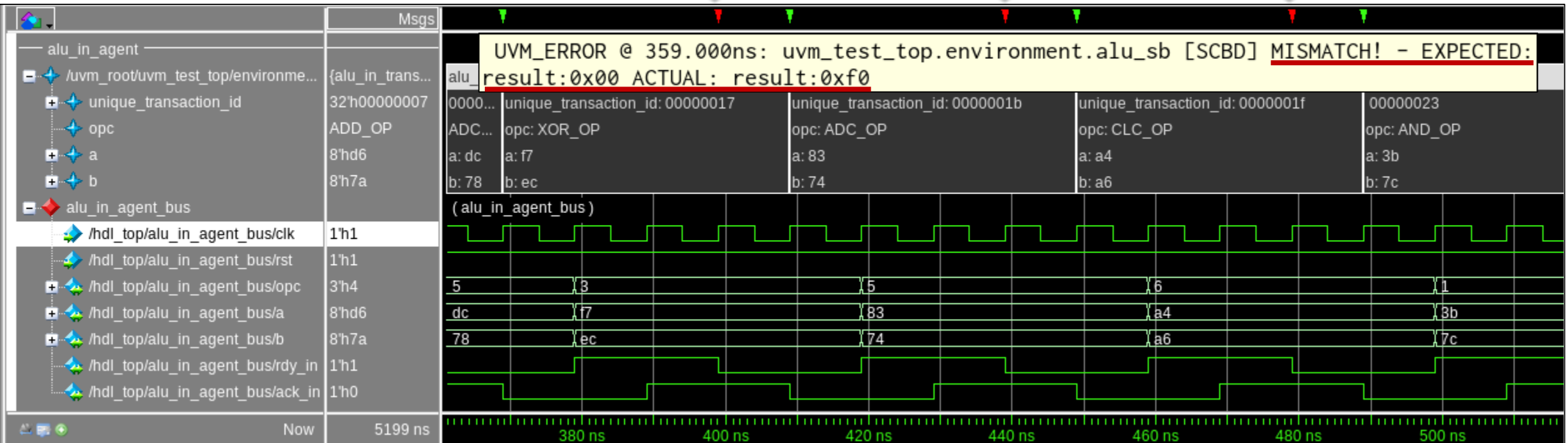
Where are the UVM Errors?

- Check the markers at the top of the waveform display

uvm_error

uvm_error

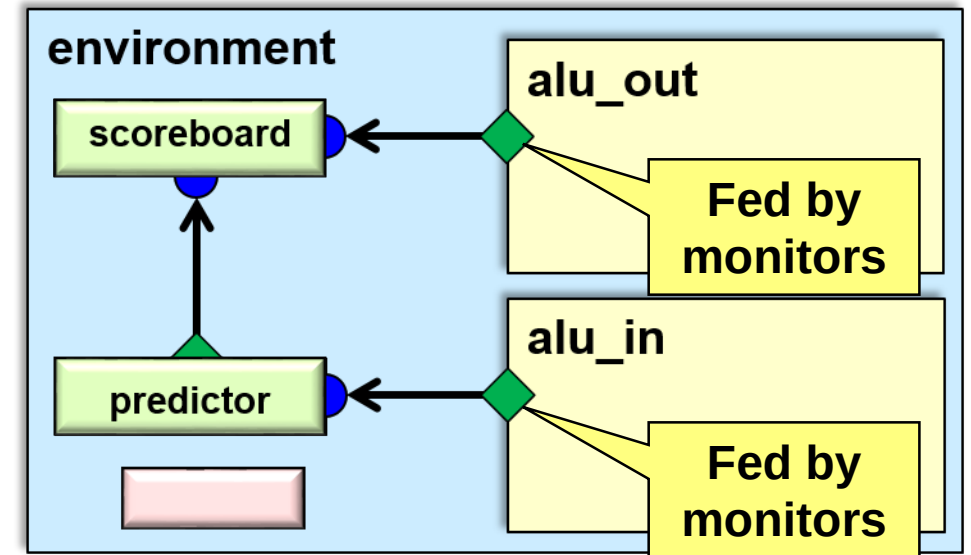
uvm_error



Review: Predicting the Response in UVM

How does the testbench calculate the result?

- In environment, agent(s) send inputs to predictor
- It turns input transactions into expected outputs
 - Reference model: SystemVerilog, C, MatLab, ...
- Scoreboard compares expected vs. actual
 - Prewritten UVMF classes
 - In order, out of order, and more
 - SB & Predictors: multiple inputs, hierarchical



Predictor: write*ae()

How to turn input xact into expect?



- Write function at the end of TLM analysis imp export:
write_alu_in_agent_ae(alu_in_transaction t)

```
// FUNCTION: write_alu_in_agent_ae
// Transactions received through alu_in_agent_ae initiate the execution of this function.
// This function performs prediction of DUT output values based on DUT input, configuration and state
virtual function void write_alu_in_agent_ae(alu_in_transaction t);
    `uvm_info("COV", "Transaction Received through alu_in_agent_ae", UVM_MEDIUM)
    `uvm_info("COV", {"Data: ", t.convert2string()}, UVM_FULL)

// Construct one of each output transaction type.
alu_sb_ap_output_transaction = alu_sb_ap_output_transaction_t::type_id::create("alu_sb_ap_output_transaction");
```

```
// UVMF_CHANGE_ME: Implement predictor model here.
`uvm_info("UNIMPLEMENTED_PREDICTOR_MODEL", "*****", UVM_NONE)
`uvm_info("UNIMPLEMENTED_PREDICTOR_MODEL", "UVMF_CHANGE_ME: The alu_predictor::write_alu_in_agent_ae function needs to be completed with DUT prediction model", UVM_NONE)
`uvm_info("UNIMPLEMENTED_PREDICTOR_MODEL", "*****", UVM_NONE)
```

```
// Code for sending output transaction out through alu_sb_ap
alu_sb_ap.write(alu_sb_ap_output_transaction);
endfunction
```

```
# alu_predictor.yaml
uvmf:
  util_components:
    alu_predictor:
      type: predictor
      analysis_exports:
        - name: alu_in_agent_ae
          type: "alu_in_transaction"
      analysis_ports:
        - name: alu_sb_ap
          type: "alu_out_transaction"
```

- Predictor output: TLM analysis port
alu_sb_ap.write(alu_sb_ap_output_transaction); // Expected result

Predicting the Response

Implementing the predictor

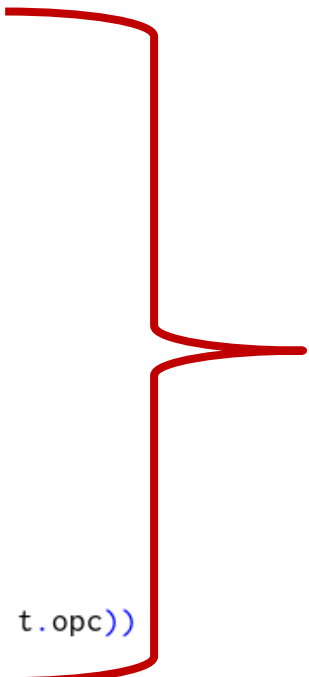


```
virtual function void write_alu_in_agent_ae(alu_in_transaction t);
static bit [7:0] acc; // Accumulator
`uvm_info("PRED", "Transaction Received through alu_in_agent_ae", UVM_MEDIUM)
`uvm_info("PRED", {"Data: ", t.convert2string()}, UVM_FULL)

// Construct one of each output transaction type.
alu_sb_ap_output_transaction = alu_sb_ap_output_transaction_t::type_id::create("alu_sb_ap_output_transaction");

// UVMF_CHANGE_DONE: Implement predictor model here.
case (t.opc)
  NO_OP: alu_sb_ap_output_transaction.result = '0;
  AND_OP: alu_sb_ap_output_transaction.result = t.a & t.b;
  OR_OP: alu_sb_ap_output_transaction.result = t.a | t.b;
  XOR_OP: alu_sb_ap_output_transaction.result = t.a ^ t.b;
  ADD_OP: alu_sb_ap_output_transaction.result = t.a + t.b;
  ADC_OP:
    begin
      acc = t.a + acc;
      alu_sb_ap_output_transaction.result = acc;
    end
  CLC_OP:
    begin
      acc = 0;
      alu_sb_ap_output_transaction.result = 0;
    end
  default:
    `uvm_error("UNK_OP", $sformatf("Received an unknown opcode %b", t.opc))
endcase

// Code for sending output transaction out through alu_sb_ap
alu_sb_ap.write(alu_sb_ap_output_transaction);
endfunction
```



Reference Model

Predicting the Response

Simulation results with reference model

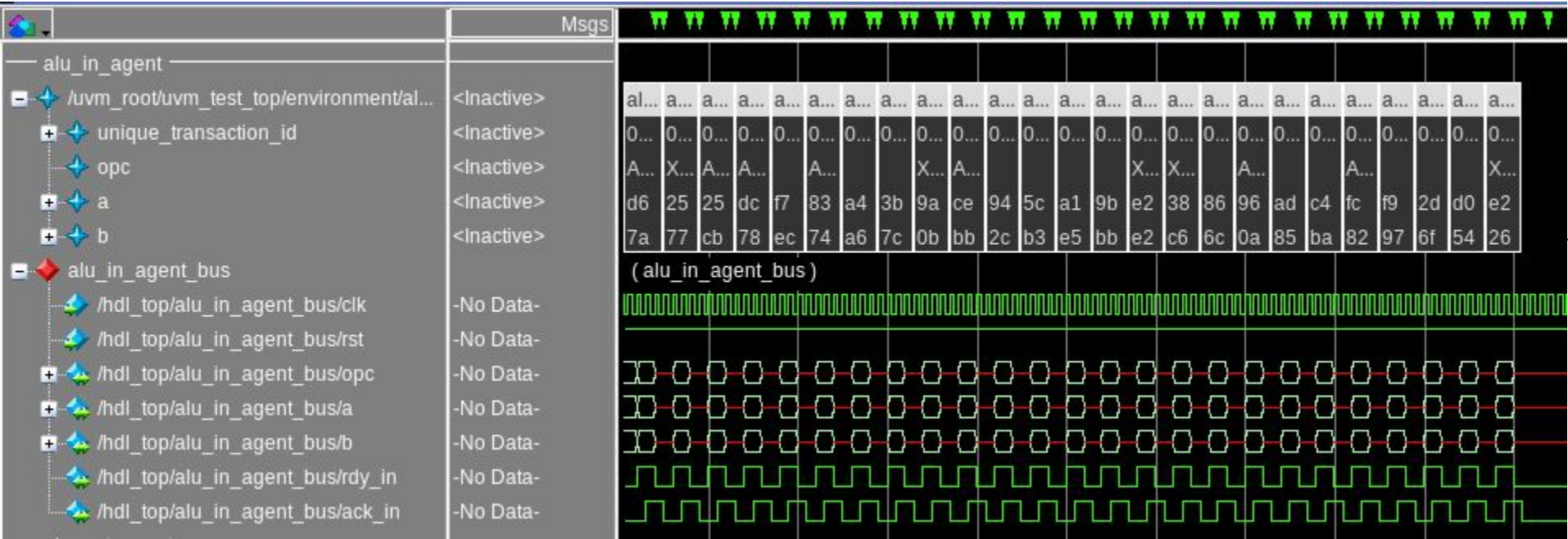
- No other changes needed

```
# UVM_INFO @ 5199.000ns: uvm_test_top.environment.alu_sb [SCBD] SCOREBOARD_RESULTS:  PREDICTED_TRANSACTIONS=25 MATCHES=25 MISMATCHES=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    81
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [PRED]      25
# [Questa UVM]  2
# [RNTST]      1
# [SCBD]      26
# [SEQ]       25
# [TEST_DONE]  1
```

No errors

Success!

No mismatches



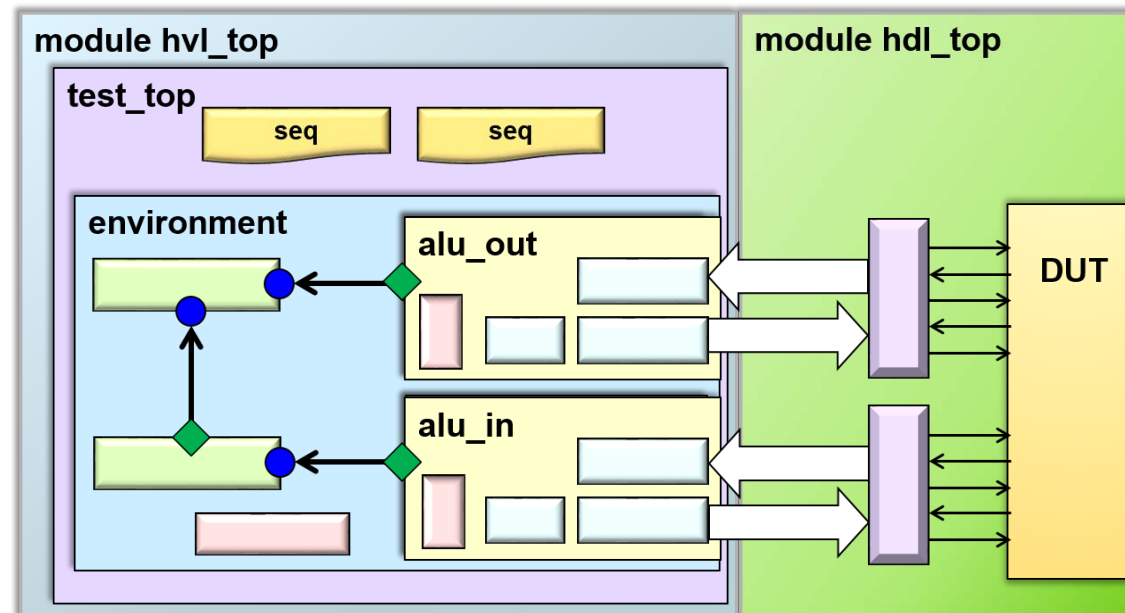
Complete Testbench Created

UVM Framework steps

1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*

What has been accomplished?

- Reviewed DUT
- Wrote input & output protocol YAML, then generated
- Wrote predictor & environment YAML, then generated
- Wrote YAML for bench, then generated
- Added protocol details to driver and monitor BFMs, reference model to predictor
- *UVM: Custom tests, sequences, transactions, configurations*



Knowledge Check – Complete Bench

- For the alu_out interface, who drives and receives the result signal?
- The reference model is called from?

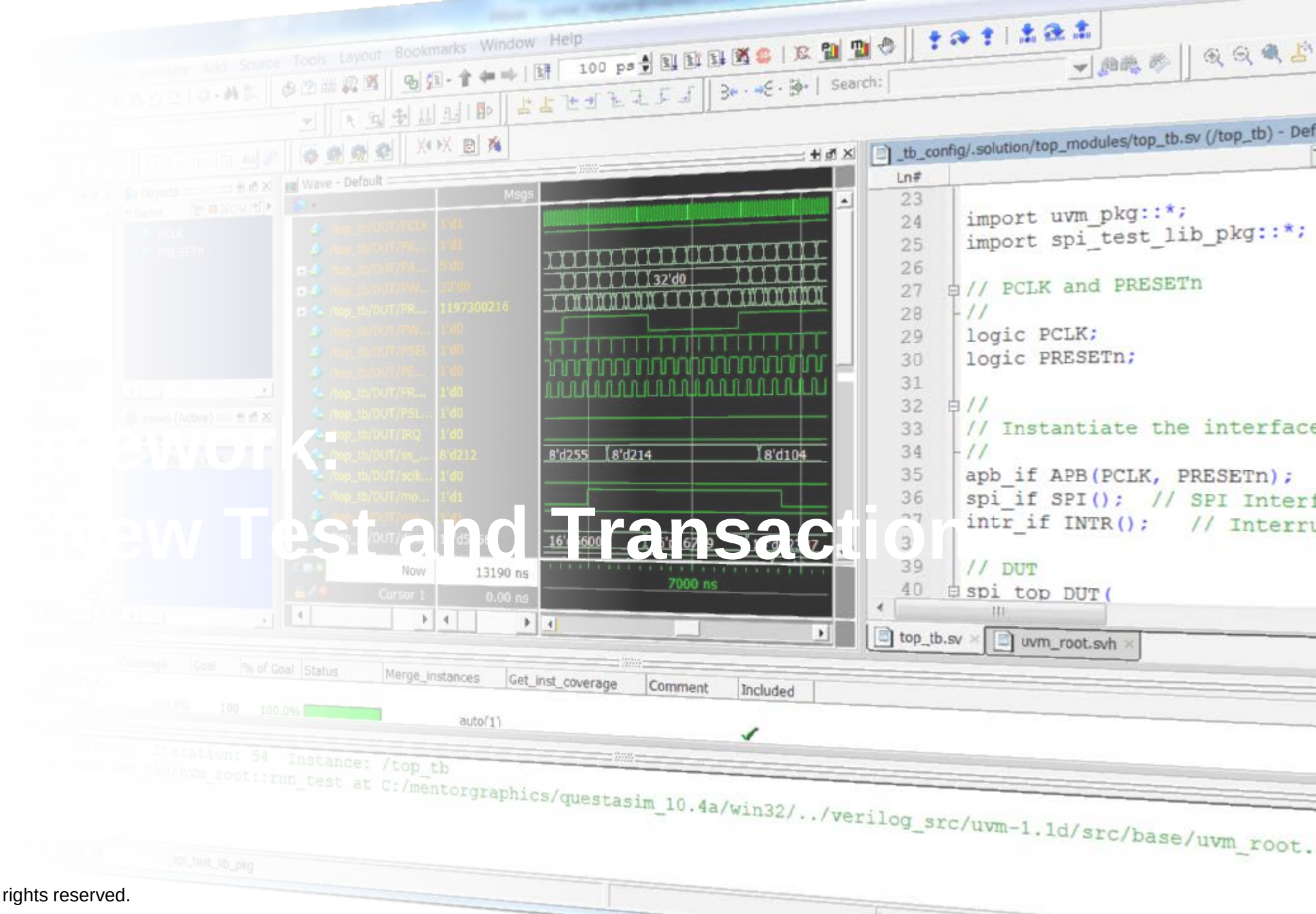
Lab 6 – Predict the Results

Goal: Predict output transactions from input transactions

- Edit `alu_predictor.svh`
- Simulate with a complete UVMF testbench



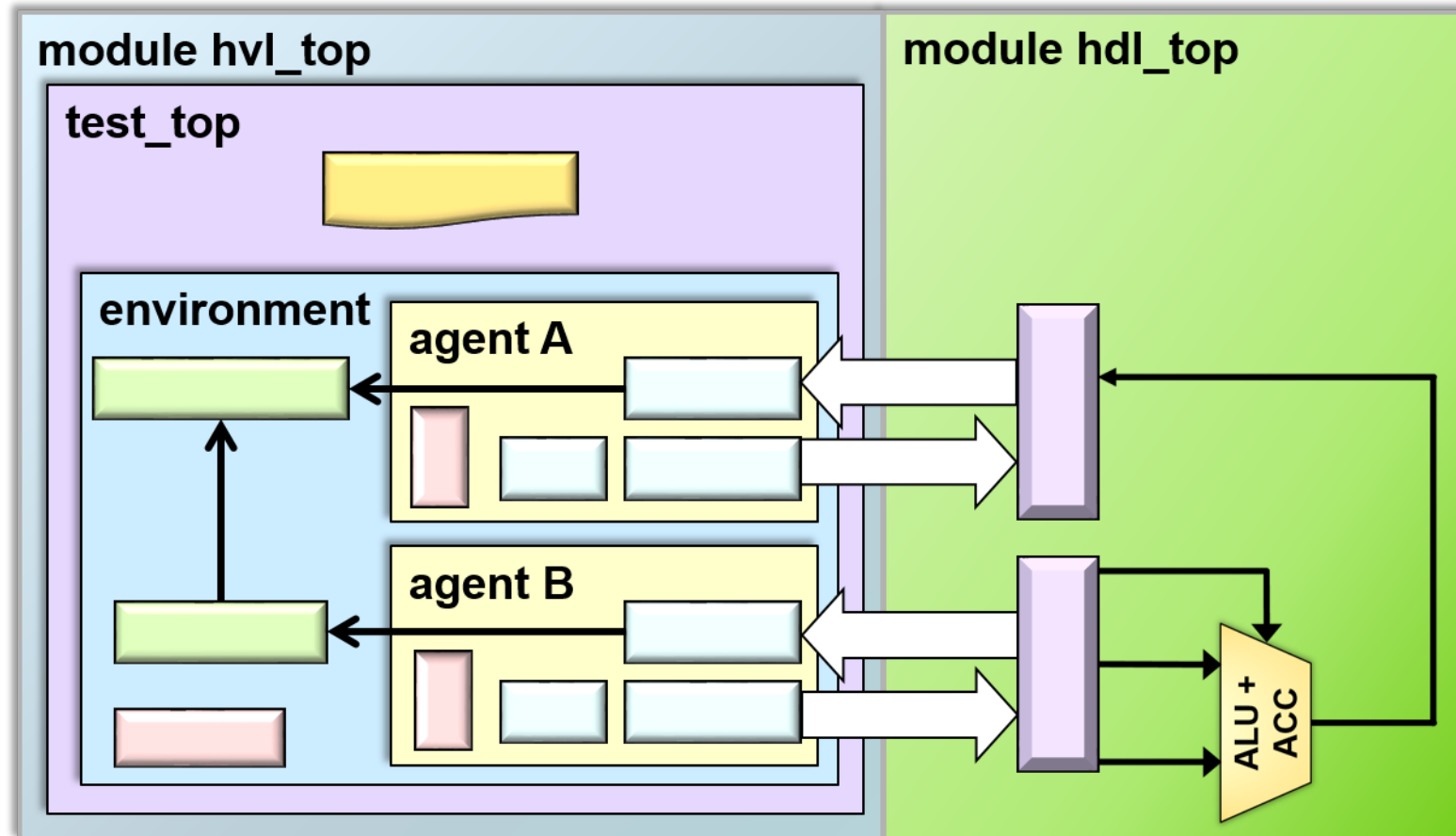
UVM Work: New Test and Transaction



Course Flow

What are the UVMF steps?

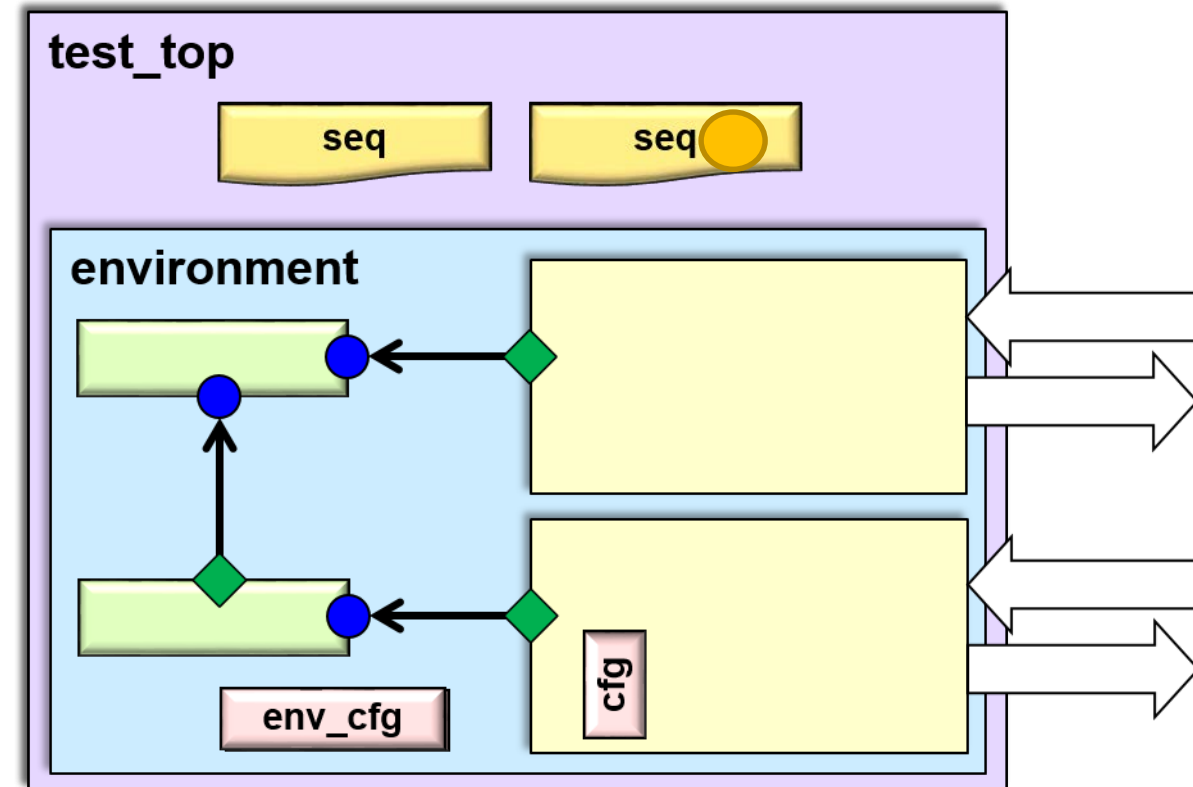
1. *Plan stimulus and checks*
2. *Create the DUT connections*
3. *Create environment*
4. *Create testbench*
5. *Complete testbench*
6. *Start verification of DUT*



Extending the Bench

How to verify design features?

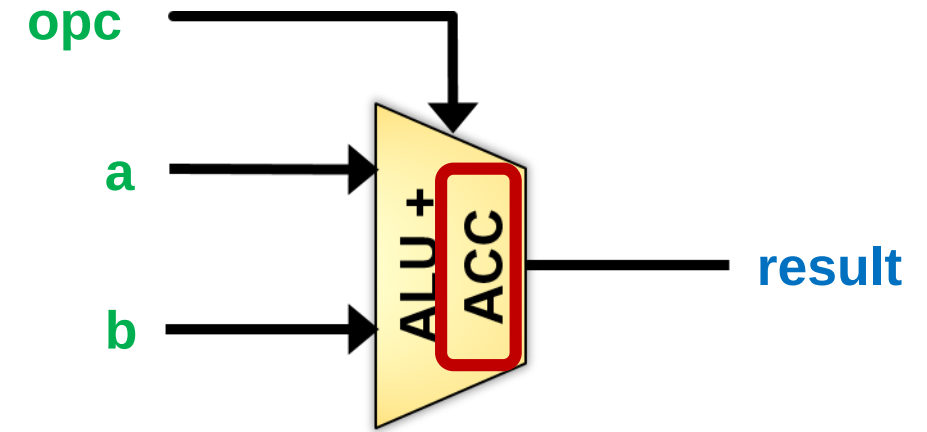
- UVMF generated base classes
 - Sequence items
 - Sequences and virtual sequences
 - Tests
 - Configuration
- Extend and combine these to shape stimulus



Target a Design Feature

How to verify the accumulator?

- Sequential logic needs multiple transactions
 - CLC: Clear Accumulator ($ACC = 0$)
 - ADC: Add Accumulator ($ACC = ACC + A$)
- Problem: Need to verify multiple ADC
- Solution: Generate **more** ADC than CLC



```
constraint accum_dist_c { soft op dist {ADC_OP:=5, CLC_OP:=1}; }
```

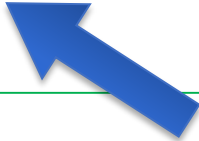
Target a Design Feature

How to add a constraint to the testbench?

verification_ip/interface_packages/alu_in_pkg/src/**alu_in_accum_transaction.svh**

- First: Extend the base transaction to create a new class
 - with the constraint

```
class alu_in_accum_transaction extends alu_in_transaction;  
  
    `uvm_object_utils( alu_in_accum_transaction )  
  
    //Constraints for the transaction variables:  
    constraint accum_dist_c { soft opc dist {ADC_OP:=5, CLC_OP:=1}; }  
  
    function new( string name = "alu_in_accum_transaction" );  
        super.new( name );  
    endfunction  
  
    virtual function string convert2string();  
        return $sformatf("alu_in_accum_transaction: %s",super.convert2string());  
    endfunction  
endclass
```



- And include in the alu_in package

```
package alu_in_pkg;  
    `include "src/alu_in_typedefs.svh"  
    `include "src/alu_in_transaction.svh"  
    `include "src/alu_in_accum_transaction.svh"
```



Target a Design Feature

How to inject the new transaction?

project_benches/alu/tb/tests/src/alu_accum_test.svh

- Next: Extend the base test to create a new class
 - Override the base transaction with the extended one

```
class alu_accum_test extends test_top;
  `uvm_component_utils( alu_accum_test );
  function new( string name, uvm_component parent);
    super.new( name, parent );
  endfunction

  virtual function void build_phase(uvm_phase phase);
    alu_in_transaction::type_id::set_type_override(alu_in_accum_transaction::get_type());
    super.build_phase(phase);
  endfunction

endclass
```

- And include inside the ALU test package

```
package alu_tests_pkg;
  `include "src/test_top.svh"
  `include "src/example_derived_test.svh"
  `include "src/alu_accum_test.svh"
endpackage
```



Simulation Results

% make debug TEST_NAME=alu_accum_test

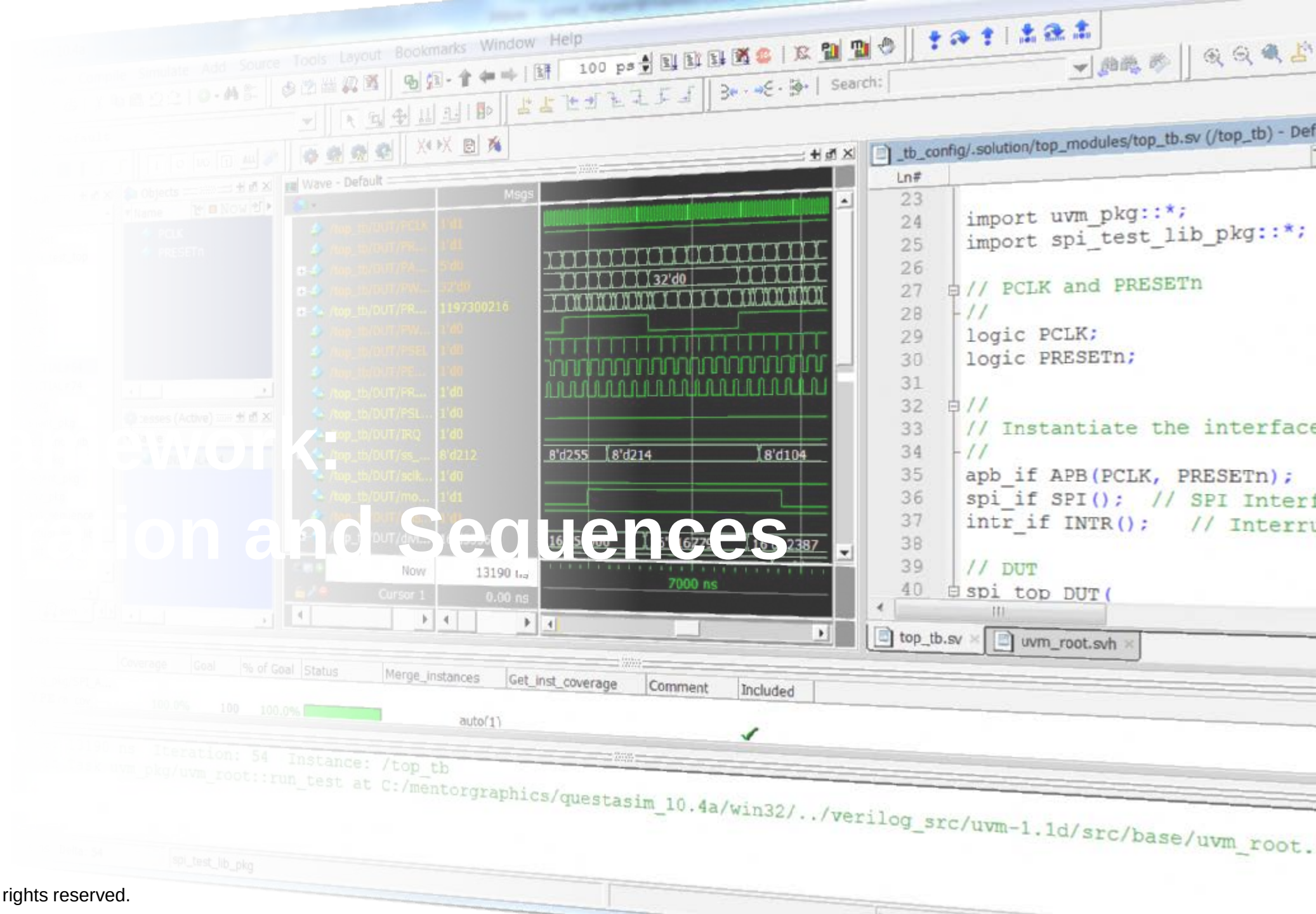
```
# UVM_INFO @ 5199.000ns: uvm_test_top.environment.alu_sb [SCBD] SCOREBOARD_RESULTS:
PREDICTED_TRANSACTIONS=25 MATCHES=25 MISMATCHES=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :      81
# UVM_WARNING :    0
# UVM_ERROR :     0
# UVM_FATAL :     0
```

- 19 ADC transactions
- 6 CLC transactions

/uvm_root/uvm_test_top/e...	{alu_in_...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...	alu_i...
+ unique_transaction_id	32'h00...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...	0000...
+ opc	ADC_OP	ADC...	ADC...	CLC...	ADC...	ADC...	ADC...	ADC...	ADC...	CLC...	ADC...	ADC...	ADC...	ADC...	CLC...	CLC...	ADC...	ADC...
+ a	8'h66	a: 66	a: ff	a: 12	a: 66	a: 64	a: 04	a: d7	a: 1c	a: aa	a: 73	a: 4e	a: 7f	a: ee	a: 73	a: c2	a: e2	a: 43
+ b	8'h70	b: 70	b: 07	b: 90	b: 1f	b: 81	b: 06	b: 5f	b: b2	b: 1d	b: fd	b: 42	b: 87	b: 53	b: b2	b: e7	b: ab	b: ae



UVM Work: Sequences and Sequences



Improved Testbench Control

How to gain more control of the test?

`project_benches/alu/tb/sequences/src/alu_bench_sequence.svh`

```
class alu_bench_sequence_base extends uvmf_sequence_base #(uvm_sequence_item);  
virtual task body();  
    // Construct sequences here  
    alu_in_agent_random_seq      = alu_in_agent_random_seq_t::type_id::create("alu_in_agent_random_seq");  
    alu_out_agent_responder_seq  = alu_out_agent_responder_seq_t::type_id::create("alu_out_agent_responder_seq");  
    fork  
        alu_in_agent_config.wait_for_reset();  
        alu_out_agent_config.wait_for_reset();  
    join  
    // Start RESPONDER sequences here  
    fork  
        alu_out_agent_responder_seq.start(alu_out_agent_sequencer);  
    join_none  
    // Start INITIATOR sequences here  
    fork  
        repeat (25) alu_in_agent_random_seq.start(alu_in_agent_sequencer);  
    join  
    // UVMF_CHANGE_ME : Extend the simulation XXX number of clocks after  
    // the last sequence to allow for the last sequence item to flow  
    // through the design.  
    fork  
        alu_in_agent_config.wait_for_num_clocks(400);  
        alu_out_agent_config.wait_for_num_clocks(400);  
    join  
endtask
```

Always send 25 transactions

Always wait 400 clock cycles

Make these configurable!

Environment Configuration

How is the configuration organized?

- Group config variables into config objects: environment & agent
- Environment config object has handles to agent config
- Modify generated file? Put under revision control.

`verification_ip/environment_packages/alu_env_pkg/src/alu_env_configuration.svh`

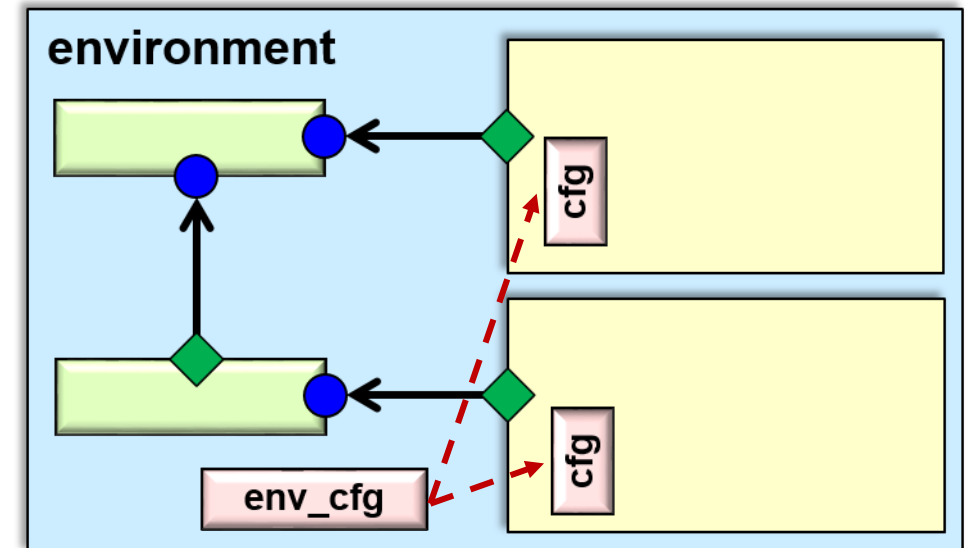
```
class alu_env_configuration
    extends uvmf_environment_configuration_base;
    `uvm_object_utils( alu_env_configuration );

    typedef alu_in_configuration alu_in_agent_config_t;
    alu_in_agent_config_t alu_in_agent_config;

    typedef alu_out_configuration alu_out_agent_config_t;
    alu_out_agent_config_t alu_out_agent_config;

    rand bit [15:0] drain_cycles; // Drain time at end of test
    constraint c_drain_default {soft drain_cycles == 400; }
```

Default value



Agent Configuration

How to add new control to an agent configuration?

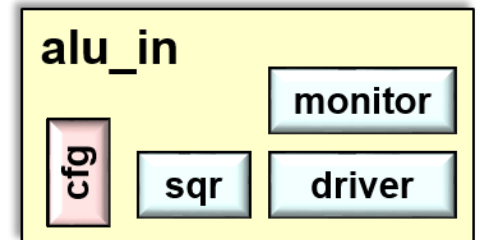
`verification_ip/interface_packages/alu_in_pkg/src/alu_in_configuration.svh`

- Add number of transactions

```
class alu_in_configuration extends uvmf_parameterized_agent_configuration_base #(
    .DRIVER_BFM_BIND_T(virtual alu_in_driver_bfm ),
    .MONITOR_BFM_BIND_T( virtual alu_in_monitor_bfm ));
`uvm_object_utils( alu_in_configuration )

//Constraints for the configuration variables:
rand bit [15:0] transactions; // Number of transactions to generate
constraint c_xact_default {soft transactions == 25; }
```

Default value



- Or, put in env config
- The alu_out agent is a responder – no transaction count needed

Reading the Configuration Variables

Configurable virtual sequence

project_benches/alu/tb/sequences/src/alu_cfg_bench_sequence.svh

```
virtual task body();
  // Construct sequences here
  alu_in_agent_random_seq      = alu_in_agent_random_seq_t::type_id::create("alu_in_agent_random_seq");
  alu_out_agent_responder_seq  = alu_out_agent_responder_seq_t::type_id::create("alu_out_agent_responder_seq");
  fork
    alu_in_agent_config.wait_for_reset();
    alu_out_agent_config.wait_for_reset();
  join
  // Start RESPONDER sequences here
  fork
    alu_out_agent_responder_seq.start(alu_out_agent_sequencer);
  join_none
  // Start INITIATOR sequences here
  fork
    repeat (alu_in_agent_config.transactions)
      alu_in_agent_random_seq.start(alu_in_agent_sequencer);
  join
  // UVMF_CHANGE_ME : Extend the simulation XXX number of clocks after
  // the last sequence to allow for the last sequence item to flow
  // through the design.
  fork
    alu_in_agent_config.wait_for_num_clocks(top_configuration.drain_cycles);
    alu_out_agent_config.wait_for_num_clocks(top_configuration.drain_cycles);
  join
endtask
```

Agent config has
of transactions

Env config
has drain time

Setting the Configuration Variables

How does a test control configuration?

`project_benches/alu/tb/tests/src/alu_accum_test.svh`

```
class alu_accum_test extends test_top;
  `uvm_component_utils( alu_accum_test );
  function new( string name = "", uvm_component parent = null );
    super.new( name ,parent );
  endfunction

  virtual function void build_phase(uvm_phase phase);
    // Factory overrides
    alu_bench_sequence_base::type_id::set_type_override(alu_cfg_bench_sequence::get_type());
    alu_in_transaction::type_id::set_type_override(alu_in_accum_transaction::get_type());

    super.build_phase(phase); // Create & init config objects

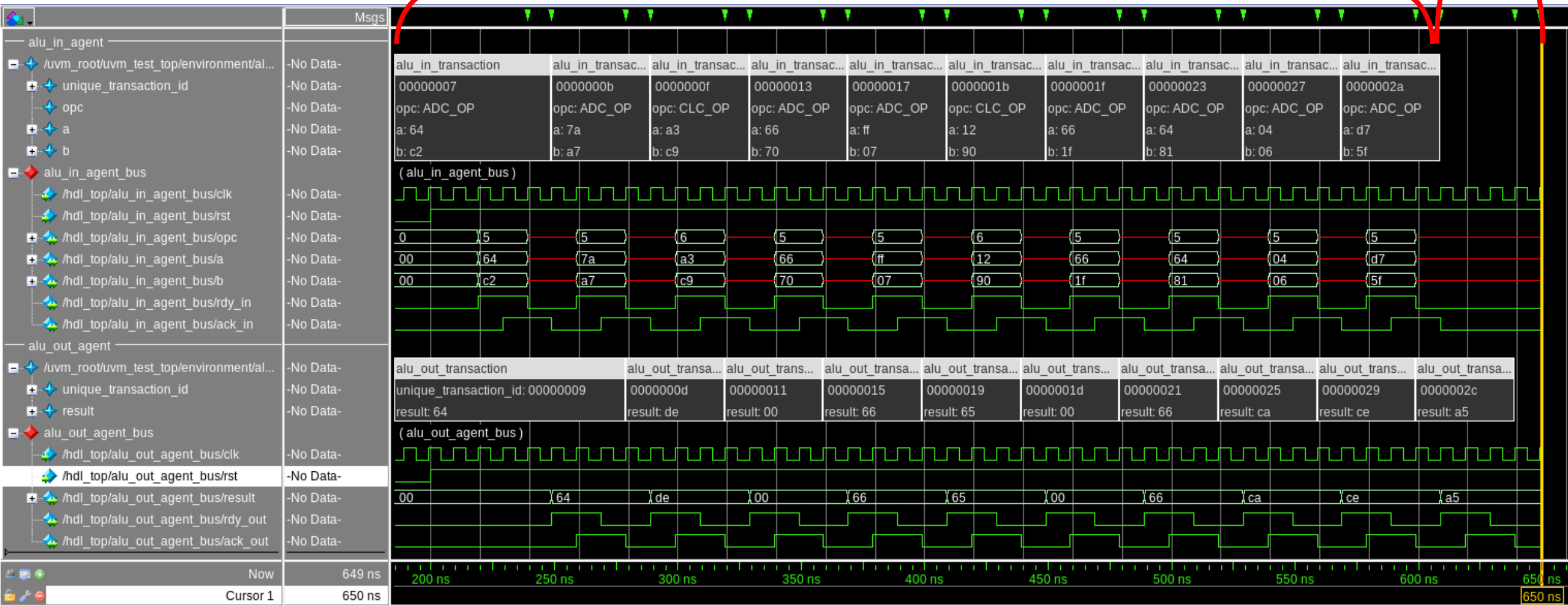
    // Update config values
    configuration.drain_cycles = 5;
    configuration.alu_in_agent_config.transactions = 10;
  endfunction
endclass
```


Running the New Test

What is the (shortened) result?

10 xact

5 cycles



Congratulations!

6. Start verification of DUT



- Yadda

Knowledge Check – Start Verification



- What steps should you take to create new stimulus?
- When you define a new class in UVMF, what else do you always need to do?



Lab 7 – Start Verification of DUT



- 1: Generate specific transactions to verify the accumulator
 - Create **alu_in_accum_transaction.svh** with new constraints
 - Create **alu_accum_test.svh** that injects the new transaction
 - Simulate the new test and observe the new transactions

- 2: Control the test length and drain time through the configuration
 - Edit **alu_in_configuration.svh** to control the number of transactions
 - Edit **alu_env_configuration.svh** to control the drain time
 - Create **alu_cfg_bench_sequence.svh** to switch to new controls
 - Update **alu_accum_test.svh** to run the new virtual sequence and set controls
 - Simulate the new test and observe the new transactions